

基于保守可行域与连续覆盖判据的无线电干扰源自动定位与清除

摘要

无线电干扰源的自动定位同时受到测向误差，接收半径不确定，频道未知和定向发射等因素影响。题面只给出误差界与半径范围，不能把一次测量当作服从某个已知概率分布。本文把定位，发现，清除和路径安排分成相互约束的四层，并用解析判据，保守集合和离线回归分别核对安全性与效率。

针对问题一：把每次测向结果表示为闭角楔，将多次观测，目标圆域和接收半径上界相交得到保守可行域。对满足题面约束的合成实例，定位域有 5 个顶点，直径为 44.434748 m，最小覆盖圆半径为 22.662939 m；有一个顶点超出直径圆 4.507831 m，因此不能用直径的一半直接作为清除依据。

针对问题二：先将首测位置域平移并旋转 to 标准坐标，再求所有可能源均能收到信号的第二点区域，并加入连续最坏交会角不低于 30° 的条件。40 m 网格得到 28 个可行点，选定点为 (760.000, -626.179)m，最大接收距离为 986.333 m，连续最坏交会角为 38.667545° 。

针对问题三：用半径 998.924638 m 的 7 点正多边形环覆盖目标圆域，解析计算表明 6 点同族布局的最坏距离为 1039.230485 m，无法满足 1000 m 下限。结合保守定位，保证清除区域和 19.11 m 三角格兜底，50 局独立离线留出实验全部清除，平均每源时间为 297.205 s，相对已优化基线平均降低 4.1550%，区间为 2.7846% 至 5.3846%。三次正式测试共清除 38 个源，平均定位清除时间为 281.093 s/个，结果与原始加密日志及结果元数据逐一对应。

针对问题四：用原点，半径 996 m 的 8 点内环和半径 1863.756 m 的 12 点外环组成 21 点扫描集，在给定的保守几何检验设置下，验证其对任意定向半平面都能保留正向接收点。55 局重放和另一组 24 局共享误差压力集均全部清除，平均降时分别为 1.157% 和 1.126%。三次正式测试共清除 33 个源，平均定位清除时间为 583.260 s/个，结果与原始加密日志及结果元数据逐一对应。

关键词：有界误差定位；连续覆盖；定向源搜索；最小覆盖圆；安全调度

一、问题重述

1.1 问题背景

题目研究圆形区域内无线电干扰源的快速自动定位与清除。机器狗从区域中心出发，通过移动，切换频道和测量逐步获取信息，再在靠近源的位置进行光学定位和激光清除。每个干扰源占用一个频道，且不同干扰源的频道不重复；频道是否存在，源的位置和有效接收半径均需要通过过程中的反馈判断。问题的难点不在于一次测量给出一个坐标，而在于整个过程中不能因为误差或定向发射而把真实源排除。

目标区域为以原点为中心，半径 1800 m 的圆域。源数在 10 至 16 个之间，每个源占用 1 至 20 号频道中的一个不同频道。机器狗最高移动速度为 5 m/s，一次检测耗时 5 s，切换频道另耗时 1 s。正常接收时返回从检测点指向源的示向度，示向误差不超过 1° ，同一地点重复检测不产生新的误差样本。源的有效接收半径属于 1000 m 至 1500 m，且定向源可能在检测点处因发射方向背向而无信号。

当机器狗距源不超过 5 m 且处于有效接收角内时，接口返回距离过近。清除点距目标源不超过 20 m 即成功。按本文的动作计时口径，失败清除耗时 3 s，成功清除耗时 5 s，其中包括 3 s 的光学定位和 2 s 的激光清除。问题三和问题四要求在确保全部干扰源被清除的前提下缩短总虚拟时间，因此本文把完整性作为硬约束，把路径和动作安排作为次级优化目标。

1.2 问题提出

问题一：根据多次测向点和示向度，给出能够包含真实源的定位区域，计算区域直径，并判断以该直径为直径，以直径端点中点为圆心的圆能否覆盖该区域。

问题二：在只有一次有效观测的情况下，选择第二个检测点，使再次接收具有保证，并使两条测向线的交会几何尽量稳定。

问题三：针对全向发射源，设计完整的频道扫描，保守定位，清除和路径调度方案，使任意目标位置都能被发现，并在保证全部清除的条件下减少虚拟时间。

问题四：针对未知发射方向的定向源，设计能够覆盖任意正向半平面的扫描集，处理正负观测共同产生的约束，并给出带安全兜底的在线调度方案。

1.3 总体问题分析

四个问题可看作同一条安全链的逐级扩展。问题一先回答“观测后哪些位置仍可能是真源”，问题二把一次观测后的可能位置域转化为保证再次接收的第二点集合，问题三在全向条件下用连续圆覆盖保证至少得到一次正反馈，问题四再把圆覆盖改成对任意发射方向都有效的正向包围。最后，只有在安全集合和覆盖判据不变的前提下，才使用路线优化缩短时间。总体流程如图 1 所示，图中每一步的输出都成为下一步的约束，而

不是只作为经验性的排序指标。

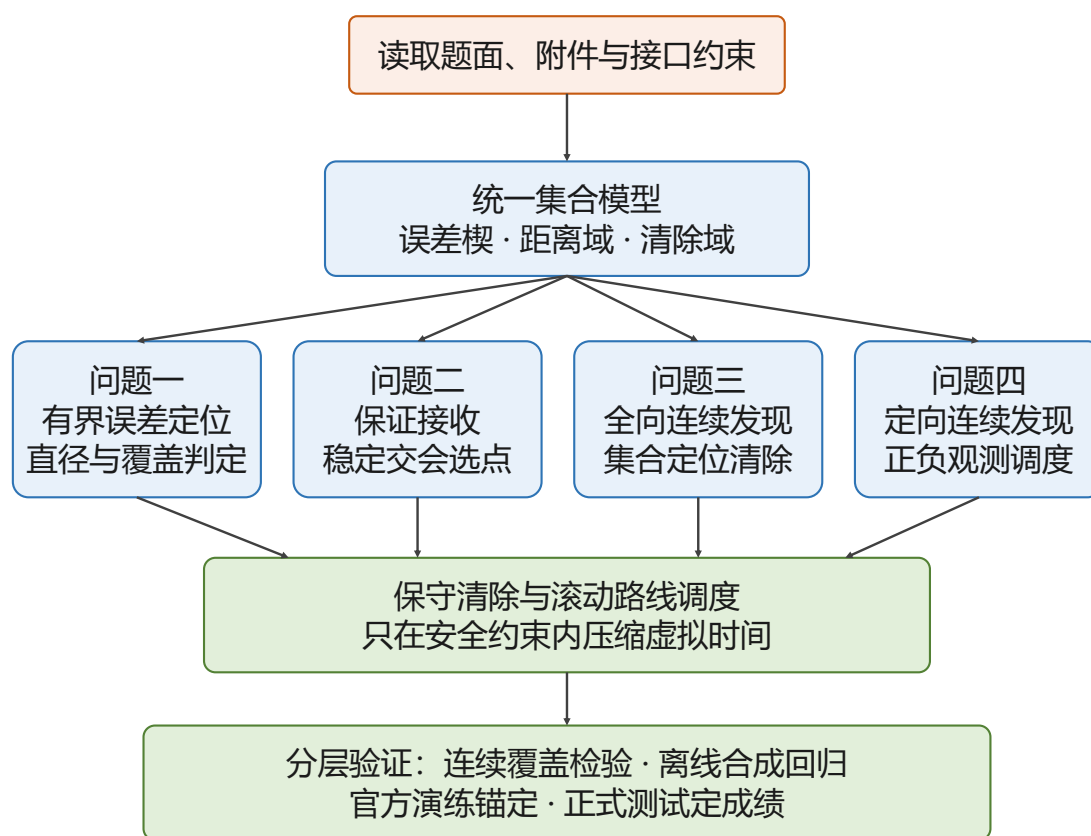


图 1： 总体建模与求解流程

图 1 把本文的判据顺序画成从输入到调度的单向流程。保守定位和连续发现负责防止漏源，保证清除负责防止误报成功，时间调度只在前三层通过后改变访问顺序。

二、问题分析与总体路线

2.1 问题一分析

题面给出的是硬误差界，而不是均值，方差或独立性。若把示向度直接代入最小二乘交会，得到的点在极限误差下可能偏离真实源；若再以概率阈值删除位置，就无法保证清除。因此本文把一次示向度变成由两条半平面组成的闭角楔，把所有尚未被证明不可能的位置保留下来。集合的缩小来自明确的题面约束，不能来自一个未经给出的概率模型。

2.2 问题二分析

问题二只有一次有效示向观测，第二检测点既要进行一次观测得到的全部可能源位置保证接收，又要避免两条测向线接近平行。本文把保证接收作为硬约束，把连续最坏交会角和移动距离作为排序指标，先筛掉没有接收保证的点，再从剩余候选点中选择交会几何较稳定的位置。

2.3 问题三分析

问题三面对全向源，扫描点必须连续覆盖整个目标圆域，频道轮询也必须覆盖全部可能频道。本文先用正多边形环的极小极大距离公式确定七点扫描集，再把保守定位，保证清除和路径调度接入在线流程。覆盖判据先固定安全层，2-opt 和滚动重排只用于减少无效移动与重复动作。

2.4 问题四分析

问题四的定向源可能因发射方向背向而无信号，因此普通圆覆盖不足以保证发现。本文用内环，外环和中心点构成 21 点扫描集，以局部扫描点的凸包包含源位置作为正向包围的充分判据；正负观测只删除能够严格证明不相容的位置单元，有限前瞻和服务点选择只改变访问顺序。

本文的数值证据分为四层。第一层是题目给出的误差，半径和计时约束。第二层是解析式或计算机辅助的充分性检验，回答连续覆盖和完整性问题。第三层是固定种子的离线合成回归，用于比较策略和发现实现错误。第四层是官方接口演练，负责核对通信链路；正式测试必须依据模拟器界面和原始加密日志填写，不能用离线实验代替。

三、模型假设

以下条件用于界定模型适用范围。题面已明确给出的参数不再被包装成经验假设，表 1 只列出它们在模型中的作用及失效后果。

表 1：模型假设及其影响

编号	条件	作用范围	违反后的影响
H1	源位置不变且与机器狗同平面	全文	角楔交会和路程计算失效
H2	示向误差绝对值不超过 1°	全文	真实源可能被排除
H3	接收半径为 1000 至 1500 m	问题二至问题四	保证接收和正负观测失效
H4	接口按题面计时和重试执行	问题三和问题四	时间和完整性无法复算
H5	圆盘用内外多边形夹逼	数值几何	仅给出区间，不作精确值
H6	30° 门槛和时间权重作排序	问题二至问题四	只改候选顺序，不改接收

表 1 中 H2 和 H3 直接决定保守域的边界，H5 决定保守几何检验的误差方向，H6 只参与效率优化。题面规定不同源占用不同频道，因此一个频道最多对应一个未清除源，这一事实在问题三和问题四的频道状态更新中直接使用。题面没有给出路径损耗指数，本文不额外拟合场强函数，只使用接收半径的上下界。

四、符号说明

表 2: 主要符号说明

符号	含义	单位	类型
Ω	半径 $R = 1800\text{ m}$ 的目标圆域	m^2	已知集合
$\mathbf{s}_i$	第 i 个检测点	m	已知向量
$\mathbf{z}$	干扰源位置	m	未知向量
θ_i	第 i 次返回的示向度	$^\circ$	观测量
ε	示向误差半角, 取 1°	$^\circ$	已知参数
ρ	某源有效接收半径	m	未知参数
F	当前观测相容的保守位置域	m^2	状态集合
$D(F)$	可行域直径	m	派生量
$(\mathbf{c}^*, r^*)$	可行域最小覆盖圆圆心与半径	m	派生量
P	扫描点集合	m	决策集合
$\mathbf{u}$	定向源发射方向单位向量	1	未知向量
L	累计移动距离	m	结果量
N_m, N_s	检测次数和切频次数	次	结果量
N_c, N_f	成功和失败清除次数	次	结果量

表 2 统一全文的位置，误差和时间符号。向量用黑体表示，变量和单位在数学环境中保持斜体。问题三和问题四按题面动作计时定义虚拟时间为

$$T = \frac{L}{5} + 5N_m + N_s + 5N_c + 3N_f \quad (4-1)$$

其中移动速度单位为 m/s ，成功和失败清除分别计 5 s 和 3 s。同时记录每源时间 $\bar{T} = T/N_c$ ，完整清除比例和程序运行时间。离线实验用隐藏真值核对完整清除比例；官方演练缺少真源总数时，只报告已清除源数量与每源时间。

五、问题一模型的建立与求解

5.1 问题一分析

问题一的核心不是找一个最可能坐标，而是保留所有符合误差界的位置。本文以闭角楔表示每次示向观测，再与目标圆域和接收半径上界相交。由于相交后的边界可能包含圆弧，几何计算同时给出多边形近似和内外夹逼^[1,8]，最后用顶点对与最小覆盖圆回答直径圆是否足够^[2]。输出核验结果同时包含直径端点，点积判据和最小覆盖圆，三条信息用于相互核对。这样可以避免把一个程序返回的半径数值直接当作清除授权。

5.2 单次观测的闭角楔

令 $\mathbf{u}(\alpha) = (\cos\alpha, \sin\alpha)^\top$ ，二维叉积记为 $cr(\mathbf{a}, \mathbf{b}) = a_x b_y - a_y b_x$ 。第 i 次检测点为 $\mathbf{s}_i$ ，返回示向度为 θ_i ，则真实射线方位属于 $[\theta_i - \varepsilon, \theta_i + \varepsilon]$ ，真源必位于闭角楔

$$S_i = \{\mathbf{x} : cr(\mathbf{u}(\theta_i - \varepsilon), \mathbf{x} - \mathbf{s}_i) \geq 0, \quad cr(\mathbf{u}(\theta_i + \varepsilon), \mathbf{x} - \mathbf{s}_i) \leq 0\} \quad (5-1)$$

式 (5-1) 使用叉积而不是斜率，因而不需要单独处理竖直射线和 0° 至 360° 跨界。多次观测的纯角楔交为

$$P = \bigcap_{i=1}^m S_i \quad (5-2)$$

程序将空集，无界集，点，线段和非退化凸多边形分别标记，不能把空集时返回的数值 0 解释为精确定位。

5.3 距离约束与保守可行域

正常返回示向度意味着源在目标圆域内，且距该检测点不超过实际接收半径。实际半径最大为 1500 m，所以不会排除真源的增强可行域为

$$K = P \cap \Omega \cap \bigcap_{i=1}^m B(\mathbf{s}_i, 1500) \quad (5-3)$$

其中 $B(\mathbf{s}_i, 1500)$ 表示以检测点为圆心的闭圆盘。程序对圆盘使用内接正多边形和外切正多边形，分别得到 $K_N^- \subseteq K \subseteq K_N^+$ 。当直径或覆盖圆半径的上下界宽度超过容差时，将边数加倍。

5.4 直径与直径圆判据

设有界凸多边形的顶点按逆时针排列为 $V = \{v_1, \dots, v_h\}$ ，其直径满足

$$D = \max_{1 \leq i < j \leq h} \|v_i - v_j\|_2 \quad (5-4)$$

任取多边形内两点 $x = \sum_i \lambda_i v_i$ 与 $y = \sum_j \mu_j v_j$ ，由三角不等式有

$$\|x - y\|_2 \leq \sum_{i,j} \lambda_i \mu_j \|v_i - v_j\|_2 \leq D \quad (5-5)$$

因此直径由某一顶点对达到。当前顶点数较少，程序枚举全部顶点对，便于复核；大规模凸多边形可以改用旋转卡壳。

若达到直径的端点为 A 和 B ，中点为 $M = (A + B)/2$ ，以 $D/2$ 为半径的圆若同时包含端点，其圆心只能是 M 。因此所有顶点满足下式时且仅当直径圆覆盖定位域

$$\|v - M\|_2 \leq \frac{D}{2} \iff (v - A) \cdot (v - B) \leq 0 \quad (5-6)$$

程序还独立计算最小覆盖圆 (c^*, r^*) 。若 $r^* > D/2$ ，直径圆不能覆盖整个可行域。

5.5 问题一结果与敏感性分析

题面没有提供问题一的固定坐标，以下数值来自满足题面范围的合成实例，只用于验证算法和回答几何命题。图 2 左侧给出检测点，合成真值和定位域，右侧放大显示直径圆，最小覆盖圆以及外逸见证点。

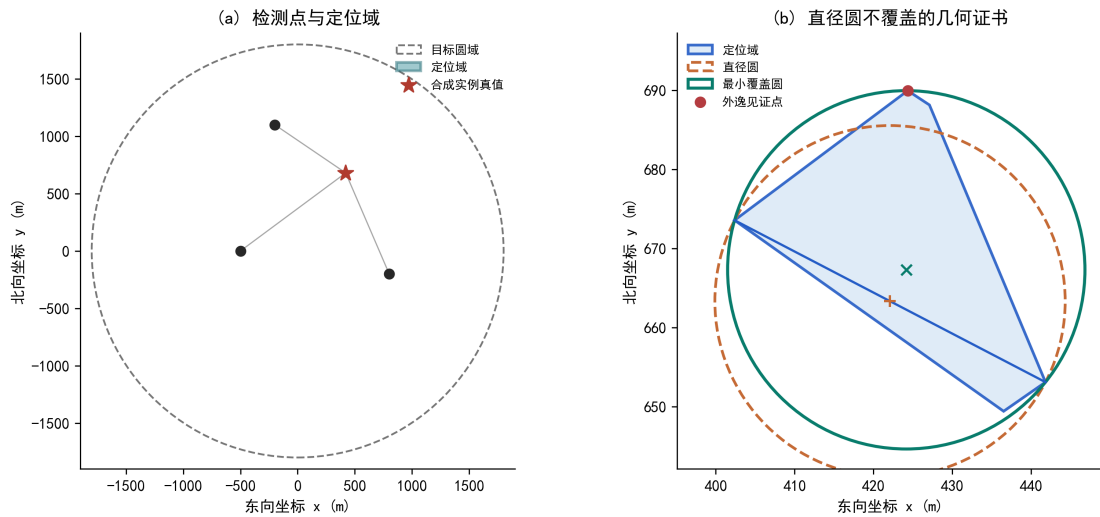


图 2：问题一有界误差定位域及直径圆覆盖判据

图 2 中直径端点之一为 $A = (441.817761, 653.100136)m$ 。另一端点为 $B = (402.382320, 673.576777)m$ ，故 $D = 44.434748m$ 。顶点 $V = (424.343246, 689.969352)m$

到中点的距离为 26.725205 m，比 $D/2 = 22.217374\text{ m}$ 大 4.507831 m。最小覆盖圆半径为 22.662939 m，且 $2r^*/D = 1.020055 > 1$ 。因此该合成实例构成一个合法反例，说明清除授权不能只使用定位域直径的一半。

表 3：问题一角误差界与圆盘离散敏感性

误差半角	圆盘边数	直径/m	最小圆半径/m	真值保留
0.50°	720	14.990647	7.691357	否
1.00°	720	44.434748	22.662939	是
1.01°	720	44.932646	22.903336	是
1.50°	720	69.392586	34.871940	是

表 3 表明，误差界增大时保守域单调扩大，0.50° 会排除合成真值，1° 及以上能够保留真值。另一个圆弧边界压力实例中，圆盘边数由 360,720 增加到 1440 时，直径夹逼宽度分别为 0.087596, 0.021976 和 0.005484 m，约按四分之一缩小。这个结果支持采用内外夹逼，而不是只使用一侧多边形近似。

5.6 夹逼误差与结果使用边界

圆盘边界离散化只是为了便于计算，并不改变可行域的定义。内接多边形位于真实圆盘内，外切多边形包含真实圆盘，所以它们与同一组角楔，目标圆域相交后仍满足

$$K_N^- \subseteq K \subseteq K_N^+, \quad D(K_N^-) \leq D(K) \leq D(K_N^+) \quad (5-7)$$

这个包含关系给出了数值结果的使用边界。内侧结果是可行域的保守下界，外侧结果用于确认没有遗漏可能位置。只有当两侧的直径或最小覆盖圆半径之差小于设定容差时，正文才报告相应的小数值；若区间仍然较宽，程序会增加圆盘边数，不会直接采用其中一侧。

问题一的输出不能脱离后续清除条件使用。直径只描述两个最远可行位置的距离，不能保证以直径中点为圆心的圆包含整个区域；式 (5-6) 才是逐顶点的必要充分判据。真正发送清除指令前，还要把保守定位域压缩到清除半径以内，或者在服务区域内选择并复核清除点。因此，几何计算回答的是“哪些位置仍可能是真源”，并不直接等同于发送清除指令的条件。

这里的 1500 m 是保全信息的上界，而不是对真实距离的估计。正常返回示向度只能说明真实源没有超出其实际接收半径，而该半径可能位于 1000 m 至 1500 m 之间。用 1500 m 构造式 (5-3) 不会误删可能位置；若直接使用 1000 m，反而可能把真实源排除在

定位域外。这个区别也解释了为什么问题一的定位域适合做后续筛选，却不能直接当作清除点。

实际计算先裁剪角楔和目标圆域，再逐次加入圆盘约束，并在每次裁剪后检查顶点数量，面积和有界性。若相邻两次边数加倍得到的直径与最小覆盖圆半径均已稳定，程序才输出小数结果；若出现空集，退化线段或夹逼区间过宽，则只记录状态，不把数值 0 或单侧近似写成定位结论。这样处理可以把几何退化和数值误差从正常结果中区分出来。

六、问题二模型的建立与求解

6.1 问题二分析

问题二只有一次有效测向，第二检测点既要落在所有可能源的保证接收区域内，又要避免两条视线接近平行。本文先标准化首测坐标，再求保证接收集合，随后对连续最坏交会角进行边界分析和网格评估。该几何条件与传感器定位交会角的稳定性分析一致^[3]。计算时先做保证接收和交会角门槛筛选，再做软目标排序。这样即使排序权重改变，第二点仍保留对所有可能源的接收保证。

6.2 一次观测域与坐标标准化

若首检测已经返回距离过近，则可以在首点直接清除，不需要选第二点。对正常示向度情形，设首点为 s_1 ，读数为 θ ，一次观测保守域为

$$F_1 = \Omega \cap B(s_1, 1500) \cap S(s_1, \theta) \quad (6-1)$$

先平移 $-s_1$ ，再旋转 $-\theta$ ，可把首点和中心示向度变为原点和 0° 。求得标准坐标第二点 p' 后，用 $p = s_1 + R(\theta)p'$ 还原。正文中的数值坐标均为标准化示例。

6.3 保证接收区域

源的有效接收半径至少为 1000 m。如果第二点 p 对 F_1 中所有可能源都满足距离不超过 1000 m，则第二次检测必定能够收到信号。保证接收区域定义为

$$C_{\text{rec}} = \left\{ p : \max_{z \in F_1} \|p - z\|_2 \leq 1000 \right\} = \bigcap_{z \in F_1} B(z, 1000) \quad (6-2)$$

距离函数关于 z 是凸函数，在紧凸多边形上的最大值由顶点达到，所以程序只需检查 F_1 的顶点。式 (6-2) 同时使用了半径下限 1000 m 和正观测带来的 1500 m 上限。

6.4 连续最坏交会角与候选排序

设两条视线的锐交会角为 γ ，以连续最坏角衡量第二点的定位几何

$$\bar{\gamma}(p) = \inf_{z \in F_1} \gamma(z; s_1, p), \quad \gamma = \arcsin \frac{|(z - s_1) \times (z - p)|}{\|z - s_1\|_2 \|z - p\|_2} \quad (6-3)$$

本文设置工程门槛 $\bar{\gamma}(\mathbf{p}) \geq 30^\circ$ ，但它不改变保证接收硬约束。候选点排序目标为

$$J(\mathbf{p}) = \bar{\gamma}(\mathbf{p}) - 0.002 \|\mathbf{p} - \mathbf{s}_1\|_2, \quad \mathbf{p} \in C_{\text{rec}} \quad (6-4)$$

在标准坐标中令源为 $r(\cos\phi, \sin\phi)$ ，第二点为 (d, δ) ，则

$$\sin\gamma = \frac{d |\sin(\delta - \phi)|}{\sqrt{r^2 + d^2 - 2rd\cos(\delta - \phi)}} \quad (6-5)$$

对固定角差，分母平方关于 r 为凸二次函数，区间 $[5, 1500]$ 上的最大值出现在端点。程序进一步检查 $\phi = \pm 1^\circ$ ，共线零点和可微驻点，避免只查角度端点而遗漏严格为 0° 的共线情形。

6.5 候选点结果与网格敏感性

图 3 左侧展示首测保守源域，右侧展示满足连续 30° 门槛的候选点及其最坏交会角。候选点位于首示向线两侧，既远离同向基线，又未离开保证接收区域。

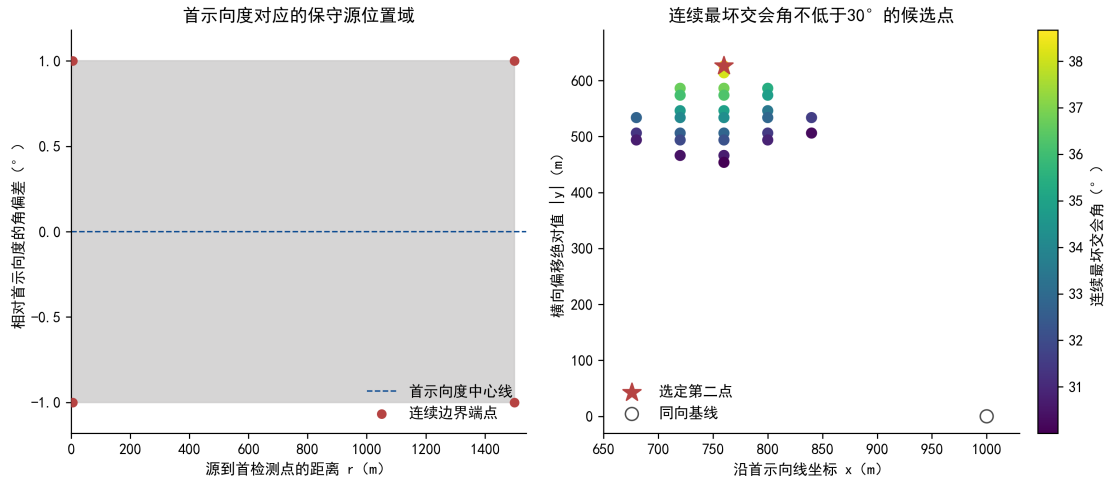


图 3：问题二保证接收且满足连续交会角约束的第二检测点候选区域

图 3 中 40m 网格共有 28 个连续可行候选点，目标函数选择 $(760.000, -626.179) \text{ m}$ ，移动距离为 984.733 m，连续最坏交会角为 38.667545° ，到一次观测域的最大距离为 986.333 m。由此，第二点既不会因接收半径取 1000 m 而失去信号，也不会落在同向基线的退化位置。

表 4： 问题二网格敏感性

网格/ m	候选数	选定点/ m	最坏角	最大距离/ m
20	109	(760, 633.821)	39.009485°	991.404589
40	28	(760, -626.179)	38.667545°	986.333438
80	6	(760, -626.179)	38.667545°	986.333438

表 4 显示，40 m 和 80 m 网格得到同一个最优点，20 m 网格得到关于首示向线近似镜像的点。三个网格均满足 30° 门槛和 1000 m 接收保证，说明“离轴并保持保证接收”的选择对网格步长不敏感。这里的网格只用于近似搜索，连续候选区域仍由式 (6-2) 和角度门槛定义。若一般实例的候选集为空，程序应降低工程门槛或沿 C_{rec} 边界补测，但不能放宽 1000 m 硬约束。

第二点选择还要区分“能够收到信号”和“能够形成稳定交会”这两个条件。前者是对 F_1 中全部位置的最坏距离约束，后者是对同一位置集合上的最坏交会角约束，二者不能用一个平均距离互相替代。若首点已经返回距离过近，问题二的第二点任务自然终止；只有在正常示向度情形下，才需要移动到候选区域并再次检测。候选区域给出的是可行位置集合，表中的坐标则是该集合在指定网格上的一个选择。

排序式中的距离惩罚系数只用于平衡移动距离和交会角，角度统一按度数记录，距离统一按米记录。它不代表概率，也不参与真实源位置的删除。这样，改变系数只会影响候选点的优先顺序，不会把不满足 1000 m 保证接收的点重新纳入候选区域。

七、问题三模型的建立与求解

7.1 问题三分析

问题三是未知频道下的全向源在线搜索。扫描点必须连续覆盖整个目标圆域，频道轮询必须覆盖所有可能的频道，定位和清除还要始终保留真实源。本文先求规则环族的极小极大扫描集^[4]，再将问题一的集合定位和问题二的保证接收思想嵌入在线流程。计算时先完成连续发现，再进行保守定位和服务点选择。扫描路线可以滚动调整，但不能删除七点环中的必要扫描点。

7.2 七点无圆心连续覆盖

设 n 个等角点构成半径为 a 的正多边形环

$$P_n(a) = \left\{ a\mathbf{u} \left(\frac{2k\pi}{n} \right) : k = 0, \dots, n-1 \right\}, \quad \alpha = \frac{\pi}{n} \quad (7-1)$$

对任意源位置 $r\mathbf{u}(\theta)$ ，选择方位最接近的环点，角差满足 $|\beta| \leq \alpha$ ，距离平方为

$$d^2(r, \beta) = r^2 + a^2 - 2ar\cos\beta \quad (7-2)$$

对固定 r ，式 (7-2) 随 $|\beta|$ 不减；对固定 β ，它关于 $r \in [0, R]$ 为凸函数。因此圆域内的最坏点只可能是圆心或目标圆边界上两个相邻点的角平分点，最坏最近距离为

$$d_{\max}(a, n) = \max \left\{ a, \sqrt{R^2 + a^2 - 2aR\cos\left(\frac{\pi}{n}\right)} \right\} \quad (7-3)$$

令两类最坏距离相等得到极小极大半径

$$a_n^* = \frac{R}{2\cos(\pi/n)}, \quad d_{\max}(a_n^*, n) = a_n^* \quad (7-4)$$

当 $R = 1800 \text{ m}$ 时， $n = 7$ 给出 $a_7^* = 998.924638 \text{ m}$ ，小于最小接收半径 1000 m ； $n = 6$ 给出 $a_6^* = 1039.230485 \text{ m}$ ，不能完成连续覆盖。因此本文不再扫描圆心，采用 7 个环点。这个结论只针对无圆心正多边形环模型族，不声称任意不规则点集的全局最少点数。

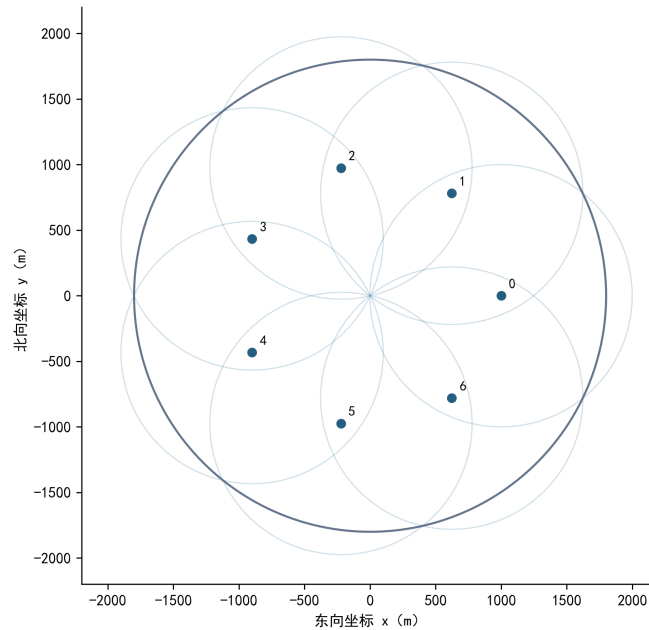


图 4：问题三七点极小极大环的连续覆盖

图 4 中的七个接收圆覆盖目标圆域，最坏点由式 (7-3) 解析确定，而不是由绘图采样猜出。不设置圆心扫描点是两类最坏距离平衡后的结果，因此没有以覆盖完整性换取速度。

7.3 未知频道扫描与终止规则

扫描开始时将 1 至 20 号频道置为未解析。每个必要扫描点依次检测尚未发现的频道，频道首次返回示向度或距离过近后记为已发现。七个扫描点全部完成后，若某频道

没有有效反馈，则该频道在全向模型下不存在干扰源。另一个合法提前终止条件是已发现 16 个不同频道，因为 16 是题面源数上限。不能因为暂时找到 10 个源或连续若干次无信号而提前停止。

对于已发现但示向条数不足的频道，若当前保守域为 F_c ，且候选补测点满足

$$\text{dist}(\mathbf{p}, F_c) > 1501 \text{ m} \quad (7-5)$$

则该点到真实源必大于 1500 m，可以安全跳过。该剪枝只对已发现频道使用，未知频道仍要完成全覆盖扫描。

7.4 保守定位与保证清除

频道 c 已有 m_c 条有效示向度时，在线保守域为

$$F_c = \Omega \cap \bigcap_{i=1}^{m_c} [S_i(1.01^\circ) \cap B(\mathbf{s}_i, 1500)] \quad (7-6)$$

理论证明仍使用题面 1° 误差，程序用 1.01° 只增加浮点包络。补测点要求 F_c 的所有顶点到该点不超过 998 m，以便在接收半径取下限时仍收到信号。对候选点，程序计算位置样本处的信息矩阵

$$I(\mathbf{p}; \mathbf{x}) = \sum_{\mathbf{s} \in O_c \cup \{\mathbf{p}\}} \frac{\mathbf{n}(\mathbf{s}, \mathbf{x}) \mathbf{n}(\mathbf{s}, \mathbf{x})^\top}{\|\mathbf{x} - \mathbf{s}\|_2^2}, \quad \mathbf{n}(\mathbf{s}, \mathbf{x}) = \frac{(-(x_y - s_y), x_x - s_x)^\top}{\|\mathbf{x} - \mathbf{s}\|_2} \quad (7-7)$$

式 (7-7) 只衡量视线是否退化，不被解释为题面未给出的统计协方差。

当 F_c 的最小覆盖圆半径 $r_c \leq 19.5 \text{ m}$ 时，在其圆心清除必成功。为减少折返，程序在保证服务区域

$$G_c = \bigcap_{\mathbf{v} \in \text{vert}(F_c)} B(\mathbf{v}, 19.5) \quad (7-8)$$

内搜索进出路程较短的服务点，并重新检查全部顶点。若主动定位在限定轮数内没有收敛，则使用覆盖半径 $h = 19.11 \text{ m}$ 的三角格覆盖 F_c 。平面三角格任意一点到最近格点的距离不超过 $h < 20 \text{ m}$ ，所以至少一个格点能够成功清除。这里的三角格只用于已经缩小的定位域，不承担未知频道的全域发现任务；发现仍由七点环和 20 个频道的轮询完成。这样可以避免把局部清除网格误当成搜索区域覆盖网格。

7.5 滚动调度与算法流程

扫描点先按从原点出发的开放路线优化。扫描中，若已发现源的估计清除点插入当前扫描边的增量路程不超过 150 m，则允许顺路试探，否则保持覆盖扫描。发现阶段结束后，以各频道保守域的覆盖圆圆心构造多起点路线，并用 2-opt 换边降低移动距离，采用旅行商问题中的常用局部改进思路^[5, 6]。每清除一个源便重新计算剩余任务顺序。表 5 概括了在线策略，表中每一行都保留了一个完整性约束。

表 5：问题三在线策略

步骤	操作	完整性约束
1	生成七点环并规划开放路线	最坏距离小于 1000 m
2	扫描未发现频道，补测已发现频道	未知频道不得剪枝
3	更新角楔交和距离上界，必要时补测	每次裁剪保留真源
4	满足服务域则清除，否则有限试探	试探失败不删可行域
5	定位未收敛时执行三角格兜底	至少一个格点距真源小于 20 m
6	清除后重排，全部完成后退出	资源不足时记录失败状态

表 5 说明，路径优化只发生在已固定的扫描和清除安全层上。由式 (7-4) 可知全向源必在七点扫描中被发现，由式 (7-6) 可知有界误差不会排除真源，由式 (7-8) 或三角格可知已发现源最终存在成功清除点。在题面理想规则，数值计算有效且接口正常的条件下，算法可以给出条件性的完整清除保证。这个保证不等于每一局都能在限定程序运行时间内结束，后者还取决于源的位置，频道顺序，路线长度和接口剩余时间。

7.6 离线验证与结果分析

先在 60 个独立合成案例上比较无圆心七点环与“圆心加七个环点”的布局。两种布局最低合成清除比例均为 100%，七点环相对旧布局逐局平均降时 1.5796%，Bootstrap 区间为 0.7879% 至 2.3717%，其中 42 局更快，18 局更慢。这里的区间采用 Bootstrap 重抽样方法计算^[7]。随后在 50 个独立留出案例上比较冻结前基线与终版策略，结果列于表 6。

表 6： 问题三 50 局独立留出验证

指标	冻结前基线	终版策略
最低合成清除比例	100.0%	100.0%
平均每源时间 s/个	310.642	297.205
每源时间 P95 s/个	389.53	372.20
逐局平均降时	0.0000%	4.1550%
Bootstrap 95% 区间	无	2.7846% 至 5.3846%
更快局数	无	42/50

表 6 给出的是同案例配对比较，终版平均每源时间减少 13.437s，且 50 局均完成合成清除。图 5 把平均每源时间和 P95 并列展示，图中终版的两个柱值都低于基线，说明收益不只来自少数异常快案例。上述结果只适用于已记录的合成留出集，不替代官方隐藏测试成绩。

问题3 独立50案例留出验证（离线合成，非官方成绩）

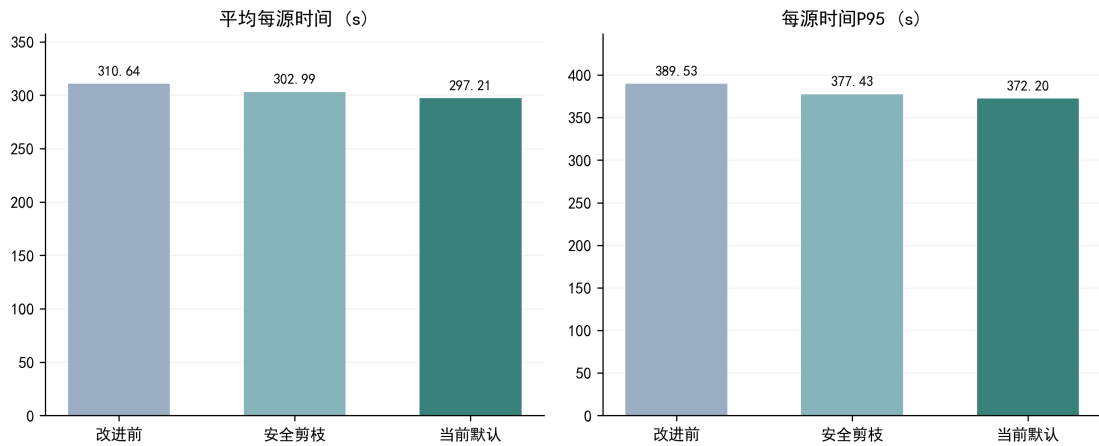


图 5： 问题三独立留出验证的每源时间比较

图 5 的左图和右图分别对应平均每源时间与 P95。两项指标都下降，说明安全剪枝和滚动调度同时减少了平均动作开销与尾部开销，但不能由此推出未知官方分布上的固定增益。

7.7 完整性判据与时间项分解

问题三的在线程序可以按发现，定位和清除三个状态理解。发现状态只判断某个频道是否已经得到有效反馈，不能因一次无信号就删除尚未查完的频道；定位状态保存该频道的角楔交与距离外包络，补测点必须先通过保证接收检查；清除状态只有在服务区或三角格兜底给出距离保证后才发送清除指令。状态由接口返回值推进，路线优化只调整待访问任务的顺序。

对全向源，七点环把“可能存在的源”对应到有限个必须访问的扫描点。式 (7-3) 控制了目标圆域内任意位置到最近扫描点的最大距离，所以源位于两个扫描点的分界线上时，仍至少有一个扫描点落在接收半径内。频道仍需从 1 至 20 号逐一检查，除非已经发现 16 个不同频道。这两个停止条件分别对应频道全集查完和题面给出的源数上限，不能用实验中常见的源数下限 10 替代。

时间按题面规定的动作逐项累计。移动距离，测量次数，切频次数以及成功和失败清除的固定耗时分别对应式 (4-1) 中的五项。只有在能够证明某个补测点距保守域超过 1501 m 时，安全剪枝才会跳过该点；2-opt 和有限前瞻也不改变七点覆盖及三角格兜底。因此，表 6 中的时间下降来自减少无效移动和重复动作，不是少做必要扫描或放宽清除距离。

7.8 正式测试记录

问题三的三次正式测试已通过模拟器完成并导出。本文已收到三份原始加密行为日志及其结果元数据，表 7 列出对应的案例编码和统计结果。表中时间单位为 s；平均定位清除时间按定位清除总时间除以清除个数计算，程序运行时间按题面定义从/enter 成功到测试结束计算。

表 7： 问题三正式测试结果

测试案例编码	清除数	平均时间/s	程序时间/s
测试 1: CAMG-7ANU- CEBJ-ZRND	11	319.136	22.539
测试 2: V8UP-GNKA- 8C3S-9VU4	11	303.959	23.874
测试 3: WGX6-PYRE- SSME-S6NQ	16	239.219	22.604

表 7 中的案例编码和三项统计结果来自模拟器正式测试记录，并与支撑材料中的原

始日志及结果元数据对应。**.jlog** 正文仍采用模拟器加密格式，未对原始文件作修改。表中结果不以离线实验数据或元数据起止时间差代替。

八、问题四模型的建立与求解

8.1 问题四分析

定向源的负观测不能直接解释为频道不存在，因此问题四需要覆盖源周围的任意正向半平面。本文用原点，内环和外环组成 21 点扫描集，并用缩小接收圆，放大目标域的保守几何检验检查连续覆盖。发现后的正负观测只删除严格不相容的位置单元，有限前瞻和服务点选择只改变顺序。正向包围判据放在流程前端，联合观测和时间前瞻放在该判据之后。任意中间步骤计算超时，都退回到上一层保证策略。

8.2 21 点正向包围扫描集

问题四采用扫描集

$$P_{21} = \{\mathbf{0}\} \cup \left\{ 996 \mathbf{u} \left(\frac{2k\pi}{8} \right) : k = 0, \dots, 7 \right\} \cup \left\{ 1863.756 \mathbf{u} \left(\frac{2k\pi}{12} \right) : k = 0, \dots, 11 \right\} \quad (8-1)$$

为使数值检验保持保守，核验时将最小接收半径收紧为 997 m，并用包含目标圆域的 720 边形外包络代替目标圆域。外环半径略大于 1800 m，机器狗位置又不受目标圆域边界限制，因此外环扫描点可以正常使用。对任意源位置 $\mathbf{z}$ ，若半径 997 m 内的扫描点凸包包含 $\mathbf{z}$ ，则任意经过 $\mathbf{z}$ 的闭半平面都至少包含其中一个扫描点。该点距源不超过 997 m，小于题面给出的最小接收半径 1000 m，因此定向源至少返回一次有效结果，结果可能是示向度，也可能是距离过近。

对全部 21 个扫描点的三点组合，代码保留三点凸包与三个 997 m 内接收圆的交集，在该数值检验中得到 260 个非空的保守凸交集。每个交集内的位置都能由对应三点凸包表示，且距这三点均不超过 997 m；这些交集的并集覆盖了 720 边形外包络，按程序设定的面积容差未发现缺口。这里的计算采用平面计算几何中的凸域处理方法^[1,8]，因此“无缺口”指在给定多边形离散和数值容差下通过了充分性检查，不把离散计算夸大为任意布局的全局最优结论。图 6 给出 21 点与目标圆域的位置关系，其中内环负责中心区域的局部正向包围，外环承担边界位置的连续覆盖。

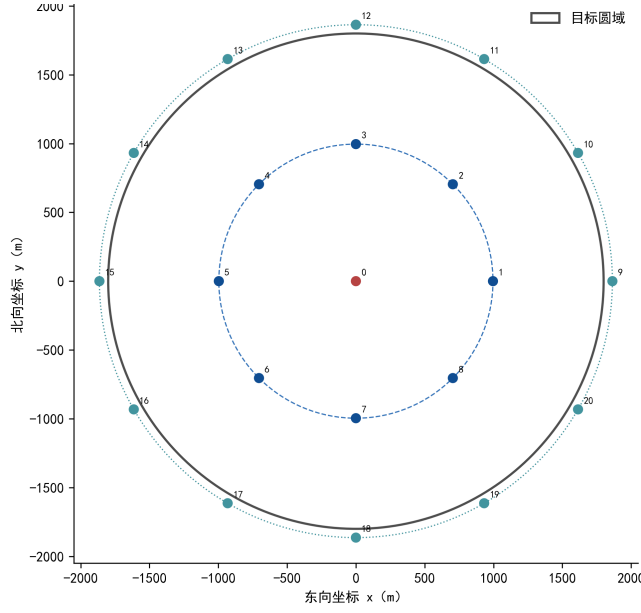


图 6：问题四 21 点扫描集的连续覆盖与正向包围

图 6 显示内环和外环均留有小于接收半径的几何余量。这个图只作结构展示，连续性结论来自上述 260 个保守凸交集的并集核验，而不是来自图像上的离散采样。

8.3 正负观测的联合相容域

设某个位置单元的源位置为 $\mathbf{z}$ ，有效接收半径为 $\rho \in [1000, 1500]$ ，定向发射方向为单位向量 $\mathbf{u}$ 。若接口返回示向度，正观测在几何上要求距离不超过 ρ ，并要求测向误差落在角楔内；若接口返回距离过近，则转入清除状态，不再人为补造示向度。负观测只能排除同时满足“距离不超过 ρ ”和“检测点位于发射半平面”的全部假设。程序把半径和方向划分为有限候选单元，并仅删除可以证明与所有候选均不相容的单元。其抽象写法为

$$\mathcal{H}_c = \{(\mathbf{z}, \rho, \mathbf{u}) : \mathbf{z} \in \Omega, 1000 \leq \rho \leq 1500, \mathbf{u} \in \mathbb{S}^1, \text{所有观测均相容}\} \quad (8-2)$$

若某单元未被严格证明为空，则保留其位置投影并作凸包外包络。对一般示向度反馈，实际距离上界仍由 1500 m 控制，不能直接收紧为 1000 m；997 m 只用于扫描集连续充分性检查所提供的安全余量。

8.4 有限前瞻与安全服务点

在硬约束筛选后，程序从位置域内部样本，半径区间和相容方向弧构造有限假设集 H ，估计下一动作的代理代价

$$\hat{C}(\mathbf{p}) = \frac{\|\mathbf{p} - \mathbf{x}_{\text{now}}\|_2}{5} + 5 + I_{\text{sw}} + \sum_{h \in H} w_h \hat{C}_{\text{after}}(\mathbf{p}, h), \quad \sum_{h \in H} w_h = 1 \quad (8-3)$$

其中 H 是有限的候选状态集, w_h 是用于排序的非负权重, I_{sw} 取 0 或 1 s, 分别表示保持当前频道或需要切换频道。只有当新候选相对几何首选至少节省 3 s 代理时间时才调整顺序。权重不是官方概率分布, 也不参与位置删除或清除判定。

保证清除仍使用式 (7-8) 的服务区域。若定位域半径不超过 100 m, 程序允许有限次数的中心线交点试探; 单示向度和疑似背向时采用对称探针或二分恢复。试探失败后仍保留测向信息并返回保守域, 最终用 19.11 m 三角格兜底。发现阶段若插入清除点的增量路程不超过 200 m 才顺路尝试, 避免用估计点破坏全域扫描顺序。

8.5 问题四验证结果

先在同案例上比较 21 点连续覆盖核验布局和 25 点紧凑布局。两者最低合成清除比例均为 100%, 21 点布局平均每源时间为 536.321 s, 25 点布局为 550.914 s, 按两组均值计算前者降低 2.649%。在冻结终版上, 55 局分层重放和另一组 24 局共享误差压力集的配对结果列于表 8。

表 8: 问题四终版配对离线结果

指标	21 点基线	最终组合
最低合成清除比例	100.0%	100.0%
55 局平均时间/s	6725.841	6644.145
55 局平均每源时间/s	546.385	540.366
55 局每源时间 P95/s	733.738	729.622
55 局配对降时	0.000%	1.157%
24 局压力集配对降时	0.000%	1.126%
发现集合错误与真源排除	0	0

表 8 显示, 两组实验均维持 100% 合成清除, 最终组合在 55 局中 49 局更快, 6 局更慢, 另一组 24 局共享误差压力集全部更快。图 7 把两组的平均时间和逐局降幅放在一起, 收益存在场景波动, 但没有以合成漏源换取时间。

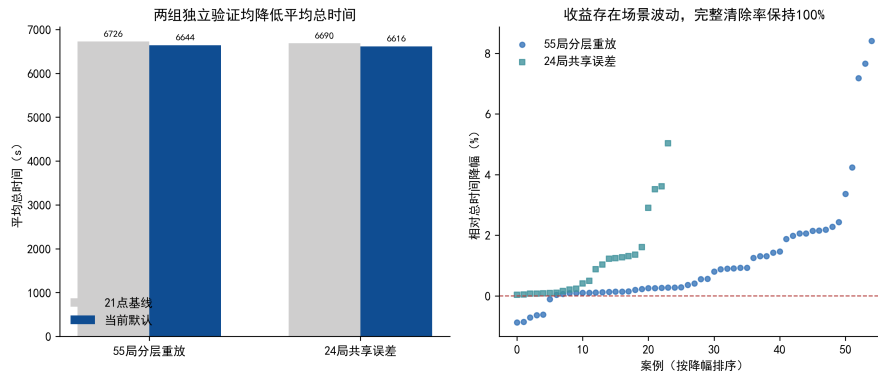


图 7： 问题四重放集与共享误差压力集的配对验证

图 7 左图显示两组平均总时间均下降，右图显示 55 局重放存在少量负收益而另一组 24 局共享误差压力集全部为正。由此只能说明当前安全细化在给定合成场景中具有平均收益，不能把区间解释为官方隐藏案例的置信保证。

表 9： 问题四 24 局共享误差压力集消融结果

策略	最低清除率	平均时间/s	配对降时	快/慢/同
基线	100.0%	6690.175	0.000%	0/0/24
仅安全服务点	100.0%	6680.275	0.148%	24/0/0
仅联合约束	100.0%	6647.505	0.630%	12/4/8
联合约束 + 安全点	100.0%	6638.592	0.762%	24/0/0
最终组合	100.0%	6616.262	1.126%	24/0/0

表 9 说明联合约束贡献了主要收益，安全服务点和有限前瞻进一步减少路线开销。图 8 也显示三个细化模块组合后收益最高，但模块之间共享状态，收益不能简单相加。消融结果的作用是解释机制，正向发现的安全条件仍由式 (8-1) 和三角格兜底保证。

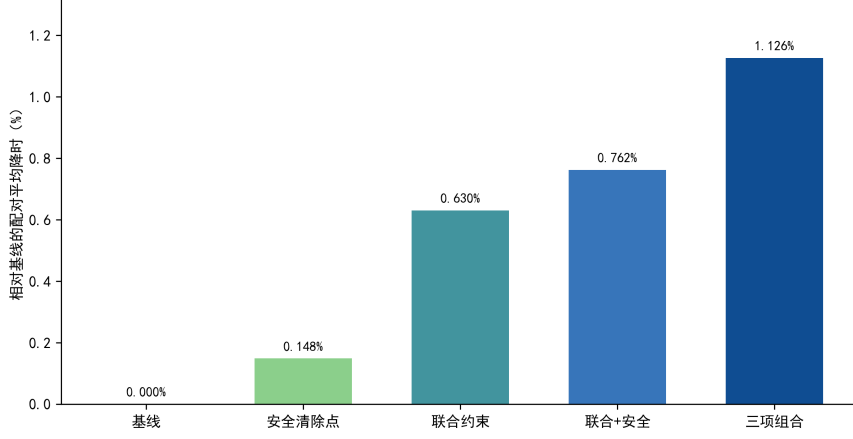


图 8：问题四安全细化模块的消融结果

图 8 把逐层细化的平均降时列出，最终组合为 1.126%。由于所有策略的最低合成清除比例均为 100%，该图的比较对象是时间，而不是安全性。

8.6 正向包围判据的推导

设源位置为 $\mathbf{z}$ ，把距离不超过 997m 的扫描点记为 $P(\mathbf{z})$ 。这里要确认的是 $\mathbf{z} \in \text{conv } P(\mathbf{z})$ ，并非要求某一个固定扫描点适应所有发射方向。在平面中，凸包表示最多需要三个点，所以代码逐组三点计算凸包与三个内接收圆的交集，再合并所有非空结果。这个局部判据把“距离足够近”和“方向不可能同时背向”放在同一个几何条件中，正是普通圆覆盖在问题四中不够用的原因。

若 $\mathbf{z}$ 属于其中一个三点凸交集，则存在不超过三个点 $\mathbf{p}_j$ 及权重 λ_j ，使得

$$\mathbf{z} = \sum_{j=1}^k \lambda_j \mathbf{p}_j, \quad \lambda_j \geq 0, \quad \sum_{j=1}^k \lambda_j = 1, \quad k \leq 3 \quad (8-4)$$

任取定向源的发射方向单位向量 $\mathbf{u}$ 。如果这些点都位于源的背向开半平面，则 $(\mathbf{p}_j - \mathbf{z}) \cdot \mathbf{u} < 0$ 对所有 j 成立。将式 (8-4) 移项并取内积，可得

$$0 = \sum_{j=1}^k \lambda_j (\mathbf{p}_j - \mathbf{z}) \cdot \mathbf{u} < 0 \quad (8-5)$$

这显然不成立。因此至少有一个扫描点满足 $(\mathbf{p}_j - \mathbf{z}) \cdot \mathbf{u} \geq 0$ 。该点距源不超过 997m，小于题面最小接收半径 1000m，所以它位于可接收的正向半平面内。等号对应定向方向两侧各 90° 的边界，题面将该边界计入有效覆盖角。

由此可见，问题四需要正向包围判据，普通圆覆盖并不够。圆覆盖只能说明“有一个近点”，不能说明这个点位于定向源的发射半平面；凸包包含关系同时约束了距离和方向。21 点方案的安全层也因此与联合正负观测，有限前瞻分开，后两者可以减少定位和路线时间，却不能替代全域扫描判据。

8.7 正式测试记录

问题四的三次正式测试已通过模拟器完成并导出。本文已收到三份原始加密行为日志及其结果元数据，表 10 列出对应的案例编码和统计结果。表中时间单位为 s；平均定位清除时间按定位清除总时间除以清除个数计算，程序运行时间按 `/enter` 成功到测试结束计算。

表 10： 问题四正式测试结果

测试案例编码	清除数	平均时间/s	程序时间/s
测试 1: PP5A-RCZ2- JVUT-69T7	11	567.883	53.870
测试 2: WYQT-GA22- Z34D-WTTG	12	559.197	51.663
测试 3: URKU-4C9V- 9R5H-2KFN	10	629.049	52.978

表 10 中的案例编码和三项统计结果来自模拟器正式测试记录，并与支撑材料中的原始日志及结果元数据对应。现有日志正文是加密行为记录，未对原始文件作修改；表中结果不以离线实验数据或元数据起止时间差代替。

九、模型评价与推广

9.1 证据分层与离线实验的统计解释

本文按结论性质安排证据。问题三的闭式极小极大覆盖推导，以及问题四在给定 21 点布局，997 m 检验半径和 720 边形外包络下的保守凸交集并集检查，用于判断理想题面条件下是否存在漏检位置。固定种子的离线回归只用于检查程序实现，比较虚拟时间和暴露脆弱场景。官方接口演练用于核对通信链路，正式测试结果则必须以模拟器界面和原始日志为准。

问题三和问题四的离线验证均采用同案例配对比较，使布局差异不会掩盖策略差异。比较时同时报告平均值，Bootstrap 区间，胜负局数，P95 和最坏变化，表 6 和表 8 也保留完整清除比例。这里的区间只描述当前合成案例中的平均配对差，不能解释为官方隐藏案例总体的置信保证。问题三有少数负收益案例，问题四的 55 局重放中也有变慢局，这说明时间启发式仍然依赖场景。

离线模拟器复现题面可见的运动，检测，切频，清除，接收半径和固定位置误差规

则。压力集另外加入边界源，最小接收半径，固定极限误差和共享位置误差，用于检验安全层的余量，但这些构造不等同于官方隐藏生成器。随机回归负责发现程序错误和比较版本，连续覆盖的安全判断仍由解析推导或保守几何检查承担。

9.2 鲁棒性与误差来源分析

(1) 有界误差与工程余量。题面给出的示向误差是确定性界 $[-1^\circ, 1^\circ]$ 而不是标准差，问题一和问题二的数学模型统一使用 1° 。在线程序使用 1.01° 只是在浮点边界处扩大包络，不会提高点估计精度，代价是可能增加少量补测或兜底清除点。接收半径下限 1000 m 用于问题二的保证再次接收，问题三的圆覆盖和问题四的定向发现，上限 1500 m 用于正观测后的保守定位域。若把正观测一律收紧为 1000 m，可能排除真实源；若只使用 1500 m，则会增加后续筛选范围和移动开销。

(2) 数值几何误差。问题一的圆弧边界采用内外正多边形夹逼，圆盘边数加倍时，直径夹逼宽度约缩小到原来的四分之一。问题四的检查反向设置安全余量，将目标域扩大，把接收半径收紧到 997 m，并使用内接多边形表示接收圆，因此数值近似不容易把真实缺口误判为覆盖。对 21 点布局，代码逐组三点计算凸包与接收圆的交集，得到 260 个非空保守凸交集，并记录并集覆盖和未覆盖面积。由于该过程仍包含多边形离散和数值容差，本文把它表述为给定检验设置下的充分性检查，不把结果扩展为任意布局的全局最优结论。

失败清除点也纳入安全处理。问题四的联合约束只在该点与候选位置的距离关系能够排除相容状态时才使用，不能因为一次失败就删除整个位置域。直接授权清除时使用的最小覆盖圆半径为 19.5 m，主动定位未收敛时使用覆盖半径 19.11 m 的三角格，两者均小于题面 20 m 的成功阈值。若试探失败，程序保留测向信息并回到保守位置域。

(3) 现实时间和接口风险。虚拟时间由式 (4-1) 逐项计算，程序实际运行时间还会受到网络延迟，模拟器负载和几何计算量影响。终版保留动作上限，请求幂等标识，截止时间和退出预留，规划预算不足时回到安全候选。这个机制可以避免启发式搜索耗尽全部运行时间，但不能保证长期断网，接口异常或外部强制终止时仍能完成任务。本文只讨论题面理想规则和接口正常时的条件性完整清除。

9.3 模型优点

(1) 模型将完整性和时间优化分层。问题三的七点解析覆盖和问题四的 21 点正向包围负责发现，集合外包络和三角格负责清除，2-opt，服务点和有限前瞻只改变访问顺序。启发式判断出现偏差时，主要代价是多移动或多检测，而不是把真实源从安全集合中删除。

(2) 四个问题使用同一套集合语言衔接。问题一给出闭角楔，可行域直径和最小覆盖圆，问题二把一次可行域变成保证接收区域，问题三加入全向连续覆盖，问题四把距离

覆盖推广为任意发射半平面下的正向包围。1000 m 下限，1500 m 上限，1° 误差和 20 m 清除半径在各问中的角色保持一致。

(3) 数值证据有明确边界。解析式负责连续性，计算机辅助检验负责复杂几何，固定种子回归负责代码比较，官方演练负责接口核对。表 6 和表 8 同时报告平均值，尾部指标和完整清除情况，避免只挑选最好的一次运行。

9.4 模型局限

(1) 问题三只在无圆心正多边形环族内得到七点可行的最小点数结论，问题四只在给定的 21 点布局 and 保守几何检验设置下验证其可行性。本文尚未得到任意不规则点集的全局最少点数，也没有得到带反馈路径优化的全局最短时间。更强的结论需要研究局部正向覆盖数和自适应路径的下界。

(2) 问题四的 260 个保守凸交集比问题三的解析公式更难人工逐行核查。后续可以按旋转对称性把这些交集归并为少数基本区域，并为每类区域给出显式不等式，以减少计算机辅助检验的审计成本。

(3) 问题二的 30° 门槛，问题三和问题四的有限前瞻权重来自工程排序，不来自官方案例分布。它们不改变 1000 m 接收保证和 20 m 清除保证，但效率收益会随隐藏案例分布变化。正式测试统计应以模拟器结果页和原始日志核验，离线平均秒数只能作为调参证据。

(4) 完整性以题面参数界和接口正常为条件。若示向误差超过 1°，接收半径低于 1000 m，清除判据改变，或现实时间被异常网络和系统负载耗尽，原有覆盖判据不再适用。程序可以记录一部分异常，但不能把超出题面的故障纳入无条件保证。

9.5 灵敏度与模型推广

(1) 误差半角和接收半径下限直接控制安全层尺度。可以在保留确定性外包络的前提下引入分层误差界，并将概率信息只用于排序。

(2) 外环点数和内环半径可以作为整数变量，与正向包围判据一起构成新的混合优化问题。接收下限改变时，必须重新计算环点数，外环半径和凸交集并集，不能直接沿用 21 点布局。

(3) 网格步长主要影响候选点数量，服务点和有限前瞻参数主要影响时间。后续可以把机器狗的实时剩余时间纳入滚动规划，在预算不足时自动选择最小安全覆盖动作。

各参数的影响可以分为安全层和效率层。示向误差半角增大时，每条角楔向两侧张开，多次观测的交集只会扩大或保持不变，单次观测域也会扩大；问题一的直径，问题二的保证接收区域以及问题三，问题四的补测次数都会随之变化。误差界不能依据离线样本随意缩小，因为平均定位误差不能替代题面给出的最坏误差界。

接收半径下限直接决定扫描集的尺度。问题三中，七点环的半径和最坏距离由目标

圆半径与环点数共同决定，六点布局已经超过 1000 m，不能为了缩短路线删去一个环点。问题四中，997 m 只是低于 1000 m 的计算检验半径，内环的 996 m 和外环的边界余量也都围绕这一安全尺度设置。若接收下限改变，就要重新计算环点数，外环半径和凸交集并集，不能直接沿用 21 点布局。

离散化参数只影响计算精度，不改变题面模型。问题一的圆盘边数，问题四的 720 边形边数以及接收圆的内接多边形边数越大，保守检验越接近真实边界，计算量也越大。本文用内接和外切多边形夹逼问题一的圆盘，用外包络处理问题四的目标圆，并在误差过大时增加边数复核。这样，精度不足会表现为区间过宽或检验不通过，不会被程序悄悄改写成一个看似精确的坐标。

效率参数都在安全判据之后发挥作用。机器狗速度，切频时间和检测时间直接进入式 (4-1)，源数和频道占用情况决定必须执行的扫描次数；2-opt，有限前瞻和服务点选择只减少路线或重复动作。现实接口出现延迟，返回格式异常或剩余时间不足时，程序应先停止启发式搜索，保留当前保守域并主动退出。这样记录出的退化状态，不能被解释成离线时间优势对正式测试的保证。

本文把“发现完整”和“在时间窗口内完成”分开报告。前者由七点环，21 点正向包围和保守位置域控制，属于几何及接口规则下的安全判断；后者还受到频道轮询顺序，移动距离，失败清除次数和程序运行时间的影响。离线实验中的 100% 清除只表示在给定合成案例和虚拟计时下找到了全部真源，不能替代表 7 和表 10 所需的正式测试统计，也不表示任意现实环境都能达到相同时间。

对问题四，正负观测的作用也不是对称的。正观测可以收紧位置和方向的相容范围，负观测只有在某个位置，半径和发射方向组合下必然应该收到信号时，才足以删除该组合。若一个候选单元仍能通过改变发射方向或接收半径解释当前无信号，程序就保留它。这个处理会增加后续测量，但能避免把定向源的背向状态误判成频道不存在。

模型推广时，外环点数，内环半径和接收余量应一起重新计算。只增加扫描点而不重新检查三点凸交集，不能保证正向包围；只缩小清除域而不核对接口计时，也不能保证程序在时间窗口内退出。实际使用中可以把几何检验结果，剩余程序时间和当前未清除频道数作为滚动规划的输入，在安全约束不变的前提下调整访问顺序。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色，代码调试和排版检查，详细使用情况见支撑材料。

参考文献

- [1] SUTHERLAND I E, HODGMAN G W. Reentrant polygon clipping[J]. Communications of the ACM, 1974, 17(1): 32-42. DOI: 10.1145/360767.360802.
- [2] WELZL E. Smallest enclosing disks[C]//New Results and New Trends in Computer Science. Berlin: Springer, 1991: 359-370. DOI: 10.1007/BFb0038202.
- [3] BISHOP A N, FIDAN B, ANDERSON B D O, et al. Optimality analysis of sensor-target localization geometries[J]. Automatica, 2010, 46(3): 479-492. DOI: 10.1016/j.automatica.2009.12.003.
- [4] KERSHNER R. The number of circles covering a set[J]. American Journal of Mathematics, 1939, 61(3): 665-671. DOI: 10.2307/2371320.
- [5] CROES G A. A method for solving traveling-salesman problems[J]. Operations Research, 1958, 6(6): 791-812. DOI: 10.1287/opre.6.6.791.
- [6] LIN S, KERNIGHAN B W. An effective heuristic algorithm for the traveling-salesman problem[J]. Operations Research, 1973, 21(2): 498-516. DOI: 10.1287/opre.21.2.498.
- [7] EFRON B. Bootstrap methods: Another look at the jackknife[J]. The Annals of Statistics, 1979, 7(1): 1-26. DOI: 10.1214/aos/1176344552.
- [8] DE BERG M, CHEONG O, VAN KREVELD M, et al. Computational Geometry: Algorithms and Applications[M]. 3rd ed. Berlin: Springer, 2008. DOI: 10.1007/978-3-540-77974-2.

附录 A 支撑材料文件列表

表 11: 支撑材料文件列表

文件名	用途说明
robot.py	机器狗策略主程序和 HTTP 接口
geometry.py	角楔、半平面、凸域和覆盖几何
offline_sim.py	离线确定性回归模拟器
q4_refinement.py	问题四联合约束和细化模块
run_experiments.py	几何实验和批量实验入口
test_solution.py	解答验证套件
benchmark_q3_improvements.py	问题三改进策略实验
benchmark_q4_improvements.py	问题四改进实验
README.md	支撑材料说明和结果边界
提交说明.txt	打包说明和运行依赖
AI 工具使用详情.pdf	AI 工具使用过程和人工核验说明
tools/make_flowcharts.py	总体流程图生成脚本
formal_test_records.csv	正式测试编码和核验状态

附录 B 完整源程序代码

B.1 geometry.py

```
1 """Geometry utilities for bearing-only localization with bounded angular error."""
2 from __future__ import annotations
3
4 import math
5 from dataclasses import dataclass
6 from typing import Iterable, Sequence
7
8 import numpy as np
9
10
11 EPS = 1e-10
12
13
14 def wrap_pi(angle: float) -> float:
15     return (angle + math.pi) % (2.0 * math.pi) - math.pi
16
17
18 def unit(angle: float) -> np.ndarray:
19     return np.array([math.cos(angle), math.sin(angle)], dtype=float)
20
21
22 def cross(a: np.ndarray, b: np.ndarray) -> float:
23     return float(a[0] * b[1] - a[1] * b[0])
24
25
26 def bearing(p: Sequence[float], q: Sequence[float]) -> float:
27     return math.atan2(q[1] - p[1], q[0] - p[0])
28
29
30 def line_intersection(p: np.ndarray, u: np.ndarray, q: np.ndarray, v: np.ndarray):
31     den = cross(u, v)
32     if abs(den) < EPS:
33         return None
34     t = cross(q - p, v) / den
35     return p + t * u
36
37
38 def _clip_halfplane(poly: list[np.ndarray], a: np.ndarray, b: float, tol=1e-9):
39     """Keep a dot x >= b, using Sutherland-Hodgman clipping."""
40     if not poly:
41         return []
42     out: list[np.ndarray] = []
43     for p, q in zip(poly, poly[1:] + poly[:1]):
44         fp = float(np.dot(a, p) - b)
45         fq = float(np.dot(a, q) - b)
46         inp, inq = fp >= -tol, fq >= -tol
47         if inp:
48             out.append(p)
49         if inp != inq:
```

```

50         den = fp - fq
51         if abs(den) > EPS:
52             lam = fp / den
53             out.append(p + lam * (q - p))
54     return out
55
56
57 def clip_convex_polygon_halfplane(
58     poly: np.ndarray,
59     a: Sequence[float],
60     b: float,
61     tol: float = 1e-9,
62 ) -> np.ndarray:
63     """Return the part of a convex polygon satisfying ``a dot x >= b``."""
64     clipped = _clip_halfplane(
65         [np.asarray(point, dtype=float) for point in np.asarray(poly, dtype=float)],
66         np.asarray(a, dtype=float),
67         float(b),
68         tol,
69     )
70     if not clipped:
71         return np.empty((0, 2), dtype=float)
72     return np.asarray(clipped, dtype=float)
73
74
75 def sector_halfplanes(observer: Sequence[float], theta: float, error_deg: float = 1.0):
76     """Return (a,b) constraints a dot z >= b for a forward angular sector."""
77     p = np.asarray(observer, dtype=float)
78     d = math.radians(error_deg)
79     lo, hi = theta - d, theta + d
80     u_lo, u_hi = unit(lo), unit(hi)
81     # cross(u_lo, z-p) >= 0 -> (-u_lo_y, u_lo_x) dot z >= cross(u_lo, p)
82     a1 = np.array([-u_lo[1], u_lo[0]])
83     b1 = float(np.dot(a1, p))
84     # cross(u_hi, z-p) <= 0 -> (u_hi_y, -u_hi_x) dot z >= -cross(u_hi, p)
85     a2 = np.array([u_hi[1], -u_hi[0]])
86     b2 = float(np.dot(a2, p))
87     return [(a1, b1), (a2, b2)]
88
89
90 def bearing_sector_polygon(
91     observations: Iterable[tuple[Sequence[float], float]],
92     error_deg: float = 1.0,
93     tol: float = 1e-8,
94 ) -> tuple[np.ndarray, str]:
95     """Intersect only bearing half-planes and classify the resulting set.
96
97     A bounded nonempty intersection is returned by its convex-hull vertices.
98     For an unbounded intersection, the returned finite vertices are diagnostic
99     only; callers should add the official target disk before computing a
100     physical diameter.
101     """
102     if not (0.0 < error_deg < 90.0):
103         raise ValueError("error_deg must lie strictly between 0 and 90 degrees")

```

```

104 halfplanes = [
105     (np.asarray(normal, dtype=float), float(offset))
106     for observer, theta in observations
107     for normal, offset in sector_halfplanes(observer, theta, error_deg)
108 ]
109 if not halfplanes:
110     return np.empty((0, 2), dtype=float), "unbounded"
111
112 candidates: list[np.ndarray] = []
113 for i, (left_normal, left_offset) in enumerate(halfplanes):
114     for right_normal, right_offset in halfplanes[i:]:
115         matrix = np.vstack([left_normal, right_normal])
116         if abs(float(np.linalg.det(matrix))) <= EPS:
117             continue
118         point = np.linalg.solve(matrix, np.array([left_offset, right_offset]))
119         if all(float(np.dot(normal, point) - offset) >= -tol for normal, offset in halfplanes):
120             candidates.append(point)
121
122 normals = np.asarray([normal for normal, _ in halfplanes])
123 normal_angles = np.sort(np.mod(np.arctan2(normals[:, 1], normals[:, 0]), 2.0 * math.pi))
124 circular_gaps = np.diff(np.r_[normal_angles, normal_angles[0] + 2.0 * math.pi])
125 unbounded = float(np.max(circular_gaps)) >= math.pi - 1e-12
126
127 if not candidates:
128     # Bearing sectors are pointed wedges. Within this domain, a feasible
129     # nonempty intersection has at least one pairwise boundary vertex.
130     return np.empty((0, 2), dtype=float), "empty"
131 hull = convex_hull(np.asarray(candidates, dtype=float))
132 if unbounded:
133     return hull, "unbounded"
134 if len(hull) == 1:
135     return hull, "point"
136 if len(hull) == 2:
137     return hull, "segment"
138 return hull, "polygon"
139
140
141 def disk_polygon(radius: float, n: int = 720, circumscribed: bool = True) -> list[np.ndarray]:
142     """Return a regular polygon approximation of a disk.
143
144     Feasible regions are used to decide whether a 20 m clear is guaranteed.
145     The default polygon therefore circumscribes the disk, so approximation
146     error can enlarge the uncertainty set but can never exclude the source.
147     """
148     if n < 8:
149         raise ValueError("a disk approximation needs at least 8 sides")
150     scale = radius / math.cos(math.pi / n) if circumscribed else radius
151     return [scale * unit(2 * math.pi * k / n) for k in range(n)]
152
153
154 def bearing_region(
155     observations: Iterable[tuple[Sequence[float], float]],
156     target_radius: float = 1800.0,
157     error_deg: float = 1.0,

```

```

158     extra_disks: Iterable[tuple[Sequence[float], float]] = (),
159     n_disk: int = 720,
160     circumscribed_disks: bool = True,
161 ) -> np.ndarray:
162     """Intersect angular sectors with polygonal target/reception disks.
163
164     Circumscribed polygons give a conservative outer feasible region. Setting
165     ``circumscribed_disks`` to false gives a contained inner region, which can
166     be paired with the outer region to audit circular-boundary approximation.
167     """
168     poly = disk_polygon(target_radius, n_disk, circumscribed=circumscribed_disks)
169     for p, theta in observations:
170         for a, b in sector_halfplanes(p, theta, error_deg):
171             poly = _clip_halfplane(poly, a, b)
172             if not poly:
173                 return np.empty((0, 2))
174     for center, radius in extra_disks:
175         # A polygonal disk is enough for planning; increase n_disk for final audit.
176         c = np.asarray(center, dtype=float)
177         disk = [
178             c + p
179             for p in disk_polygon(
180                 radius,
181                 n_disk,
182                 circumscribed=circumscribed_disks,
183             )
184         ]
185         # Intersect two convex polygons by clipping each edge of the disk polygon.
186         for p, q in zip(disk, disk[1:] + disk[:1]):
187             edge = q - p
188             # interior is left of CCW edge: cross(edge, z-p) >= 0
189             a = np.array([-edge[1], edge[0]])
190             b = float(np.dot(a, p))
191             poly = _clip_halfplane(poly, a, b)
192             if not poly:
193                 return np.empty((0, 2))
194     return np.asarray(poly, dtype=float)
195
196
197 def bearing_region_bounds(
198     observations: Iterable[tuple[Sequence[float], float]],
199     target_radius: float = 1800.0,
200     error_deg: float = 1.0,
201     extra_disks: Iterable[tuple[Sequence[float], float]] = (),
202     n_disk: int = 720,
203 ) -> tuple[np.ndarray, np.ndarray]:
204     """Return contained and conservative polygonal bounds for the feasible set."""
205     readings = [(np.asarray(point, dtype=float), float(theta)) for point, theta in observations]
206     disks = [(np.asarray(center, dtype=float), float(radius)) for center, radius in extra_disks]
207     inner = bearing_region(
208         readings,
209         target_radius,
210         error_deg,
211         disks,

```

```

212         n_disk,
213         circumscribed_disks=False,
214     )
215     outer = bearing_region(
216         readings,
217         target_radius,
218         error_deg,
219         disks,
220         n_disk,
221         circumscribed_disks=True,
222     )
223     return inner, outer
224
225
226 def convex_hull(points: np.ndarray) -> np.ndarray:
227     if len(points) <= 1:
228         return points.copy()
229     pts = sorted({(float(x), float(y)) for x, y in points})
230     if len(pts) <= 1:
231         return np.asarray(pts, dtype=float)
232
233     def orient(o, a, b):
234         return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])
235
236     lower = []
237     for p in pts:
238         while len(lower) >= 2 and orient(lower[-2], lower[-1], p) <= 1e-12:
239             lower.pop()
240         lower.append(p)
241     upper = []
242     for p in reversed(pts):
243         while len(upper) >= 2 and orient(upper[-2], upper[-1], p) <= 1e-12:
244             upper.pop()
245         upper.append(p)
246     return np.asarray(lower[:-1] + upper[:1], dtype=float)
247
248
249 def intersect_convex_polygons(subject: np.ndarray, clipper: np.ndarray) -> np.ndarray:
250     """Intersect two counter-clockwise convex polygons."""
251     poly = [np.asarray(point, dtype=float) for point in np.asarray(subject)]
252     clip = [np.asarray(point, dtype=float) for point in np.asarray(clipper)]
253     if not poly or not clip:
254         return np.empty((0, 2), dtype=float)
255     for left, right in zip(clip, clip[1:] + clip[:1]):
256         edge = right - left
257         normal = np.array([-edge[1], edge[0]], dtype=float)
258         poly = _clip_halfplane(poly, normal, float(np.dot(normal, left)))
259         if not poly:
260             return np.empty((0, 2), dtype=float)
261     return np.asarray(poly, dtype=float)
262
263
264 def diameter(poly: np.ndarray) -> float:
265     if len(poly) < 2:

```

```

266         return 0.0
267     hull = convex_hull(poly)
268     if len(hull) < 2:
269         return 0.0
270     return float(max(np.linalg.norm(hull[i] - hull[j]) for i in range(len(hull)) for j in range(i)))
271
272
273 @dataclass
274 class Circle:
275     center: np.ndarray
276     radius: float
277
278
279 def _circle_two(a: np.ndarray, b: np.ndarray) -> Circle:
280     return Circle((a + b) / 2.0, float(np.linalg.norm(a - b) / 2.0))
281
282
283 def _circle_three(a: np.ndarray, b: np.ndarray, c: np.ndarray) -> Circle | None:
284     d = 2 * cross(b - a, c - a)
285     if abs(d) < 1e-12:
286         return None
287     aa, bb, cc = np.dot(a, a), np.dot(b, b), np.dot(c, c)
288     ux = (aa * (b[1] - c[1]) + bb * (c[1] - a[1]) + cc * (a[1] - b[1])) / d
289     uy = (aa * (c[0] - b[0]) + bb * (a[0] - c[0]) + cc * (b[0] - a[0])) / d
290     o = np.array([ux, uy])
291     return Circle(o, float(np.linalg.norm(o - a)))
292
293
294 def _contains(circle: Circle, point: np.ndarray, tol: float = 1e-7) -> bool:
295     return float(np.linalg.norm(point - circle.center)) <= circle.radius + tol
296
297
298 def _circle_collinear(a: np.ndarray, b: np.ndarray, c: np.ndarray) -> Circle:
299     pairs = [(a, b), (a, c), (b, c)]
300     x, y = max(pairs, key=lambda pair: np.linalg.norm(pair[0] - pair[1]))
301     return _circle_two(x, y)
302
303
304 def min_enclosing_circle(points: np.ndarray) -> Circle:
305     """Smallest enclosing circle of a finite point set.
306
307     This is the deterministic incremental form of the standard randomized
308     algorithm. It is exact up to floating-point tolerance and avoids the
309     quartic candidate enumeration used by the first prototype.
310     """
311     pts = convex_hull(np.asarray(points, dtype=float))
312     if len(pts) == 0:
313         return Circle(np.zeros(2), math.inf)
314
315     circle: Circle | None = None
316     for i, p in enumerate(pts):
317         if circle is not None and _contains(circle, p):
318             continue
319         circle = Circle(p.copy(), 0.0)

```

```

320         for j, q in enumerate(pts[:i]):
321             if _contains(circle, q):
322                 continue
323             circle = _circle_two(p, q)
324             for r in pts[:j]:
325                 if _contains(circle, r):
326                     continue
327                 candidate = _circle_three(p, q, r)
328                 circle = candidate if candidate is not None else _circle_collinear(p, q, r)
329     assert circle is not None
330     return circle
331
332
333 def diameter_circle_certificate(points: np.ndarray, tol: float = 1e-7) -> dict[str, object]:
334     """Build a directly checkable certificate for the diameter-circle question."""
335     hull = convex_hull(np.asarray(points, dtype=float))
336     if len(hull) == 0:
337         return {
338             "status": "empty",
339             "diameter": 0.0,
340             "diameter_endpoints": None,
341             "diameter_center": None,
342             "diameter_radius": 0.0,
343             "witness": None,
344             "witness_distance": math.inf,
345             "overflow": math.inf,
346             "min_enclosing_circle": min_enclosing_circle(hull),
347             "support_points": np.empty((0, 2), dtype=float),
348             "covers": None,
349             "circle_inflation_ratio": None,
350         }
351
352     if len(hull) == 1:
353         circle = min_enclosing_circle(hull)
354         return {
355             "status": "point",
356             "diameter": 0.0,
357             "diameter_endpoints": (hull[0].copy(), hull[0].copy()),
358             "diameter_center": hull[0].copy(),
359             "diameter_radius": 0.0,
360             "witness": hull[0].copy(),
361             "witness_distance": 0.0,
362             "overflow": 0.0,
363             "min_enclosing_circle": circle,
364             "support_points": hull.copy(),
365             "covers": True,
366             "circle_inflation_ratio": None,
367         }
368
369     distance_value, left_index, right_index = max(
370         (
371             float(np.linalg.norm(hull[i] - hull[j])),
372             i,
373             j,

```

```

374     )
375     for i in range(len(hull))
376     for j in range(i)
377 )
378 left, right = hull[left_index].copy(), hull[right_index].copy()
379 midpoint = (left + right) / 2.0
380 distances = np.linalg.norm(hull - midpoint, axis=1)
381 witness_index = int(np.argmax(distances))
382 witness_distance = float(distances[witness_index])
383 diameter_radius = distance_value / 2.0
384 overflow = witness_distance - diameter_radius
385 circle = min_enclosing_circle(hull)
386 support_tol = max(tol, 1e-7)
387 support = hull[
388     np.abs(np.linalg.norm(hull - circle.center, axis=1) - circle.radius)
389     <= support_tol
390 ]
391 return {
392     "status": "segment" if len(hull) == 2 else "polygon",
393     "diameter": distance_value,
394     "diameter_endpoints": (left, right),
395     "diameter_center": midpoint,
396     "diameter_radius": diameter_radius,
397     "witness": hull[witness_index].copy(),
398     "witness_distance": witness_distance,
399     "overflow": overflow,
400     "min_enclosing_circle": circle,
401     "support_points": support,
402     "covers": overflow <= tol,
403     "circle_inflation_ratio": 2.0 * circle.radius / distance_value,
404 }
405
406
407 def robust_estimate(observations, target_radius=1800.0, error_deg=1.0):
408     poly = bearing_region(observations, target_radius, error_deg)
409     return poly, min_enclosing_circle(poly), diameter(poly)
410
411
412 def point_in_convex_polygon(point: Sequence[float], poly: np.ndarray, tol: float = 1e-8) -> bool:
413     """Membership of a CCW convex polygon, including point/segment limits."""
414     vertices = np.asarray(poly, dtype=float)
415     if len(vertices) == 0:
416         return False
417     q = np.asarray(point, dtype=float)
418     if not np.all(np.isfinite(q)) or not np.all(np.isfinite(vertices)):
419         return False
420     if len(vertices) == 1:
421         return float(np.linalg.norm(q - vertices[0])) <= tol
422     if len(vertices) == 2:
423         return point_segment_distance(q, vertices[0], vertices[1]) <= tol
424     # Clipping can leave repeated or collinear vertices. Cross products alone
425     # would otherwise accept an entire line (or plane) as inside that set.
426     shifted = vertices - vertices[0]
427     area2 = float(np.sum(shifted[:, 0] * np.roll(shifted[:, 1], -1)

```

```

428         - shifted[:, 1] * np.roll(shifted[:, 0], -1)))
429     if abs(area2) <= EPS:
430         return min(point_segment_distance(q, vertices[i], vertices[(i + 1) % len(vertices)])
431                     for i in range(len(vertices))) <= tol
432     return all(cross(vertices[(i + 1) % len(vertices)] - vertices[i], q - vertices[i]) >= -tol
433               for i in range(len(vertices)))
434
435
436 def point_segment_distance(point: Sequence[float], a: Sequence[float], b: Sequence[float]) -> float:
437     p, x, y = map(lambda z: np.asarray(z, dtype=float), (point, a, b))
438     d = y - x
439     if float(np.dot(d, d)) <= EPS:
440         return float(np.linalg.norm(p - x))
441     t = min(1.0, max(0.0, float(np.dot(p - x, d) / np.dot(d, d))))
442     return float(np.linalg.norm(p - (x + t * d)))
443
444
445 def point_polygon_distance(point: Sequence[float], poly: np.ndarray) -> float:
446     if len(poly) == 0:
447         return math.inf
448     if point_in_convex_polygon(point, poly):
449         return 0.0
450     return min(point_segment_distance(point, poly[i], poly[(i + 1) % len(poly)]) for i in
451               range(len(poly)))
452
453
454 def q3_cover_points(ring_radius: float = 1150.0) -> np.ndarray:
455     """Seven-point cover of the radius-1800 arena by radius-1000 disks."""
456     points = [np.zeros(2)]
457     points.extend(ring_radius * unit(k * math.pi / 3.0) for k in range(6))
458     return np.asarray(points, dtype=float)
459
460
461 def q3_ring_cover_points(
462     target_radius: float = 1800.0,
463     min_receive_radius: float = 1000.0,
464     sides: int = 12,
465     boundary_margin: float = 2.0,
466 ) -> np.ndarray:
467     """Center plus a regular ring with a continuous disk-cover certificate.
468
469     The center covers source radii through ``min_receive_radius``. In each
470     outer angular sector, distance to the nearest ring point is maximized at
471     the arena boundary and sector bisector. The closed-form radius below
472     places that worst point strictly inside the minimum reception disk.
473
474     """
475     if sides < 3:
476         raise ValueError("sides must be at least 3")
477     if not (0.0 <= boundary_margin < min_receive_radius):
478         raise ValueError("boundary_margin must lie inside the reception radius")
479     half_step = math.pi / sides
480     discriminant = min_receive_radius**2 - target_radius**2 * math.sin(half_step) ** 2
481     if discriminant <= 0.0:
482         raise ValueError("too few ring points to cover the target boundary")

```

```

481     ring_radius = (
482         target_radius * math.cos(half_step)
483         - math.sqrt(discriminant)
484         + boundary_margin
485     )
486     boundary_gap = math.sqrt(
487         target_radius**2
488         + ring_radius**2
489         - 2.0 * target_radius * ring_radius * math.cos(half_step)
490     )
491     if boundary_gap >= min_receive_radius:
492         raise ValueError("ring construction does not cover the target boundary")
493     points = [np.zeros(2)]
494     points.extend(ring_radius * unit(2.0 * math.pi * k / sides) for k in range(sides))
495     return np.asarray(points, dtype=float)
496
497
498 def q3_custom_ring_points(
499     ring_radius: float,
500     sides: int,
501     include_center: bool = True,
502 ) -> np.ndarray:
503     """Construct a regular Q3 ring for time-objective screening.
504
505     Coverage is audited by the experiment driver; the online policy still
506     rejects configurations that fail its explicit geometric audit.
507     """
508     if sides < 3 or ring_radius <= 0.0:
509         raise ValueError("invalid custom ring")
510     points = []
511     if include_center:
512         points.append(np.zeros(2))
513     points.extend(
514         ring_radius * unit(2.0 * math.pi * k / sides)
515         for k in range(sides)
516     )
517     return np.asarray(points, dtype=float)
518
519
520 def q3_minimax_ring_points(
521     target_radius: float = 1800.0,
522     min_receive_radius: float = 1000.0,
523     sides: int = 7,
524 ) -> np.ndarray:
525     """Regular ring minimizing the worst center-or-boundary cover distance."""
526     if sides < 3:
527         raise ValueError("sides must be at least 3")
528     half_step = math.pi / sides
529     ring_radius = target_radius / (2.0 * math.cos(half_step))
530     boundary_gap = math.sqrt(
531         target_radius**2
532         + ring_radius**2
533         - 2.0 * target_radius * ring_radius * math.cos(half_step)
534     )

```

```

535     if max(ring_radius, boundary_gap) >= min_receive_radius:
536         raise ValueError("ring has too few points to cover the target disk")
537     return np.asarray(
538         [ring_radius * unit(2.0 * math.pi * k / sides) for k in range(sides)],
539         dtype=float,
540     )
541
542
543 def _triangle_intersects_origin_disk(points: list[np.ndarray], radius: float) -> bool:
544     if any(float(np.linalg.norm(p)) <= radius + 1e-9 for p in points):
545         return True
546     if any(point_segment_distance((0.0, 0.0), points[i], points[(i + 1) % 3]) <= radius + 1e-9 for i
547         in range(3)):
548         return True
549     signs = [cross(points[(i + 1) % 3] - points[i], -points[i]) for i in range(3)]
550     return all(v >= -1e-9 for v in signs) or all(v <= 1e-9 for v in signs)
551
552 def q4_triangular_cover_points(target_radius: float = 1800.0, edge: float = 980.0) -> np.ndarray:
553     """Vertices of every triangular-lattice cell intersecting the arena.
554
555     Each retained triangle has side length ``edge < 1000``. A source in that
556     triangle is within 1000 m of all three vertices and is in their convex
557     hull; consequently every closed directional half-plane through the source
558     contains at least one detectable vertex.
559     """
560     if not (0.0 < edge < 1000.0):
561         raise ValueError("edge must be strictly between 0 and 1000 m")
562     vertical = math.sqrt(3.0) * edge / 2.0
563     span = int(math.ceil((target_radius + edge) / min(edge, vertical))) + 3
564
565     def lattice(i: int, j: int) -> np.ndarray:
566         return np.array([edge * (i + 0.5 * j), vertical * j], dtype=float)
567
568     used: set[tuple[int, int]] = set()
569     for i in range(-2 * span, 2 * span + 1):
570         for j in range(-2 * span, 2 * span + 1):
571             triangles = [
572                 ((i, j), (i + 1, j), (i, j + 1)),
573                 ((i + 1, j + 1), (i + 1, j), (i, j + 1)),
574             ]
575             for triangle in triangles:
576                 vertices = [lattice(*index) for index in triangle]
577                 if _triangle_intersects_origin_disk(vertices, target_radius):
578                     used.update(triangle)
579     return np.asarray([lattice(i, j) for i, j in sorted(used, key=lambda z: (z[1], z[0]))])
580
581
582 def q4_certified21_cover_points(
583     target_radius: float = 1800.0,
584     min_receive_radius: float = 1000.0,
585     boundary_margin_fraction: float = 0.00025,
586 ) -> np.ndarray:
587     """Return the continuously certified 21-point Q4 discovery cover.
```

```

588
589 For the official ratio ``target_radius / min_receive_radius <= 1.8``, the
590 origin, an eight-point ring at 0.996 of the reception radius, and a
591 circumscribed twelve-point boundary ring positively span every source in
592 the target disk using points within 0.997 of the reception radius. Hence
593 every closed 180-degree emitting half-plane contains a detectable point.
594 The 0.003 reception margin absorbs the numerical and boundary tolerances.
595 """
596 if min_receive_radius <= 0.0:
597     raise ValueError("min_receive_radius must be positive")
598 if target_radius <= 0.0:
599     raise ValueError("target_radius must be positive")
600 if target_radius > 1.8 * min_receive_radius + 1e-9:
601     raise ValueError(
602         "certified 21-point cover requires target_radius / "
603         "min_receive_radius <= 1.8"
604     )
605 if boundary_margin_fraction < 0.0:
606     raise ValueError("boundary_margin_fraction must be nonnegative")
607
608 inner_radius = 0.996 * min_receive_radius
609 boundary_margin = boundary_margin_fraction * min_receive_radius
610 outer_radius = (target_radius + boundary_margin) / math.cos(math.pi / 12.0)
611 points = [np.zeros(2)]
612 points.extend(
613     inner_radius * unit(2.0 * math.pi * k / 8.0)
614     for k in range(8)
615 )
616 points.extend(
617     outer_radius * unit(2.0 * math.pi * k / 12.0)
618     for k in range(12)
619 )
620 return np.asarray(points, dtype=float)
621
622
623 def q4_compact_cover_points(
624     target_radius: float = 1800.0,
625     min_receive_radius: float = 1000.0,
626     sides: int = 12,
627     inner_radius: float = 930.0,
628     boundary_margin: float = 1.0,
629 ) -> np.ndarray:
630     """Return a compact triangulation that guarantees directional discovery.
631
632     The outer regular polygon circumscribes the target disk. Alternating inner
633     and outer vertices, together with the origin, triangulate that polygon.
634     Every edge in the default construction is shorter than the guaranteed
635     reception radius. Thus every source is positively spanned by three nearby
636     scan points, so every directional 180-degree half-plane contains one.
637     """
638     if sides < 3:
639         raise ValueError("sides must be at least 3")
640     if min_receive_radius <= 0.0:
641         raise ValueError("min_receive_radius must be positive")

```

```

642     if not (0.0 < inner_radius < min_receive_radius):
643         raise ValueError("inner_radius must lie inside the reception radius")
644     if boundary_margin < 0.0:
645         raise ValueError("boundary_margin must be nonnegative")
646
647     half_step = math.pi / sides
648     outer_radius = (target_radius + boundary_margin) / math.cos(half_step)
649     inner_edge = 2.0 * inner_radius * math.sin(half_step)
650     outer_edge = 2.0 * outer_radius * math.sin(half_step)
651     bridge = math.sqrt(
652         inner_radius * inner_radius
653         + outer_radius * outer_radius
654         - 2.0 * inner_radius * outer_radius * math.cos(half_step)
655     )
656     longest_edge = max(inner_radius, inner_edge, outer_edge, bridge)
657     if longest_edge >= min_receive_radius:
658         raise ValueError(
659             "compact cover is not guaranteed: longest triangle edge "
660             f"{longest_edge:.6f} m must be below {min_receive_radius:.6f} m"
661         )
662
663     points = [np.zeros(2)]
664     points.extend(inner_radius * unit(2.0 * math.pi * k / sides) for k in range(sides))
665     points.extend(
666         outer_radius * unit(2.0 * math.pi * (k + 0.5) / sides)
667         for k in range(sides)
668     )
669     return np.asarray(points, dtype=float)
670
671
672 def q4_compact_cover_triangles(
673     target_radius: float = 1800.0,
674     min_receive_radius: float = 1000.0,
675     sides: int = 12,
676     inner_radius: float = 930.0,
677     boundary_margin: float = 1.0,
678 ) -> list[np.ndarray]:
679     """Return the short-edge triangles underlying the compact Q4 cover."""
680     points = q4_compact_cover_points(
681         target_radius,
682         min_receive_radius,
683         sides,
684         inner_radius,
685         boundary_margin,
686     )
687     center = 0
688     inner_start = 1
689     outer_start = 1 + sides
690     triangles = []
691     for index in range(sides):
692         inner = inner_start + index
693         inner_next = inner_start + (index + 1) % sides
694         outer = outer_start + index
695         outer_previous = outer_start + (index - 1) % sides

```

```

696         triangles.extend(
697             (
698                 points[[center, inner, inner_next]],
699                 points[[inner, inner_next, outer]],
700                 points[[inner, outer_previous, outer]],
701             )
702         )
703     return triangles
704
705
706 def _source_samples(region: np.ndarray, first: np.ndarray) -> np.ndarray:
707     if len(region) == 0:
708         return region
709     samples = [p for p in region]
710     samples.extend((region[i] + region[(i + 1) % len(region)]) / 2.0 for i in range(len(region)))
711     samples.append(np.mean(region, axis=0))
712     return np.asarray([p for p in samples if np.linalg.norm(p - first) > 5.0 + 1e-6])
713
714
715 def second_point_candidate_region(
716     first: Sequence[float],
717     theta: float,
718     target_radius: float = 1800.0,
719     error_deg: float = 1.0,
720     min_receive_radius: float = 1000.0,
721     max_receive_radius: float = 1500.0,
722     grid_step: float = 40.0,
723     min_cross_angle_deg: float = 30.0,
724     min_separation: float = 100.0,
725 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
726     """Compute the robust Q2 candidate set and a quality score for each point.
727
728     A candidate must receive the source for every location compatible with the
729     first reading (using the guaranteed 1000 m radius), and its worst sampled
730     crossing angle must exceed ``min_cross_angle_deg``. Scores reward angular
731     conditioning and mildly penalize travel.
732     """
733     p = np.asarray(first, dtype=float)
734     region = bearing_region(
735         [(p, theta)],
736         target_radius=target_radius,
737         error_deg=error_deg,
738         extra_disks=[(p, max_receive_radius)],
739     )
740     if len(region) == 0:
741         return region, np.empty((0, 2)), np.empty(0)
742     samples = _source_samples(region, p)
743     lower = np.min(region, axis=0) - min_receive_radius
744     upper = np.max(region, axis=0) + min_receive_radius
745     candidates: list[np.ndarray] = []
746     scores: list[float] = []
747     for x in np.arange(lower[0], upper[0] + 0.5 * grid_step, grid_step):
748         for y in np.arange(lower[1], upper[1] + 0.5 * grid_step, grid_step):
749             q = np.array([x, y], dtype=float)

```

```

750         travel = float(np.linalg.norm(q - p))
751         if travel < min_separation:
752             continue
753         if float(np.max(np.linalg.norm(region - q, axis=1))) > min_receive_radius - 1.0:
754             continue
755         angles = []
756         for source in samples:
757             a, b = source - p, source - q
758             den = float(np.linalg.norm(a) * np.linalg.norm(b))
759             if den <= EPS:
760                 angles.append(math.pi / 2.0)
761             else:
762                 angles.append(math.asin(min(1.0, abs(cross(a, b)) / den)))
763         worst = min(angles, default=0.0)
764         if worst + 1e-12 < math.radians(min_cross_angle_deg):
765             continue
766         candidates.append(q)
767         scores.append(math.degrees(worst) - 0.002 * travel)
768     return region, np.asarray(candidates), np.asarray(scores)
769
770
771 def optimal_second_point(
772     first: Sequence[float], theta: float, target_radius: float = 1800.0, grid_step: float = 40.0
773 ) -> tuple[np.ndarray, np.ndarray]:
774     """Return the best robust second point and all feasible Q2 candidates."""
775     _, candidates, scores = second_point_candidate_region(first, theta, target_radius,
776         grid_step=grid_step)
777     if len(candidates) == 0:
778         raise ValueError("no robust second detection point exists for this reading")
779     return candidates[int(np.argmax(scores))], candidates
780
781 if __name__ == "__main__":
782     obs = [(0.0, 0.0), math.radians(20)], ((300.0, 0.0), math.radians(100))]
783     p, c, d = robust_estimate(obs)
784     print(len(p), c.center, c.radius, d)

```

B.2 offline_sim.py

```

1  """Deterministic offline simulator matching the official B-question rules.
2
3  This is not used to claim official scores. It is a regression harness for the
4  robot policy before connecting to the supplied simulator.
5  """
6  from __future__ import annotations
7
8  import json
9  import hashlib
10 import math
11 import random
12 from dataclasses import asdict, dataclass
13 from pathlib import Path
14 from typing import Any

```

```

15
16 from geometry import bearing, wrap_pi
17
18
19 ARENA_RADIUS = 1800.0
20 SPEED = 5.0
21
22
23 @dataclass
24 class Source:
25     channel: int
26     x: float
27     y: float
28     receive_radius: float
29     directional_angle: float | None = None
30     cleared: bool = False
31
32     @property
33     def position(self):
34         return (self.x, self.y)
35
36     @property
37     def directional(self):
38         return self.directional_angle is not None
39
40
41 class JammerWorld:
42     def __init__(self, sources: list[Source], seed: int = 0, error_deg: float = 1.0):
43         self.sources = sources
44         self.seed = int(seed)
45         self.rng = random.Random(seed)
46         self.error_deg = error_deg
47         self.position = (0.0, 0.0)
48         self.channel = 1
49         self.virtual_time = 0.0
50         self.entered = False
51         self.events: list[dict[str, Any]] = []
52
53     @classmethod
54     def random_case(
55         cls,
56         seed: int,
57         problem: int = 3,
58         directional_probability: float | None = None,
59         count: int | None = None,
60         receive_radius_range: tuple[float, float] = (1000.0, 1500.0),
61         boundary_bias: bool = False,
62     ) -> "JammerWorld":
63         rng = random.Random(seed)
64         count = rng.randint(10, 16) if count is None else int(count)
65         if not 10 <= count <= 16:
66             raise ValueError("source count must be in [10, 16]")
67         channels = rng.sample(range(1, 21), count)

```

```

68     directional_probability = (0.45 if problem == 4 else 0.0) if directional_probability is None
    else directional_probability
69     sources = []
70     for ch in channels:
71         radial_draw = rng.random() ** (0.15 if boundary_bias else 0.5)
72         r = radial_draw * ARENA_RADIUS
73         a = rng.random() * 2 * math.pi
74         x, y = r * math.cos(a), r * math.sin(a)
75         recv = rng.uniform(*receive_radius_range)
76         direction = None
77         if problem == 4 and rng.random() < directional_probability:
78             direction = rng.random() * 2 * math.pi
79         sources.append(Source(ch, x, y, recv, direction))
80     return cls(sources, seed=seed)
81
82     def _record(self, path: str, request: dict[str, Any], result: dict[str, Any], dt: float):
83         self.events.append({"path": path, "request": request, "response": result, "dt": dt})
84
85     def _base(self, request_id: str):
86         return {"accepted": True, "virtual_time_s": self.virtual_time, "request_id": request_id}
87
88     def enter(self, request_id: str = "enter-1"):
89         if self.entered:
90             return {"accepted": False, "virtual_time_s": 0.0}
91         self.entered = True
92         result = self._base(request_id)
93         result.update({"max_virtual_duration_s": 360000.0, "max_real_duration_s": 1200,
94             "remaining_real_duration_s": 1200})
95         self._record("/enter", {"request_id": request_id}, result, 0.0)
96         return result
97
98     def _move(self, position):
99         x, y = float(position["x"]), float(position["y"])
100         d = math.hypot(x - self.position[0], y - self.position[1])
101         self.position = (x, y)
102         return d / SPEED
103
104     def _source_visible(self, source: Source, position):
105         d = math.hypot(position["x"] - source.x, position["y"] - source.y)
106         if d > source.receive_radius + 1e-9:
107             return False, d
108         if source.directional:
109             if d <= 1e-12:
110                 return True, d
111             from_source = math.atan2(position["y"] - source.y, position["x"] - source.x)
112             if abs(wrap_pi(from_source - source.directional_angle)) > math.pi / 2 + 1e-12:
113                 return False, d
114             return True, d
115
116     def _fixed_error_deg(self, source: Source, position: dict[str, float]) -> float:
117         """Stable location-specific error, independent of Python hash randomization."""
118         key = (
119             f"{self.seed}|{source.channel}|{float(position['x']):.6f}|"
120             f"{float(position['y']):.6f}"

```

```

120         ).encode("ascii")
121         raw = int.from_bytes(hashlib.blake2b(key, digest_size=8).digest(), "big")
122         unit_value = raw / float((1 << 64) - 1)
123         return (2.0 * unit_value - 1.0) * self.error_deg
124
125     def measure(self, position: dict[str, float], channel: int, request_id: str):
126         if not self.entered:
127             return {"accepted": False, "virtual_time_s": 0.0}
128         move_dt = self._move(position)
129         switch_dt = 1.0 if int(channel) != self.channel else 0.0
130         self.channel = int(channel)
131         dt = move_dt + switch_dt + 5.0
132         self.virtual_time += dt
133         source = next((s for s in self.sources if s.channel == int(channel) and not s.cleared), None)
134         result = "no_signal"
135         response = self._base(request_id)
136         if source is not None:
137             visible, d = self._source_visible(source, position)
138             if visible:
139                 if d <= 5.0:
140                     result = "near"
141                 else:
142                     result = "direction"
143                     # A fixed error at a fixed position, as stated in the task.
144                     e = self._fixed_error_deg(source, position)
145                     true = math.atan2(source.y - float(position["y"]), source.x -
146                                     float(position["x"]))
147                     response["svd_deg"] = round((math.degrees(true + math.radians(e))) % 360.0, 2)
148         response["measure_result"] = result
149         self._record("/measure", {"position": position, "channel": channel, "request_id":
150                                 request_id}, response, dt)
151         return response
152
153     def clear(self, position: dict[str, float], channel: int, request_id: str):
154         if not self.entered:
155             return {"accepted": False, "virtual_time_s": 0.0}
156         move_dt = self._move(position)
157         source = next((s for s in self.sources if s.channel == int(channel) and not s.cleared), None)
158         ok = source is not None and math.hypot(position["x"] - source.x, position["y"] - source.y) <=
159             20.0 + 1e-9
160         action_dt = 5.0 if ok else 3.0
161         if ok:
162             source.cleared = True
163             dt = move_dt + action_dt
164             self.virtual_time += dt
165             response = self._base(request_id)
166             response["clear_result"] = "success" if ok else "no_target_in_range"
167             self._record("/clear", {"position": position, "channel": channel, "request_id": request_id},
168                             response, dt)
169             return response
170
171     def exit(self, request_id: str = "exit-1"):
172         response = self._base(request_id)
173         response["exit_reason"] = "user_exit"

```

```

170         self._record("/exit", {"request_id": request_id}, response, 0.0)
171         self.entered = False
172         return response
173
174     @property
175     def cleared_count(self):
176         return sum(s.cleared for s in self.sources)
177
178     def save(self, path: str | Path):
179         p = Path(path)
180         p.parent.mkdir(parents=True, exist_ok=True)
181         p.write_text(json.dumps({"sources": [asdict(s) for s in self.sources], "events":
182             self.events}, ensure_ascii=False, indent=2), encoding="utf-8")
183
184 if __name__ == "__main__":
185     w = JammerWorld.random_case(42, 4)
186     print(len(w.sources), sum(s.directional for s in w.sources))

```

B.3 robot.py

```

1  """Guaranteed-coverage search and bounded-error localization for CUMCM 2026 B.
2
3  The same policy runs against the supplied HTTP simulator and the deterministic
4  offline regression world. New actions are strictly serial, and a network
5  retry reuses the complete original payload and request_id.
6  """
7  from __future__ import annotations
8
9  import argparse
10 import hashlib
11 import http.client
12 import json
13 import math
14 import socket
15 import threading
16 import time
17 import urllib.error
18 import urllib.request
19 from contextlib import contextmanager
20 from dataclasses import asdict, dataclass
21 from pathlib import Path
22 from typing import Any, Sequence
23
24 import numpy as np
25
26 from geometry import (
27     EPS,
28     bearing_region,
29     clip_convex_polygon_halfplane,
30     convex_hull,
31     cross,
32     diameter,

```

```

33     intersect_convex_polygons,
34     min_enclosing_circle,
35     point_in_convex_polygon,
36     point_polygon_distance,
37     q3_cover_points,
38     q3_minimax_ring_points,
39     q3_ring_cover_points,
40     q3_custom_ring_points,
41     q4_certified21_cover_points,
42     q4_compact_cover_points,
43     q4_compact_cover_triangles,
44     q4_triangular_cover_points,
45     unit,
46 )
47
48
49 @dataclass
50 class Observation:
51     position: tuple[float, float]
52     channel: int
53     result: str
54     angle_deg: float | None
55     virtual_time: float
56
57
58 @dataclass
59 class RobotConfig:
60     problem: int = 3
61     target_radius: float = 1800.0
62     max_error_deg: float = 1.01
63     min_receive_radius: float = 1000.0
64     max_receive_radius: float = 1500.0
65     clear_radius: float = 20.0
66     clear_guarantee_radius: float = 19.5
67     q3_ring_radius: float = 1150.0
68     q3_custom_ring: bool = False
69     q3_include_center: bool = True
70     q3_layout: str = "minimax"
71     q3_ring_sides: int = 7
72     q3_min_discovery_directions: int = 2
73     q3_view_travel_weight: float = 0.2
74     q3_entry_candidates: bool = True
75     q3_information_floor_fraction: float = 0.1
76     # For an omnidirectional source, a positive/negative pair on one channel
77     # gives the valid ordering d(positive, source) < d(negative, source).
78     # Kept as an explicit experiment switch; the conservative default is off.
79     q3_range_order_constraints: bool = False
80     q3_estimated_clear_radius: float = 100.0
81     q3_estimated_clear_attempts: int = 1
82     q3_measure_after_failed_estimate: bool = True
83     q3_scan_estimated_clear: bool = True
84     q3_scan_clear_max_detour: float = 150.0
85     q3_multistart_localization_route: bool = True
86     q3_skip_impossible_rechecks: bool = True

```

```

87 # Independently switchable experiments; defaults are selected on held-out cases.
88 q3_service_region: bool = True
89 q3_joint_time_routing: bool = False
90 q3_joint_candidate_limit: int = 4
91 budget_aware: bool = True
92 exit_reserve_s: float = 2.0
93 planning_guard_s: float = 30.0
94 q4_mesh_edge: float = 980.0
95 q4_layout: str = "compact"
96 # A directional source may be visible from only one mesh vertex. During
97 # Q4 discovery keeps a found channel active until enough positive bearings
98 # are available. Three is the measured time optimum in the matched-case
99 # benchmark; the deterministic fallback remains valid when fewer are seen.
100 q4_min_discovery_directions: int = 3
101 q4_symmetric_recovery: bool = True
102 q4_recovery_rounds: int = 6
103 q4_information_floor_fraction: float = 0.10
104 q4_integrated_scan_clear: bool = False
105 q4_speculative_clear_radius: float = 19.5
106 q4_skip_impossible_rechecks: bool = True
107 q4_local_positive_cap: bool = True
108 q4_local_triangle_support: bool = False
109 q4_positive_hull_views: bool = False
110 q4_estimated_clear_radius: float = 100.0
111 q4_estimated_clear_attempts: int = 2
112 q4_estimator: str = "pairwise"
113 q4_measure_after_failed_estimate: bool = True
114 q4_shadow_bisection: bool = True
115 q4_shadow_bisection_rounds: int = 4
116 q4_scan_estimated_clear: bool = True
117 q4_scan_clear_max_detour: float = 200.0
118 q4_scan_shadow_recovery: bool = False
119 q4_scan_clear_lookahead: bool = False
120 q4_scan_clear_success_probability: float = 0.90
121 q4_adaptive_outer_route: bool = False
122 q4_estimate_route_centers: bool = False
123 q4_multistart_localization_route: bool = False
124 q4_one_bearing_posterior: str = "off"
125 q4_one_bearing_initial_probe: bool = True
126 # Stop collecting additional bearings as soon as the robust feasible
127 # region is small enough for a guaranteed clear. The fixed discovery
128 # count remains the minimum search-stage requirement.
129 adaptive_clear_stop: bool = True
130 max_source_count: int = 16
131 max_view_attempts: int = 8
132 max_actions: int = 10000
133 real_time_reserve_s: float = 3.0
134 # Q4 refinements are separately switchable for matched ablation tests.
135 q4_joint_constraints: bool = True
136 q4_time_lookahead: bool = True
137 q4_safe_clear_route: bool = True
138 q4_planning_budget_s: float = 1.0
139 q4_constraint_cells: int = 96
140

```

```

141
142 class RuntimeBudgetExceeded(TimeoutError):
143     """The remaining real-time budget is reserved for orderly exit."""
144
145
146 class HttpClient:
147     """Strict serial JSON client with protocol-correct idempotent retries."""
148
149     def __init__(self, base_url: str, robot_id: str, log_path: str | Path):
150         self.base_url = base_url.rstrip("/")
151         self.robot_id = robot_id
152         self.log_path = Path(log_path)
153         self.log_path.parent.mkdir(parents=True, exist_ok=True)
154         self.counter = 0
155         self.run_tag = str(int(time.time() * 1000))
156         self.last_response: dict[str, Any] | None = None
157         self.deadline: float | None = None
158         self.exit_reserve_s = 0.0
159
160     def set_deadline(self, deadline: float | None, reserve_s: float = 0.0) -> None:
161         self.deadline = deadline
162         self.exit_reserve_s = max(0.0, reserve_s)
163
164     def _remaining(self, path: str) -> float:
165         if self.deadline is None:
166             return math.inf
167         reserve = 0.0 if path == "/exit" else self.exit_reserve_s
168         remaining = self.deadline - reserve - time.monotonic()
169         if remaining <= 0.02:
170             raise RuntimeBudgetExceeded(f"real-time budget exhausted before {path}")
171         return remaining
172
173     @contextmanager
174     def _open_response(self, request, timeout, path):
175         if self.deadline is None:
176             with urllib.request.urlopen(request, timeout=timeout) as response:
177                 yield response
178             return
179
180         deadline = self.deadline - (0.0 if path == "/exit" else self.exit_reserve_s)
181         state = {"socket": None, "expired": False}
182         lock = threading.Lock()
183
184         def expire():
185             with lock:
186                 state["expired"] = True
187                 connected = state["socket"]
188                 if connected is not None:
189                     try:
190                         connected.shutdown(socket.SHUT_RDWR)
191                     except OSError:
192                         pass
193
194         def factory(connection_type):

```

```

195     def create(*args, **kwargs):
196         connection = connection_type(*args, **kwargs)
197         original_connect = connection._create_connection
198
199         def connect(*connect_args, **connect_kwargs):
200             connected = original_connect(*connect_args, **connect_kwargs)
201             with lock:
202                 if state["expired"] or time.monotonic() >= deadline:
203                     connected.close()
204                     raise RuntimeBudgetExceeded(
205                         "request deadline expired during connect"
206                     )
207                 old = state["socket"]
208                 if old is not None:
209                     old.close()
210                 state["socket"] = connected.dup()
211             return connected
212
213         connection._create_connection = connect
214         return connection
215
216     return create
217
218     http_factory = factory(http.client.HTTPConnection)
219     https_factory = factory(http.client.HTTPSConnection)
220
221     class TimedHTTPHandler(urllib.request.HTTPHandler):
222         def http_open(self, req):
223             return self.do_open(http_factory, req)
224
225     class TimedHTTPSHandler(urllib.request.HTTPSHandler):
226         def https_open(self, req):
227             return self.do_open(https_factory, req, context=self._context)
228
229     timer = threading.Timer(max(0.0, deadline - time.monotonic()), expire)
230     timer.daemon = True
231     timer.start()
232     try:
233         opener = urllib.request.build_opener(TimedHTTPHandler(), TimedHTTPSHandler())
234         with opener.open(request, timeout=timeout) as response:
235             yield response
236             if state["expired"]:
237                 raise RuntimeBudgetExceeded(
238                     "request exceeded its absolute deadline"
239                 )
240     except Exception as error:
241         if state["expired"]:
242             raise RuntimeBudgetExceeded(
243                 "request exceeded its absolute deadline"
244             ) from error
245         raise
246     finally:
247         timer.cancel()
248         timer.join()

```

```

249         with lock:
250             if state["socket"] is not None:
251                 state["socket"].close()
252
253     def _write_log(self, record: dict[str, Any]) -> None:
254         record = dict(record)
255         if "request" in record:
256             request = dict(record["request"])
257             if "robot_id" in request:
258                 request["robot_id"] = "<ROBOT_ID>"
259             record["request"] = request
260         with self.log_path.open("a", encoding="utf-8") as stream:
261             stream.write(json.dumps(record, ensure_ascii=False, separators=(",", ":")) + "\n")
262
263     def post(self, path: str, body: dict[str, Any], retries: int = 4):
264         if retries < 1:
265             raise ValueError("retries must be at least one")
266         self.counter += 1
267         request_id = body.get("request_id") or f"{path.strip('/')}-{self.run_tag}-{self.counter}"
268         payload = {"arena_id": "default", "robot_id": self.robot_id, **body, "request_id":
269             request_id}
270         encoded = json.dumps(payload, ensure_ascii=False, separators=(",", ":")).encode("utf-8")
271
272         for attempt in range(1, retries + 1):
273             timeout = min(8.0, self._remaining(path) - 0.01)
274             request = urllib.request.Request(
275                 self.base_url + path,
276                 data=encoded,
277                 headers={"Content-Type": "application/json"},
278                 method="POST",
279             )
280             try:
281                 with self._open_response(request, timeout, path) as response_stream:
282                     response = json.loads(response_stream.read().decode("utf-8"))
283                     status = int(response_stream.status)
284                     self.last_response = response
285                     self._write_log(
286                         {"wall_time": time.time(), "path": path, "attempt": attempt, "http_status":
287                             status,
288                             "request": payload, "response": response}
289                     )
290                     return response
291             except urllib.error.HTTPError as error:
292                 response = {"http_status": error.code, "reason": str(error.reason)}
293                 error.close()
294                 self._write_log(
295                     {"wall_time": time.time(), "path": path, "attempt": attempt, "http_status":
296                         error.code,
297                         "request": payload, "response": response}
298                     )
299                 raise RuntimeError(f"HTTP {error.code} for {path}: {response}") from error
300             except (
301                 urllib.error.URLError,
302                 TimeoutError,

```

```

300         ConnectionError,
301         http.client.IncompleteRead,
302         http.client.RemoteDisconnected,
303     ) as error:
304         self._write_log(
305             {"wall_time": time.time(), "path": path, "attempt": attempt,
306              "request": payload, "transport_error": repr(error)}
307         )
308         if attempt == retries:
309             raise
310         delay = 0.2 * attempt
311         if self._remaining(path) <= delay + 0.02:
312             raise RuntimeBudgetExceeded("insufficient budget for retry") from error
313         time.sleep(delay)
314     raise RuntimeError("unreachable retry state")
315
316
317 def _distance(a: Sequence[float], b: Sequence[float]) -> float:
318     return math.hypot(float(a[0]) - float(b[0]), float(a[1]) - float(b[1]))
319
320
321 def _open_route(points: Sequence[Sequence[float]], start: Sequence[float] = (0.0, 0.0)) ->
322     list[tuple[float, float]]:
323     """Nearest-neighbour route followed by deterministic open-path 2-opt."""
324     remaining = [tuple(map(float, point)) for point in points]
325     if not remaining:
326         return []
327     first_index = min(range(len(remaining)), key=lambda i: (_distance(start, remaining[i]),
328                                                             remaining[i]))
329     route = [remaining.pop(first_index)]
330     while remaining:
331         index = min(range(len(remaining)), key=lambda i: (_distance(route[-1], remaining[i]),
332                                                             remaining[i]))
333         route.append(remaining.pop(index))
334
335     improved = True
336     passes = 0
337     while improved and passes < 12:
338         improved = False
339         passes += 1
340         for i in range(0, len(route) - 1):
341             for j in range(i + 1, len(route)):
342                 left = start if i == 0 else route[i - 1]
343                 before = _distance(left, route[i])
344                 after = _distance(left, route[j])
345                 if j + 1 < len(route):
346                     before += _distance(route[j], route[j + 1])
347                     after += _distance(route[i], route[j + 1])
348                 if after + 1e-8 < before:
349                     route[i:j + 1] = reversed(route[i:j + 1])
350                     improved = True
351     return route

```

```

351 def _route_length(points: Sequence[Sequence[float]], start: Sequence[float]) -> float:
352     route = _open_route(points, start=start)
353     return sum(
354         _distance(start if index == 0 else route[index - 1], point)
355         for index, point in enumerate(route)
356     )
357
358
359 def _joint_route_first(
360     tasks: Sequence[tuple[str, int, Sequence[float]]],
361     start: Sequence[float],
362 ) -> int:
363     """Return the first task in a deterministic open-path route."""
364     if not tasks:
365         raise ValueError("at least one route task is required")
366     points = [tuple(map(float, task[2])) for task in tasks]
367     remaining = list(range(len(tasks)))
368
369     def tie_key(index: int) -> tuple[float, str, int]:
370         return (_distance(start, points[index]), tasks[index][0], tasks[index][1])
371
372     first = min(remaining, key=tie_key)
373     route = [first]
374     remaining.remove(first)
375     while remaining:
376         previous = points[route[-1]]
377         chosen = min(
378             remaining,
379             key=lambda index: (
380                 _distance(previous, points[index]),
381                 tasks[index][0],
382                 tasks[index][1],
383             ),
384         )
385         route.append(chosen)
386         remaining.remove(chosen)
387
388     improved = True
389     passes = 0
390     while improved and passes < 12:
391         improved = False
392         passes += 1
393         for i in range(0, len(route) - 1):
394             for j in range(i + 1, len(route)):
395                 left = start if i == 0 else points[route[i - 1]]
396                 before = _distance(left, points[route[i]])
397                 after = _distance(left, points[route[j]])
398                 if j + 1 < len(route):
399                     before += _distance(points[route[j]], points[route[j + 1]])
400                     after += _distance(points[route[i]], points[route[j + 1]])
401                 if after + 1e-8 < before:
402                     route[i:j + 1] = reversed(route[i:j + 1])
403                     improved = True
404     return route[0]

```

```

405
406
407 def _multistart_open_order(
408     points: dict[int, np.ndarray],
409     start: Sequence[float],
410 ) -> list[int]:
411     """Build a deterministic 2-opt open route from every possible first stop."""
412     if not points:
413         return []
414     start_point = np.asarray(start, dtype=float)
415
416     def distance(left: int | None, right: int) -> float:
417         origin = start_point if left is None else points[left]
418         return float(np.linalg.norm(origin - points[right]))
419
420     def improve(route: list[int]) -> list[int]:
421         changed = True
422         passes = 0
423         while changed and passes < 12:
424             changed = False
425             passes += 1
426             for left_index in range(len(route) - 1):
427                 for right_index in range(left_index + 1, len(route)):
428                     left = None if left_index == 0 else route[left_index - 1]
429                     before = distance(left, route[left_index])
430                     after = distance(left, route[right_index])
431                     if right_index + 1 < len(route):
432                         next_stop = route[right_index + 1]
433                         before += distance(route[right_index], next_stop)
434                         after += distance(route[left_index], next_stop)
435                     if after + 1e-8 < before:
436                         route[left_index : right_index + 1] = reversed(
437                             route[left_index : right_index + 1]
438                         )
439                     changed = True
440         return route
441
442     def route_length(route: list[int]) -> float:
443         return sum(
444             distance(None if index == 0 else route[index - 1], stop)
445             for index, stop in enumerate(route)
446         )
447
448     routes = []
449     for first in sorted(points):
450         route = [first]
451         remaining = set(points) - {first}
452         while remaining:
453             chosen = min(
454                 remaining,
455                 key=lambda stop: (distance(route[-1], stop), stop),
456             )
457             route.append(chosen)
458             remaining.remove(chosen)

```

```

459         improved = improve(route)
460         routes.append((route_length(improved), improved))
461     return min(routes, key=lambda item: (item[0], item[1]))[1]
462
463
464 class Robot:
465     def __init__(self, client: Any, config: RobotConfig, result_dir: str | Path):
466         self.client = client
467         self.cfg = config
468         self.result_dir = Path(result_dir)
469         self.result_dir.mkdir(parents=True, exist_ok=True)
470         self.position = (0.0, 0.0)
471         self.channel = 1
472         self.obs: list[Observation] = []
473         self.discovered: set[int] = set()
474         self.cleared: set[int] = set()
475         self.absent: set[int] = set()
476         self.action_count = 0
477         self.last_virtual_time = 0.0
478         self.failed_clear_points: dict[int, list[tuple[float, float]]] = {}
479         self.region_halfplanes: dict[int, list[tuple[np.ndarray, float]]] = {}
480         self.local_positive_cap_channels: set[int] = set()
481         self.discovery_support_polygons: dict[int, np.ndarray] = {}
482         self.estimated_clear_attempts: dict[int, int] = {}
483         self.estimated_clear_bearing_counts: dict[int, int] = {}
484         self.one_bearing_posterior_attempted: set[int] = set()
485         self.start_wall_time = 0.0
486         self.remaining_real_duration_s: float | None = None
487         self.real_deadline: float | None = None
488         self.exit_reserve_s = 0.0
489         self.next_task_hint: np.ndarray | None = None
490         self.refinement_stats = {
491             "constraint_updates": 0,
492             "constraint_shrinks": 0,
493             "lookahead_choices": 0,
494             "safe_clear_choices": 0,
495             "planning_fallbacks": 0,
496         }
497         self._constraint_cache: dict[int, tuple[Any, np.ndarray]] = {}
498         self.deadline_monotonic: float | None = None
499         self.inferred_no_signal: dict[int, set[tuple[float, float]]] = {}
500         self.skipped_rechecks = 0
501         self.region_cache_hits = 0
502         self.region_cache_misses = 0
503         self._region_cache: dict[int, tuple[Any, Any]] = {}
504         self.budget_degraded = False
505         self.discovery_complete = False
506         self.service_point_uses = 0
507         self.joint_order_changes = 0
508         self._q3_following_hint: np.ndarray | None = None
509         self.distance_m = 0.0
510         self.measure_count = 0
511         self.switch_count = 0
512         self.clear_success_count = 0

```

```

513         self.clear_failure_count = 0
514
515     def _check_budget(self) -> None:
516         if (
517             self.cfg.budget_aware
518             and self.real_deadline is not None
519             and time.monotonic() >= self.real_deadline - self.exit_reserve_s
520         ):
521             raise RuntimeBudgetExceeded("real-time action budget exhausted; exiting")
522
523     def _planning_deadline(self) -> float:
524         self._check_budget()
525         deadline = time.monotonic() + max(0.001, self.cfg.q4_planning_budget_s)
526         if self.cfg.budget_aware and self.real_deadline is not None:
527             deadline = min(deadline, self.real_deadline - self.exit_reserve_s)
528         return deadline
529
530     def _remaining_wall_time(self) -> float:
531         if self.deadline_monotonic is None:
532             return math.inf
533         return max(0.0, self.deadline_monotonic - time.monotonic())
534
535     def _low_budget(self) -> bool:
536         low = self.cfg.budget_aware and self._remaining_wall_time() <= self.cfg.planning_guard_s
537         self.budget_degraded = self.budget_degraded or low
538         return low
539
540     @property
541     def deadline_monotonic(self) -> float | None:
542         """Compatibility alias for the shared absolute real-time deadline."""
543         return self.real_deadline
544
545     @deadline_monotonic.setter
546     def deadline_monotonic(self, value: float | None) -> None:
547         self.real_deadline = value
548
549     def _post(self, path: str, **body):
550         if path != "/exit":
551             self._check_budget()
552         if path != "/exit" and self.action_count >= max(1, self.cfg.max_actions - 1):
553             raise RuntimeError(f"action limit {self.cfg.max_actions} reached; exit reserved")
554         self.action_count += 1
555         response = self.client.post(path, body)
556         if response.get("accepted") is not True:
557             raise RuntimeError(f"rejected {path}: {response}")
558         self.last_virtual_time = float(response.get("virtual_time_s", self.last_virtual_time))
559         if "position" in body:
560             new_position = (float(body["position"]["x"]), float(body["position"]["y"]))
561             self.distance_m += _distance(self.position, new_position)
562             self.position = new_position
563         if path == "/clear":
564             self.clear_success_count += int(response.get("clear_result") == "success")
565             self.clear_failure_count += int(response.get("clear_result") != "success")
566         if path == "/measure":

```

```

567         self.measure_count += 1
568         self.switch_count += int(self.channel != int(body["channel"]))
569         self.channel = int(body["channel"])
570         self.obs.append(
571             Observation(
572                 self.position,
573                 self.channel,
574                 str(response["measure_result"]),
575                 float(response["svd_deg"]) if "svd_deg" in response else None,
576                 self.last_virtual_time,
577             )
578         )
579         return response
580
581     def measure(self, position: Sequence[float], channel: int):
582         return self._post(
583             "/measure",
584             position={"x": float(position[0]), "y": float(position[1])},
585             channel=int(channel),
586         )
587
588     def clear(self, position: Sequence[float], channel: int):
589         point = (float(position[0]), float(position[1]))
590         response = self._post(
591             "/clear", position={"x": point[0], "y": point[1]}, channel=int(channel)
592         )
593         if response.get("clear_result") == "success":
594             self.cleared.add(int(channel))
595         else:
596             self.failed_clear_points.setdefault(int(channel), []).append(point)
597         return response
598
599     def _scan_points(self) -> list[tuple[float, float]]:
600         if self.cfg.problem == 3:
601             if self.cfg.q3_custom_ring:
602                 points = q3_custom_ring_points(
603                     self.cfg.q3_ring_radius,
604                     self.cfg.q3_ring_sides,
605                     self.cfg.q3_include_center,
606                 )
607             elif self.cfg.q3_layout == "legacy":
608                 points = q3_cover_points(self.cfg.q3_ring_radius)
609             elif self.cfg.q3_layout == "ring":
610                 points = q3_ring_cover_points(
611                     self.cfg.target_radius,
612                     self.cfg.min_receive_radius,
613                     self.cfg.q3_ring_sides,
614                 )
615             elif self.cfg.q3_layout == "minimax":
616                 points = q3_minimax_ring_points(
617                     self.cfg.target_radius,
618                     self.cfg.min_receive_radius,
619                     self.cfg.q3_ring_sides,
620                 )

```

```

621         else:
622             raise ValueError(f"unknown Q3 scan layout: {self.cfg.q3_layout}")
623         return _open_route(points, start=(0.0, 0.0))
624     if self.cfg.q4_layout in {"compact", "certified21"}:
625         mesh = q4_certified21_cover_points(
626             self.cfg.target_radius,
627             self.cfg.min_receive_radius,
628         )
629     elif self.cfg.q4_layout == "compact25":
630         mesh = q4_compact_cover_points(
631             self.cfg.target_radius,
632             self.cfg.min_receive_radius,
633         )
634     elif self.cfg.q4_layout == "legacy":
635         mesh = q4_triangular_cover_points(self.cfg.target_radius, self.cfg.q4_mesh_edge)
636     else:
637         raise ValueError(f"unknown Q4 scan layout: {self.cfg.q4_layout}")
638     return _open_route(mesh, start=(0.0, 0.0))
639
640     def _channel_order(self, channels: set[int]) -> list[int]:
641         if self.channel in channels:
642             return [self.channel] + sorted(channels - {self.channel})
643         return sorted(channels)
644
645     def _direction_observations(self, channel: int) -> list[Observation]:
646         return [o for o in self.obs if o.channel == channel and o.angle_deg is not None]
647
648     def _region_for(self, channel: int) -> tuple[np.ndarray, float, Any]:
649         """Cache by all constraint inputs, never by observation count alone."""
650         support = self.discovery_support_polygons.get(channel)
651         key = (
652             tuple((o.position, o.result, o.angle_deg) for o in self.obs if o.channel == channel),
653             tuple(sorted(self.inferred_no_signal.get(channel, set()))),
654             tuple((tuple(np.asarray(a, dtype=float)), float(b))
655                  for a, b in self.region_halfplanes.get(channel, [])),
656             None if support is None else np.asarray(support, dtype=float).tobytes(),
657             channel in self.local_positive_cap_channels,
658             self.cfg.problem, self.cfg.q3_range_order_constraints,
659             self.cfg.target_radius, self.cfg.max_error_deg,
660             self.cfg.min_receive_radius, self.cfg.max_receive_radius,
661             tuple(self.failed_clear_points.get(channel, [])),
662             self.cfg.q4_joint_constraints, self.cfg.q4_constraint_cells,
663             self.cfg.q4_planning_budget_s, self.cfg.clear_radius,
664         )
665         cached = self._region_cache.get(channel)
666         if cached is not None and cached[0] == key:
667             self.region_cache_hits += 1
668             return cached[1]
669         self.region_cache_misses += 1
670         value = self._compute_region_for(channel)
671         self._region_cache[channel] = (key, value)
672         return value
673
674     def _compute_region_for(self, channel: int) -> tuple[np.ndarray, float, Any]:

```

```

675 observations = self._direction_observations(channel)
676 if not observations:
677     return np.empty((0, 2)), math.inf, None
678 bearing_data = [(o.position, math.radians(float(o.angle_deg))) for o in observations]
679 if channel in self.local_positive_cap_channels:
680     components = []
681     for capped_index in range(len(observations)):
682         receive_disks = [
683             (
684                 observation.position,
685                 self.cfg.min_receive_radius
686                 if index == capped_index
687                 else self.cfg.max_receive_radius,
688             )
689             for index, observation in enumerate(observations)
690         ]
691         component = bearing_region(
692             bearing_data,
693             target_radius=self.cfg.target_radius,
694             error_deg=self.cfg.max_error_deg,
695             extra_disks=receive_disks,
696             n_disk=360,
697         )
698         if len(component):
699             components.append(component)
700         region = (
701             convex_hull(np.vstack(components))
702             if components
703             else np.empty((0, 2))
704         )
705     else:
706         receive_disks = [
707             (o.position, self.cfg.max_receive_radius)
708             for o in observations
709         ]
710         region = bearing_region(
711             bearing_data,
712             target_radius=self.cfg.target_radius,
713             error_deg=self.cfg.max_error_deg,
714             extra_disks=receive_disks,
715             n_disk=360,
716         )
717 support = self.discovery_support_polygons.get(channel)
718 if support is not None and len(support):
719     region = intersect_convex_polygons(region, support)
720 if self.cfg.problem == 3 and self.cfg.q3_range_order_constraints:
721     # With an omnidirectional source and one fixed reception radius,
722     # every positive observer p is closer than every no-signal
723     # observer n. Squared distances give the exact half-plane below.
724     positive_points = [
725         np.asarray(observation.position, dtype=float)
726         for observation in self.obs
727         if observation.channel == channel
728         and observation.result in {"direction", "near"}

```

```

729     ]
730     negative_points = [
731         np.asarray(observation.position, dtype=float)
732         for observation in self.obs
733         if observation.channel == channel
734         and observation.result == "no_signal"
735     ]
736     # These are proved no-signal locations, not fabricated observations.
737     negative_points.extend(np.asarray(point, dtype=float)
738                           for point in sorted(self.inferred_no_signal.get(channel, set())))
739     for positive in positive_points:
740         for negative in negative_points:
741             normal = positive - negative
742             bound = 0.5 * float(
743                 np.dot(positive, positive) - np.dot(negative, negative)
744             )
745             region = clip_convex_polygon_halfplane(region, normal, bound)
746             if len(region) == 0:
747                 return region, math.inf, None
748     for normal, bound in self.region_halfplanes.get(channel, []):
749         region = clip_convex_polygon_halfplane(region, normal, bound)
750         if len(region) == 0:
751             return region, math.inf, None
752     if len(region) == 0:
753         return region, math.inf, None
754     if self.cfg.problem == 4 and self.cfg.q4_joint_constraints and len(region) >= 3:
755         from q4_refinement import refine_region
756
757         relevant = [o for o in self.obs if o.channel == channel]
758         failed = self.failed_clear_points.get(channel, [])
759         signature = (
760             tuple((o.position, o.result, o.angle_deg) for o in relevant),
761             tuple(failed),
762             region.tobytes(),
763             self.cfg.min_receive_radius,
764             self.cfg.max_receive_radius,
765             self.cfg.clear_radius,
766             self.cfg.q4_constraint_cells,
767             self.cfg.q4_planning_budget_s,
768         )
769         cached = self._constraint_cache.get(channel)
770         if cached is not None and cached[0] == signature:
771             region = cached[1].copy()
772         else:
773             refined = refine_region(
774                 region,
775                 relevant,
776                 failed,
777                 min_radius=self.cfg.min_receive_radius,
778                 max_radius=self.cfg.max_receive_radius,
779                 clear_radius=self.cfg.clear_radius,
780                 max_cells=self.cfg.q4_constraint_cells,
781                 deadline=self._planning_deadline(),
782             )

```

```

783         self.refinement_stats["constraint_updates"] += 1
784         if refined.timed_out:
785             self.refinement_stats["planning_fallbacks"] += 1
786         elif refined.rejected_cells:
787             self.refinement_stats["constraint_shrinks"] += 1
788             region = refined.region
789             self._constraint_cache[channel] = (signature, region.copy())
790         circle = min_enclosing_circle(region)
791         return region, diameter(region), circle
792
793     def _view_candidates(
794         self,
795         channel: int,
796         region: np.ndarray,
797         prefer_travel: bool = True,
798     ) -> list[tuple[float, float]]:
799         candidates = self._geometry_view_candidates(channel, region, prefer_travel)
800         if self.cfg.problem != 4 or not self.cfg.q4_time_lookahead or not candidates:
801             return candidates
802         from q4_refinement import rank_measurements
803
804         ranked, changed = rank_measurements(
805             candidates,
806             region,
807             [o for o in self.obs if o.channel == channel],
808             self.position,
809             self.failed_clear_points.get(channel, []),
810             error_deg=self.cfg.max_error_deg,
811             switch_cost=float(self.channel != channel),
812             deadline=self._planning_deadline(),
813         )
814         self.refinement_stats["lookahead_choices"] += int(changed)
815         return ranked
816
817     def _geometry_view_candidates(
818         self,
819         channel: int,
820         region: np.ndarray,
821         prefer_travel: bool = True,
822     ) -> list[tuple[float, float]]:
823         observations = self._direction_observations(channel)
824         if not observations or len(region) == 0:
825             return []
826         tried = [np.asarray(o.position, dtype=float) for o in self.obs if o.channel == channel]
827         vertices = np.asarray(region, dtype=float)
828         samples = [p for p in vertices]
829         samples.extend((vertices[i] + vertices[(i + 1) % len(vertices)]) / 2.0 for i in
830             range(len(vertices)))
831         samples.append(np.mean(vertices, axis=0))
832         samples_array = np.asarray(samples)
833
834         raw: list[np.ndarray] = []
835         guaranteed_positive: list[np.ndarray] = []
836         circle = min_enclosing_circle(vertices)

```

```

836 raw.append(circle.center.copy())
837 if len(observations) == 1:
838     first = observations[0]
839     p = np.asarray(first.position, dtype=float)
840     u = unit(math.radians(float(first.angle_deg)))
841     normal = np.array([-u[1], u[0]])
842     projection = (vertices - p) @ u
843     for fraction in (0.42, 0.50, 0.58):
844         anchor = p + (float(np.min(projection)) + fraction * float(np.ptp(projection))) * u
845         for offset in (650.0, 600.0, 550.0, 475.0, 400.0, 300.0, 200.0):
846             raw.extend((anchor + offset * normal, anchor - offset * normal))
847
848 # A reception disk and a directional emitting half-plane are both
849 # convex. Therefore every convex combination of two positive
850 # observers is guaranteed to remain receivable for the true source,
851 # even when its actual reception radius is greater than the known
852 # 1000 m lower bound.
853 if self.cfg.problem == 4 and self.cfg.q4_positive_hull_views and len(observations) >= 2:
854     for left_index in range(len(existing_positions := [np.asarray(o.position, dtype=float)
855     for o in observations])):
856         for right_index in range(left_index + 1, len(existing_positions)):
857             left = existing_positions[left_index]
858             right = existing_positions[right_index]
859             for fraction in (0.20, 0.35, 0.50, 0.65, 0.80):
860                 guaranteed_positive.append((1.0 - fraction) * left + fraction * right)
861             guaranteed_positive.append(np.mean(existing_positions, axis=0))
862             raw.extend(guaranteed_positive)
863
864 for radius in (120.0, 250.0, 450.0, 650.0, 850.0):
865     for k in range(24):
866         raw.append(circle.center + radius * unit(2.0 * math.pi * k / 24.0))
867
868 current = np.asarray(self.position, dtype=float)
869 reception_limit = self.cfg.min_receive_radius - 2.0
870 if self.cfg.problem == 3 and self.cfg.q3_entry_candidates:
871     feasible_seeds = [
872         candidate
873         for candidate in raw
874         if float(np.max(np.linalg.norm(vertices - candidate, axis=1)))
875         <= reception_limit
876     ]
877     current_max_distance = float(
878         np.max(np.linalg.norm(vertices - current, axis=1))
879     )
880     if current_max_distance <= reception_limit:
881         raw.append(current.copy())
882     else:
883         # The guaranteed-reception set is convex. Enter it along
884         # informative rays instead of travelling to their far ends.
885         for target in feasible_seeds:
886             low, high = 0.0, 1.0
887             for _ in range(36):
888                 fraction = 0.5 * (low + high)
889                 candidate = current + fraction * (target - current)

```

```

889         maximum = float(
890             np.max(np.linalg.norm(vertices - candidate, axis=1))
891         )
892         if maximum <= reception_limit:
893             high = fraction
894         else:
895             low = fraction
896         fraction = min(1.0, high + 1e-8)
897         raw.append(current + fraction * (target - current))
898
899     unique: list[np.ndarray] = []
900     for candidate in raw:
901         if any(np.linalg.norm(candidate - prior) < 1.0 for prior in tried):
902             continue
903         if any(np.linalg.norm(candidate - prior) < 1e-6 for prior in unique):
904             continue
905         safe_by_positive_hull = any(
906             np.linalg.norm(candidate - safe) < 1e-6
907             for safe in guaranteed_positive
908         )
909         if (
910             not safe_by_positive_hull
911             and float(np.max(np.linalg.norm(vertices - candidate, axis=1))) > reception_limit
912         ):
913             continue
914         unique.append(candidate)
915
916     if self._low_budget():
917         # The candidate feasibility tests remain unchanged; only expensive
918         # information ranking is replaced by nearest certified reception.
919         unique.sort(key=lambda p: (_distance(p, self.position), float(p[0]), float(p[1])))
920         return [tuple(map(float, p)) for p in unique]
921
922     existing_positions = [np.asarray(o.position, dtype=float) for o in observations]
923
924     def information_score(candidate: np.ndarray) -> float:
925         worst_information = math.inf
926         for source in samples_array:
927             information = np.zeros((2, 2), dtype=float)
928             for observer in existing_positions + [candidate]:
929                 vector = source - observer
930                 distance_value = max(5.0, float(np.linalg.norm(vector)))
931                 normal = np.array([-vector[1], vector[0]]) / distance_value
932                 information += np.outer(normal, normal) / (distance_value * distance_value)
933                 worst_information = min(worst_information, float(np.linalg.eigvalsh(information)[0]))
934             return max(0.0, worst_information)
935
936     scored = [
937         (
938             point,
939             information_score(point),
940             float(np.linalg.norm(point - np.asarray(self.position))),
941         )
942         for point in unique

```

```

943 ]
944 if self.cfg.problem == 3 and self.cfg.q3_information_floor_fraction > 0.0:
945     best_information = max((item[1] for item in scored), default=0.0)
946     floor = self.cfg.q3_information_floor_fraction * best_information
947     eligible = [item for item in scored if item[1] + EPS >= floor]
948     rejected = [item for item in scored if item[1] + EPS < floor]
949     eligible.sort(key=lambda item: (item[2], -item[1], item[0][0], item[0][1]))
950     rejected.sort(key=lambda item: (-item[1], item[2], item[0][0], item[0][1]))
951     return [tuple(map(float, item[0])) for item in eligible + rejected]
952
953 if (
954     self.cfg.problem == 4
955     and prefer_travel
956     and self.cfg.q4_information_floor_fraction > 0.0
957 ):
958     best_information = max((item[1] for item in scored), default=0.0)
959     floor = self.cfg.q4_information_floor_fraction * best_information
960     eligible = [item for item in scored if item[1] + EPS >= floor]
961     rejected = [item for item in scored if item[1] + EPS < floor]
962     eligible.sort(key=lambda item: (item[2], -item[1], item[0][0], item[0][1]))
963     rejected.sort(key=lambda item: (-item[1], item[2], item[0][0], item[0][1]))
964     return [tuple(map(float, item[0])) for item in eligible + rejected]
965
966 def score(item: tuple[np.ndarray, float, float]) -> float:
967     _, worst_information, travel = item
968     travel_weight = (
969         self.cfg.q3_view_travel_weight
970         if self.cfg.problem == 3
971         else 2e-4
972     )
973     travel_penalty = travel_weight * travel
974     return 1e6 * worst_information - travel_penalty
975
976 return [
977     tuple(map(float, item[0]))
978     for item in sorted(scored, key=score, reverse=True)
979 ]
980
981 def _one_bearing_posterior_estimate(
982     self,
983     channel: int,
984     region: np.ndarray,
985 ) -> np.ndarray:
986     """Estimate range along one bearing from the scan-stage shadow pattern."""
987     observations = self._direction_observations(channel)
988     if len(observations) != 1:
989         return min_enclosing_circle(region).center.copy()
990     mode = self.cfg.q4_one_bearing_posterior
991     if mode not in {"map", "mean"}:
992         raise ValueError(f"unknown one-bearing posterior mode: {mode}")
993
994     positive = observations[0]
995     origin = np.asarray(positive.position, dtype=float)
996     measured = math.radians(float(positive.angle_deg))

```

```

997     negative_points = np.asarray(
998         [
999             observation.position
1000             for observation in self.obs
1001             if observation.channel == channel
1002             and observation.result == "no_signal"
1003         ],
1004         dtype=float,
1005     )
1006     orientations = 2.0 * math.pi * np.arange(72) / 72.0
1007     orientation_vectors = np.column_stack(
1008         (np.cos(orientations), np.sin(orientations))
1009     )
1010     receive_radai = np.linspace(
1011         self.cfg.min_receive_radius,
1012         self.cfg.max_receive_radius,
1013         11,
1014     )
1015     candidates: list[np.ndarray] = []
1016     weights: list[float] = []
1017     for error_deg in np.linspace(-self.cfg.max_error_deg, self.cfg.max_error_deg, 9):
1018         direction = unit(measured + math.radians(float(error_deg)))
1019         for source_distance in np.linspace(10.0, self.cfg.max_receive_radius, 150):
1020             source = origin + source_distance * direction
1021             if not point_in_convex_polygon(source, region, tol=1e-7):
1022                 continue
1023             positive_mask = orientation_vectors @ (origin - source) >= -1e-9
1024             if len(negative_points):
1025                 negative_vectors = negative_points - source
1026                 negative_distances = np.linalg.norm(negative_vectors, axis=1)
1027             else:
1028                 negative_vectors = np.empty((0, 2))
1029                 negative_distances = np.empty(0)
1030             weight = 0.0
1031             for receive_radius in receive_radai:
1032                 if source_distance > receive_radius + 1e-9:
1033                     continue
1034                 mask = positive_mask.copy()
1035                 if len(negative_points):
1036                     close = negative_vectors[
1037                         negative_distances <= receive_radius + 1e-9
1038                     ]
1039                     if len(close):
1040                         mask &= np.all(
1041                             orientation_vectors @ close.T < 1e-9,
1042                             axis=1,
1043                         )
1044                 weight += float(np.mean(mask))
1045             weight *= max(source_distance, 1.0)
1046             if weight > 0.0:
1047                 candidates.append(source)
1048                 weights.append(weight)
1049     if not candidates:
1050         return min_enclosing_circle(region).center.copy()

```

```

1051     candidate_array = np.asarray(candidates)
1052     weight_array = np.asarray(weights)
1053     if mode == "map":
1054         return candidate_array[int(np.argmax(weight_array))].copy()
1055     return np.average(candidate_array, axis=0, weights=weight_array)
1056
1057 @staticmethod
1058 def _polygon_centroid(region: np.ndarray) -> np.ndarray:
1059     """Return the area centroid of a nonempty convex polygon."""
1060     if len(region) < 3:
1061         return np.mean(region, axis=0)
1062     following = np.roll(region, -1, axis=0)
1063     signed_cross = region[:, 0] * following[:, 1] - following[:, 0] * region[:, 1]
1064     cross_sum = float(np.sum(signed_cross))
1065     if abs(cross_sum) <= EPS:
1066         return np.mean(region, axis=0)
1067     return np.sum((region + following) * signed_cross[:, None], axis=0) / (3.0 * cross_sum)
1068
1069 @staticmethod
1070 def _project_to_region(point: np.ndarray, region: np.ndarray) -> np.ndarray:
1071     """Project a point onto a convex polygon when an estimate leaves it."""
1072     if point_in_convex_polygon(point, region, tol=1e-8):
1073         return point.copy()
1074     best_point = region[0].copy()
1075     best_distance = math.inf
1076     for index, left in enumerate(region):
1077         right = region[(index + 1) % len(region)]
1078         edge = right - left
1079         scale = float(np.dot(edge, edge))
1080         fraction = 0.0 if scale <= EPS else float(np.dot(point - left, edge) / scale)
1081         candidate = left + min(1.0, max(0.0, fraction)) * edge
1082         distance_value = float(np.linalg.norm(point - candidate))
1083         if distance_value < best_distance:
1084             best_distance = distance_value
1085             best_point = candidate
1086     return best_point
1087
1088 def _bearing_center_estimate(self, channel: int, region: np.ndarray) -> np.ndarray:
1089     """Estimate the source from central bearing-line intersections.
1090
1091     Only intersections inside the bounded-error feasible region are used.
1092     Squared crossing-angle weights suppress nearly parallel line pairs.
1093     This is an expected-time fast path; the region itself remains the
1094     deterministic certificate used by all fallback logic.
1095     """
1096     observations = self._direction_observations(channel)
1097     intersections: list[np.ndarray] = []
1098     weights: list[float] = []
1099     for left_index in range(len(observations)):
1100         left = observations[left_index]
1101         left_origin = np.asarray(left.position, dtype=float)
1102         left_direction = unit(math.radians(float(left.angle_deg)))
1103         for right_index in range(left_index + 1, len(observations)):
1104             right = observations[right_index]

```

```

1105         right_origin = np.asarray(right.position, dtype=float)
1106         right_direction = unit(math.radians(float(right.angle_deg)))
1107         denominator = cross(left_direction, right_direction)
1108         if abs(denominator) < math.sin(math.radians(3.0)):
1109             continue
1110         delta = right_origin - left_origin
1111         left_distance = cross(delta, right_direction) / denominator
1112         right_distance = cross(delta, left_direction) / denominator
1113         if left_distance < 0.0 or right_distance < 0.0:
1114             continue
1115         candidate = left_origin + left_distance * left_direction
1116         if not point_in_convex_polygon(candidate, region, tol=1e-7):
1117             continue
1118         intersections.append(candidate)
1119         weights.append(float(denominator * denominator))
1120     pairwise = (
1121         np.average(np.asarray(intersections), axis=0, weights=np.asarray(weights))
1122         if intersections
1123         else min_enclosing_circle(region).center.copy()
1124     )
1125     mode = self.cfg.q4_estimator
1126     if mode == "pairwise":
1127         return pairwise
1128
1129     centroid = self._polygon_centroid(region)
1130     if mode == "centroid":
1131         return centroid
1132     if mode == "blend":
1133         return self._project_to_region(0.75 * pairwise + 0.25 * centroid, region)
1134     if mode != "angular_irls":
1135         raise ValueError(f"unknown Q4 bearing estimator: {mode}")
1136
1137     # For a small angular error, perpendicular line residual divided by
1138     # observer range is the angular residual. Iteratively updating those
1139     # ranges gives a lightweight bearing-only nonlinear least-squares fit.
1140     estimate = pairwise.copy()
1141     normals = []
1142     origins = []
1143     for observation in observations:
1144         direction = unit(math.radians(float(observation.angle_deg)))
1145         normals.append(np.array([-direction[1], direction[0]], dtype=float))
1146         origins.append(np.asarray(observation.position, dtype=float))
1147     normal_matrix = np.asarray(normals)
1148     origin_matrix = np.asarray(origins)
1149     right_hand_side = np.sum(normal_matrix * origin_matrix, axis=1)
1150     for _ in range(12):
1151         ranges = np.maximum(20.0, np.linalg.norm(estimate - origin_matrix, axis=1))
1152         weighted = normal_matrix / ranges[:, None]
1153         target = right_hand_side / ranges
1154         candidate, *_ = np.linalg.lstsq(weighted, target, rcond=None)
1155         candidate = self._project_to_region(candidate, region)
1156         if float(np.linalg.norm(candidate - estimate)) <= 1e-5:
1157             estimate = candidate
1158     break

```

```

1159         estimate = candidate
1160     return estimate
1161
1162 def _q4_recover_from_shadow(
1163     self,
1164     channel: int,
1165     negative_point: Sequence[float],
1166 ) -> tuple[bool, bool]:
1167     """Bisect from a known shadow point toward a positive observer.
1168
1169     The segment stays inside the actual reception disk. Its positive end
1170     is in the emitting half-plane, so finite bisection either obtains a
1171     fresh bearing or moves monotonically toward a point known to receive.
1172     Returns ``(cleared, new_bearing)``.
1173     """
1174     observations = self._direction_observations(channel)
1175     if not observations:
1176         return False, False
1177     negative = np.asarray(negative_point, dtype=float)
1178     positive = min(
1179         (np.asarray(observation.position, dtype=float) for observation in observations),
1180         key=lambda point: float(np.linalg.norm(point - negative)),
1181     )
1182     for _ in range(max(1, int(self.cfg.q4_shadow_bisection_rounds))):
1183         probe = 0.5 * (negative + positive)
1184         response = self.measure(probe, channel)
1185         result = response.get("measure_result")
1186         if result == "near":
1187             return self.clear(probe, channel).get("clear_result") == "success", False
1188         if result == "direction":
1189             return False, True
1190         negative = probe
1191     return False, False
1192
1193 def _certified_service_point(
1194     self, region: np.ndarray, circle: Any, following: Sequence[float] | None = None
1195 ) -> np.ndarray:
1196     """Choose a shorter-route point in the intersection of clear disks.
1197
1198     Convex line searches generate candidates, not a claim of global SOCP
1199     optimality. Every returned point is checked against ALL region vertices.
1200     The original enclosing-circle center is always a feasible fallback.
1201     """
1202     center = np.asarray(circle.center, dtype=float)
1203     radius = self.cfg.clear_guarantee_radius
1204     if circle.radius > radius or self._low_budget():
1205         return center.copy()
1206     vertices = np.asarray(region, dtype=float)
1207     if not len(vertices) or float(np.max(np.linalg.norm(vertices - center, axis=1))) > radius +
1208         1e-8:
1209         return center.copy()
1210     start = np.asarray(self.position, dtype=float)
1211     end = None if following is None else np.asarray(following, dtype=float)

```

```

1212 def cost(point: np.ndarray) -> float:
1213     return float(np.linalg.norm(point - start)) + (
1214         0.0 if end is None else float(np.linalg.norm(point - end))
1215     )
1216
1217 directions = [start - center]
1218 if end is not None:
1219     directions.extend([end - center, 0.5 * (start + end) - center])
1220 directions.extend(unit(2.0 * math.pi * k / 16.0) for k in range(16))
1221 best, best_cost = center.copy(), cost(center)
1222 for vector in directions:
1223     norm = float(np.linalg.norm(vector))
1224     if norm < 1e-10:
1225         continue
1226     direction = vector / norm
1227     # Along each ray, circle constraints give an analytic interval.
1228     delta = center - vertices
1229     projection = delta @ direction
1230     discriminants = np.maximum(0.0, projection**2 + radius**2 - np.sum(delta**2, axis=1))
1231     maximum = max(0.0, float(np.min(-projection + np.sqrt(discriminants))))
1232     if maximum <= 1e-8:
1233         continue
1234     if end is None:
1235         distance_along = min(maximum, max(0.0, float(np.dot(start - center, direction))))
1236     else:
1237         left, right = 0.0, maximum
1238         for _ in range(30):
1239             one = left + (right - left) / 3.0
1240             two = right - (right - left) / 3.0
1241             if cost(center + one * direction) <= cost(center + two * direction):
1242                 right = two
1243             else:
1244                 left = one
1245         distance_along = 0.5 * (left + right)
1246     point = center + distance_along * direction
1247     value = cost(point)
1248     if value + 1e-7 < best_cost and float(np.max(np.linalg.norm(vertices - point, axis=1)))
1249         <= radius + 1e-8:
1250         best, best_cost = point, value
1251 return best
1252
1253 def _clear_if_certain(self, channel: int, circle: Any) -> bool:
1254     if circle is None or circle.radius > self.cfg.clear_guarantee_radius:
1255         return False
1256     point = circle.center
1257     if self.cfg.problem == 4 and self.cfg.q4_safe_clear_route:
1258         from q4_refinement import safe_clear_point
1259
1260     region, _, _ = self._region_for(channel)
1261     if len(region):
1262         candidate = safe_clear_point(
1263             region,
1264             self.position,
1265             self.next_task_hint,

```

```

1265         radius=self.cfg.clear_guarantee_radius,
1266         reference=circle.center,
1267         deadline=self._planning_deadline(),
1268     )
1269     if candidate is not None:
1270         self.refinement_stats["safe_clear_choices"] += int(
1271             _distance(candidate, point) > 1e-5
1272         )
1273         point = candidate
1274     if self.cfg.problem == 3 and self.cfg.q3_service_region:
1275         region, _, _ = self._region_for(channel)
1276         point = self._certified_service_point(region, circle, self._q3_following_hint)
1277         if _distance(point, circle.center) > 1e-6:
1278             self.service_point_uses += 1
1279         response = self.clear(point, channel)
1280         return response.get("clear_result") == "success"
1281
1282     def _clear_cover_points(self, region: np.ndarray) -> list[tuple[float, float]]:
1283         """Triangular-lattice cover whose covering radius is below 20 m."""
1284         cover_radius = self.cfg.clear_guarantee_radius
1285         edge = math.sqrt(3.0) * cover_radius * 0.98
1286         vertical = math.sqrt(3.0) * edge / 2.0
1287         lower = np.min(region, axis=0) - cover_radius
1288         upper = np.max(region, axis=0) + cover_radius
1289         j_min = int(math.floor(lower[1] / vertical)) - 1
1290         j_max = int(math.ceil(upper[1] / vertical)) + 1
1291         points: list[tuple[float, float]] = []
1292         for j in range(j_min, j_max + 1):
1293             self._check_budget()
1294             y = j * vertical
1295             shift = 0.5 * edge * j
1296             i_min = int(math.floor((lower[0] - shift) / edge)) - 1
1297             i_max = int(math.ceil((upper[0] - shift) / edge)) + 1
1298             for i in range(i_min, i_max + 1):
1299                 point = np.array([i * edge + shift, y], dtype=float)
1300                 if point_polygon_distance(point, region) <= cover_radius + 1e-7:
1301                     points.append((float(point[0]), float(point[1])))
1302         return _open_route(points, start=self.position)
1303
1304     def _fallback_clear(self, channel: int, region: np.ndarray) -> bool:
1305         points = self._clear_cover_points(region)
1306         for point in points:
1307             if any(_distance(point, old) < 1e-6 for old in self.failed_clear_points.get(channel, [])):
1308                 continue
1309             response = self.clear(point, channel)
1310             if response.get("clear_result") == "success":
1311                 return True
1312         return False
1313
1314     def _q4_recover_single_bearing(self, channel: int, region: np.ndarray) -> tuple[bool, np.ndarray]:
1315         """Resolve a one-bearing directional source by symmetric bisection.
1316

```

```

1317 The first positive observer is inside the source's emitting half-plane.
1318 At a chosen ray split, two lateral probes bracket every orientation
1319 compatible with that observation. Both probes are also required to
1320 lie within the guaranteed 1000 m reception distance of the complete
1321 feasible region. Therefore two no-signal responses safely imply that
1322 the source lies before the split, while either positive response adds
1323 a second bearing without blind clear attempts.
1324 """
1325 observations = self._direction_observations(channel)
1326 if len(observations) != 1 or len(region) == 0:
1327     return False, region
1328
1329 first = observations[0]
1330 origin = np.asarray(first.position, dtype=float)
1331 direction = unit(math.radians(float(first.angle_deg)))
1332 normal = np.array([-direction[1], direction[0]], dtype=float)
1333 narrowed = np.asarray(region, dtype=float)
1334 reception_limit = self.cfg.min_receive_radius - 2.0
1335 tangent = math.tan(math.radians(self.cfg.max_error_deg))
1336
1337 for _ in range(max(1, int(self.cfg.q4_recovery_rounds))):
1338     circle = min_enclosing_circle(narrowed)
1339     if circle.radius <= self.cfg.clear_guarantee_radius:
1340         return self._clear_if_certain(channel, circle), narrowed
1341
1342     projections = (narrowed - origin) @ direction
1343     lower = max(0.0, float(np.min(projections)))
1344     upper = float(np.max(projections))
1345     if upper <= lower + 1e-7:
1346         break
1347     split = 0.5 * (lower + upper)
1348
1349     # For any feasible source  $q=t*u+s*v$  with  $t \geq \text{split}$ , the bearing
1350     # wedge gives  $|s| \leq \text{upper} * \tan(\text{error})$ . This lateral separation
1351     # guarantees at least one probe remains in every emitting
1352     # half-plane that contains the original positive observer.
1353     lateral = upper * tangent + 0.5
1354     midpoint = origin + split * direction
1355     probes = [midpoint + lateral * normal, midpoint - lateral * normal]
1356     if any(
1357         float(np.max(np.linalg.norm(narrowed - probe, axis=1))) > reception_limit
1358         for probe in probes
1359     ):
1360         break
1361     probes.sort(key=lambda probe: float(np.linalg.norm(probe - np.asarray(self.position))))
1362
1363     positive = False
1364     for probe in probes:
1365         response = self.measure(probe, channel)
1366         result = response.get("measure_result")
1367         if result == "near":
1368             return self.clear(probe, channel).get("clear_result") == "success", narrowed
1369         if result == "direction":
1370             positive = True

```

```

1371         break
1372     if positive:
1373         return False, narrowed
1374
1375     # Both guaranteed-reception probes returned no_signal. The
1376     # positive-span argument above makes t < split a hard constraint.
1377     bound = -split - float(np.dot(direction, origin))
1378     self.region_halfplanes.setdefault(channel, []).append((-direction.copy(), bound))
1379     narrowed = clip_convex_polygon_halfplane(narrowed, -direction, bound)
1380     if len(narrowed) == 0:
1381         return False, region
1382
1383     circle = min_enclosing_circle(narrowed)
1384     if circle.radius <= self.cfg.clear_guarantee_radius:
1385         return self._clear_if_certain(channel, circle), narrowed
1386     return False, narrowed
1387
1388 def _localize_and_clear(self, channel: int) -> bool:
1389     region, _, _ = self._region_for(channel)
1390     if (
1391         self.cfg.problem == 4
1392         and self.cfg.q4_symmetric_recovery
1393         and len(self._direction_observations(channel)) == 1
1394         and len(region) > 0
1395     ):
1396         posterior_used = (
1397             self.cfg.q4_one_bearing_posterior != "off"
1398             and channel not in self.one_bearing_posterior_attempted
1399         )
1400         if posterior_used:
1401             self.one_bearing_posterior_attempted.add(channel)
1402             estimate = self._one_bearing_posterior_estimate(channel, region)
1403             if self.clear(estimate, channel).get("clear_result") == "success":
1404                 return True
1405             response = self.measure(estimate, channel)
1406             if response.get("measure_result") == "near":
1407                 return self.clear(estimate, channel).get("clear_result") == "success"
1408             if response.get("measure_result") == "direction":
1409                 region, _, _ = self._region_for(channel)
1410             elif self.cfg.q4_one_bearing_initial_probe:
1411                 # Preserve the legacy fast path when the posterior experiment
1412                 # is disabled: one high-information probe often gets a second
1413                 # bearing immediately.
1414                 candidates = self._view_candidates(channel, region)
1415                 if candidates:
1416                     response = self.measure(candidates[0], channel)
1417                     if response.get("measure_result") == "near":
1418                         return self.clear(candidates[0], channel).get("clear_result") == "success"
1419                     if response.get("measure_result") == "direction":
1420                         region, _, _ = self._region_for(channel)
1421             cleared, narrowed = self._q4_recover_single_bearing(channel, region)
1422             if cleared:
1423                 return True
1424             if len(self._direction_observations(channel)) == 1:

```

```

1425         return self._fallback_clear(channel, narrowed)
1426
1427     consecutive_no_signal = 0
1428     for _ in range(self.cfg.max_view_attempts):
1429         region, _, circle = self._region_for(channel)
1430         if len(region) == 0:
1431             return False
1432         if self._clear_if_certain(channel, circle):
1433             return True
1434         estimated_clear_radius = (
1435             self.cfg.q4_estimated_clear_radius
1436             if self.cfg.problem == 4
1437             else self.cfg.q3_estimated_clear_radius
1438         )
1439         estimated_clear_attempts = (
1440             self.cfg.q4_estimated_clear_attempts
1441             if self.cfg.problem == 4
1442             else self.cfg.q3_estimated_clear_attempts
1443         )
1444         measure_after_failed_estimate = (
1445             self.cfg.q4_measure_after_failed_estimate
1446             if self.cfg.problem == 4
1447             else self.cfg.q3_measure_after_failed_estimate
1448         )
1449         if (
1450             estimated_clear_radius > 0.0
1451             and circle.radius <= estimated_clear_radius
1452             and len(self._direction_observations(channel)) >= 2
1453             and self.estimated_clear_attempts.get(channel, 0)
1454             < estimated_clear_attempts
1455             and self.estimated_clear_bearing_counts.get(channel)
1456             != len(self._direction_observations(channel))
1457         ):
1458             estimate = self._bearing_center_estimate(channel, region)
1459             self.estimated_clear_attempts[channel] = (
1460                 self.estimated_clear_attempts.get(channel, 0) + 1
1461             )
1462             self.estimated_clear_bearing_counts[channel] = len(
1463                 self._direction_observations(channel)
1464             )
1465             if self.clear(estimate, channel).get("clear_result") == "success":
1466                 return True
1467             if measure_after_failed_estimate:
1468                 response = self.measure(estimate, channel)
1469                 if response.get("measure_result") == "near":
1470                     return self.clear(estimate, channel).get("clear_result") == "success"
1471                 if (
1472                     self.cfg.problem == 4
1473                     and response.get("measure_result") == "no_signal"
1474                     and self.cfg.q4_shadow_bisection
1475                     and 2.0 * circle.radius < self.cfg.min_receive_radius - 2.0
1476                 ):
1477                     cleared, new_bearing = self._q4_recover_from_shadow(
1478                         channel, estimate

```

```

1479         )
1480         if cleared:
1481             return True
1482             consecutive_no_signal = 0 if new_bearing else consecutive_no_signal + 1
1483             continue
1484             consecutive_no_signal = (
1485                 0 if response.get("measure_result") == "direction" else consecutive_no_signal
1486                 + 1
1487             )
1488             continue
1489             speculative_radius = (
1490                 self.cfg.q4_speculative_clear_radius
1491                 if self.cfg.problem == 4
1492                 else 30.0
1493             )
1494             if circle.radius <= speculative_radius and not any(
1495                 _distance(circle.center, old) < 1e-6 for old in self.failed_clear_points.get(channel,
1496                 []))
1497             ):
1498                 if self.clear(circle.center, channel).get("clear_result") == "success":
1499                     return True
1500
1501             candidates = self._view_candidates(
1502                 channel,
1503                 region,
1504                 prefer_travel=consecutive_no_signal == 0,
1505             )
1506             if not candidates:
1507                 break
1508             response = self.measure(candidates[0], channel)
1509             if response.get("measure_result") == "near":
1510                 return self.clear(candidates[0], channel).get("clear_result") == "success"
1511             if response.get("measure_result") == "direction":
1512                 consecutive_no_signal = 0
1513             else:
1514                 consecutive_no_signal += 1
1515
1516             region, _, _ = self._region_for(channel)
1517             return len(region) > 0 and self._fallback_clear(channel, region)
1518
1519     def _measure_discovery_point(
1520         self,
1521         point: tuple[float, float],
1522         unresolved: set[int],
1523         positive_counts: dict[int, int],
1524         target_directions: int,
1525     ) -> None:
1526         if target_directions > 1:
1527             active = set(unresolved)
1528             for channel in self.discovered - self.cleared:
1529                 if positive_counts.get(channel, 0) >= target_directions:
1530                     continue
1531             skip_impossible = (
1532                 self.cfg.problem == 4 and self.cfg.q4_skip_impossible_rechecks

```

```

1531         ) or (
1532             self.cfg.problem == 3 and self.cfg.q3_skip_impossible_rechecks
1533         )
1534     if skip_impossible:
1535         region, _, _ = self._region_for(channel)
1536         if (
1537             len(region)
1538             and point_polygon_distance(point, region)
1539             > self.cfg.max_receive_radius + 1.0
1540         ):
1541             self.skipped_rechecks += 1
1542             if self.cfg.problem == 3:
1543                 self.inferred_no_signal.setdefault(channel, set()).add(tuple(map(float,
1544                     point)))
1545             continue
1546         active.add(channel)
1547     channels = self._channel_order(active)
1548 else:
1549     channels = self._channel_order(unresolved)
1550
1551 for channel in channels:
1552     response = self.measure(point, channel)
1553     result = response.get("measure_result")
1554     if result == "near":
1555         if self.clear(point, channel).get("clear_result") == "success":
1556             self.discovered.add(channel)
1557             unresolved.discard(channel)
1558             positive_counts[channel] = target_directions
1559     elif result == "direction":
1560         self.discovered.add(channel)
1561         unresolved.discard(channel)
1562         positive_counts[channel] = positive_counts.get(channel, 0) + 1
1563         if self.cfg.adaptive_clear_stop and positive_counts[channel] >= 2:
1564             region, _, circle = self._region_for(channel)
1565             if (
1566                 len(region)
1567                 and circle is not None
1568                 and circle.radius <= self.cfg.clear_guarantee_radius
1569             ):
1570                 positive_counts[channel] = target_directions
1571
1572 def _opportunistic_q4_scan_clear(
1573     self,
1574     next_point: tuple[float, float] | None,
1575     remaining_scan_points: Sequence[tuple[float, float]],
1576     positive_counts: dict[int, int],
1577     target_directions: int,
1578 ) -> None:
1579     """Insert one high-confidence clear when it adds little scan detour."""
1580     if self.cfg.problem == 4:
1581         enabled = self.cfg.q4_scan_estimated_clear
1582         estimated_radius = self.cfg.q4_estimated_clear_radius
1583         attempt_limit = self.cfg.q4_estimated_clear_attempts
1584         detour_limit = self.cfg.q4_scan_clear_max_detour

```

```

1584         measure_after_failure = self.cfg.q4_measure_after_failed_estimate
1585     else:
1586         enabled = self.cfg.q3_scan_estimated_clear
1587         estimated_radius = self.cfg.q3_estimated_clear_radius
1588         attempt_limit = self.cfg.q3_estimated_clear_attempts
1589         detour_limit = self.cfg.q3_scan_clear_max_detour
1590         measure_after_failure = self.cfg.q3_measure_after_failed_estimate
1591     if not enabled or estimated_radius <= 0.0:
1592         return
1593
1594     current = np.asarray(self.position, dtype=float)
1595     following = np.asarray(next_point, dtype=float) if next_point is not None else None
1596     candidates: list[tuple[float, int, np.ndarray]] = []
1597     for channel in sorted(self.discovered - self.cleared):
1598         observations = self._direction_observations(channel)
1599         if len(observations) < 2:
1600             continue
1601         if self.estimated_clear_attempts.get(channel, 0) >= attempt_limit:
1602             continue
1603         if self.estimated_clear_bearing_counts.get(channel) == len(observations):
1604             continue
1605         region, _, circle = self._region_for(channel)
1606         if (
1607             len(region) == 0
1608             or circle is None
1609             or circle.radius > estimated_radius
1610         ):
1611             continue
1612         estimate = self._bearing_center_estimate(channel, region)
1613         if (
1614             self.cfg.problem == 4
1615             and self.cfg.q4_safe_clear_route
1616             and circle.radius <= self.cfg.clear_guarantee_radius
1617         ):
1618             from q4_refinement import safe_clear_point
1619
1620             safe_point = safe_clear_point(
1621                 region,
1622                 current,
1623                 following,
1624                 radius=self.cfg.clear_guarantee_radius,
1625                 deadline=self._planning_deadline(),
1626             )
1627             if safe_point is not None:
1628                 estimate = safe_point
1629         if self.cfg.problem == 3 and self.cfg.q3_service_region and circle.radius <=
1630             self.cfg.clear_guarantee_radius:
1631             estimate = self._certified_service_point(region, circle, next_point)
1632         if any(
1633             _distance(estimate, old) < 1e-6
1634             for old in self.failed_clear_points.get(channel, [])
1635         ):
1636             continue
1637         detour = float(np.linalg.norm(estimate - current))

```

```

1637         if following is not None:
1638             detour += float(np.linalg.norm(following - estimate))
1639             detour -= float(np.linalg.norm(following - current))
1640         if detour <= detour_limit + EPS:
1641             candidates.append((detour, channel, estimate))
1642
1643     if not candidates:
1644         return
1645     if self.cfg.problem == 4 and self.cfg.q4_scan_clear_lookahead:
1646         future_start = (
1647             np.asarray(remaining_scan_points[-1], dtype=float)
1648             if remaining_scan_points
1649             else current
1650         )
1651         full_future_length = _route_length(
1652             [item[2] for item in candidates], future_start
1653         )
1654         ranked = []
1655         for detour, candidate_channel, candidate_estimate in candidates:
1656             other_centers = [
1657                 item[2] for item in candidates if item[1] != candidate_channel
1658             ]
1659             marginal_future = full_future_length - _route_length(
1660                 other_centers, future_start
1661             )
1662             expected_saving = (
1663                 self.cfg.q4_scan_clear_success_probability * marginal_future
1664                 - detour
1665             )
1666             ranked.append(
1667                 (expected_saving, -detour, -candidate_channel, candidate_estimate)
1668             )
1669             expected_saving, _, negative_channel, estimate = max(ranked)
1670             if expected_saving <= EPS:
1671                 return
1672             channel = -negative_channel
1673     else:
1674         _, channel, estimate = min(
1675             candidates,
1676             key=lambda item: (item[0], item[1]),
1677         )
1678     self.estimated_clear_attempts[channel] = (
1679         self.estimated_clear_attempts.get(channel, 0) + 1
1680     )
1681     self.estimated_clear_bearing_counts[channel] = len(
1682         self._direction_observations(channel)
1683     )
1684     if self.clear(estimate, channel).get("clear_result") == "success":
1685         positive_counts[channel] = target_directions
1686         return
1687     if not measure_after_failure:
1688         return
1689     response = self.measure(estimate, channel)
1690     result = response.get("measure_result")

```

```

1691     if result == "near":
1692         if self.clear(estimate, channel).get("clear_result") == "success":
1693             positive_counts[channel] = target_directions
1694     elif result == "direction":
1695         positive_counts[channel] = min(
1696             target_directions,
1697             positive_counts.get(channel, 0) + 1,
1698         )
1699     elif (
1700         result == "no_signal"
1701         and self.cfg.problem == 4
1702         and self.cfg.q4_shadow_bisection
1703         and self.cfg.q4_scan_shadow_recovery
1704         and 2.0 * circle.radius < self.cfg.min_receive_radius - 2.0
1705     ):
1706         cleared, new_bearing = self._q4_recover_from_shadow(channel, estimate)
1707         if cleared:
1708             positive_counts[channel] = target_directions
1709         elif new_bearing:
1710             positive_counts[channel] = min(
1711                 target_directions,
1712                 len(self._direction_observations(channel)),
1713             )
1714
1715     def _integrated_q4_discovery_scan(
1716         self,
1717         scan_points: list[tuple[float, float]],
1718     ) -> None:
1719         """Insert provable clears without changing the coverage-point order."""
1720         unresolved = set(range(1, 21))
1721         positive_counts: dict[int, int] = {}
1722         target_directions = self.cfg.q4_min_discovery_directions
1723         remaining = list(enumerate(scan_points))
1724
1725         while True:
1726             if len(self.discovered) >= self.cfg.max_source_count:
1727                 unresolved.clear()
1728                 remaining.clear()
1729
1730             clear_tasks: list[tuple[str, int, tuple[float, float]]] = []
1731             for channel in sorted(self.discovered - self.cleared):
1732                 region, _, circle = self._region_for(channel)
1733                 if (
1734                     len(region)
1735                     and circle is not None
1736                     and circle.radius <= self.cfg.clear_guarantee_radius
1737                     and not any(
1738                         _distance(circle.center, old) < 1e-6
1739                         for old in self.failed_clear_points.get(channel, [])
1740                     )
1741                 ):
1742                     clear_tasks.append(
1743                         ("clear", channel, tuple(map(float, circle.center)))
1744                     )

```

```

1745
1746     if not remaining and not clear_tasks:
1747         break
1748
1749     chosen_clear: tuple[str, int, tuple[float, float]] | None = None
1750     if clear_tasks and not remaining:
1751         chosen_clear = clear_tasks[
1752             _joint_route_first(clear_tasks, self.position)
1753         ]
1754     elif clear_tasks:
1755         base_points = [
1756             tuple(map(float, point))
1757             for _, point in remaining
1758         ]
1759         base_path = [tuple(map(float, self.position)), *base_points]
1760         eligible: list[
1761             tuple[float, float, tuple[str, int, tuple[float, float]]]
1762         ] = []
1763         for task in clear_tasks:
1764             center = task[2]
1765             insertion_costs = []
1766             for edge_index in range(len(base_path) - 1):
1767                 left = base_path[edge_index]
1768                 right = base_path[edge_index + 1]
1769                 insertion_costs.append(
1770                     _distance(left, center)
1771                     + _distance(center, right)
1772                     - _distance(left, right)
1773                 )
1774             insertion_costs.append(_distance(base_path[-1], center))
1775             best_edge = min(
1776                 range(len(insertion_costs)),
1777                 key=lambda index: (insertion_costs[index], index),
1778             )
1779             if best_edge == 0:
1780                 eligible.append(
1781                     (
1782                         insertion_costs[0],
1783                         _distance(self.position, center),
1784                         task,
1785                     )
1786                 )
1787             if eligible:
1788                 chosen_clear = min(
1789                     eligible,
1790                     key=lambda item: (item[0], item[1], item[2][1]),
1791                 )[2]
1792
1793     if chosen_clear is not None:
1794         _, identifier, point = chosen_clear
1795         self.clear(point, identifier)
1796         continue
1797
1798     _, point = remaining.pop(0)

```

```

1799         self._measure_discovery_point(
1800             point,
1801             unresolved,
1802             positive_counts,
1803             target_directions,
1804         )
1805
1806         self.absent = set(range(1, 21)) - self.discovered
1807
1808     def _plan_q4_outer_route(
1809         self,
1810         outer_points: Sequence[tuple[float, float]],
1811     ) -> list[tuple[float, float]]:
1812         """Choose the outer-ring gap using the current localization backlog."""
1813         if len(outer_points) < 3:
1814             return [(tuple(map(float, point)) for point in outer_points)]
1815         ring = sorted(
1816             (tuple(map(float, point)) for point in outer_points),
1817             key=lambda point: math.atan2(point[1], point[0]),
1818         )
1819         centers = []
1820         for channel in sorted(self.discovered - self.cleared):
1821             region, _, circle = self._region_for(channel)
1822             if len(region) == 0 or circle is None:
1823                 continue
1824             if (
1825                 self.cfg.q4_estimated_clear_radius > 0.0
1826                 and circle.radius <= self.cfg.q4_estimated_clear_radius
1827                 and len(self._direction_observations(channel)) >= 2
1828             ):
1829                 centers.append(self._bearing_center_estimate(channel, region))
1830             else:
1831                 centers.append(circle.center)
1832
1833         candidates = []
1834         for start_index in range(len(ring)):
1835             for step in (-1, 1):
1836                 order = [
1837                     ring[(start_index + step * offset) % len(ring)]
1838                     for offset in range(len(ring))
1839                 ]
1840                 scan_length = _distance(self.position, order[0]) + sum(
1841                     _distance(order[index - 1], order[index])
1842                     for index in range(1, len(order))
1843                 )
1844                 future_length = _route_length(centers, order[-1])
1845                 candidates.append((scan_length + future_length, scan_length, order))
1846         return min(candidates, key=lambda item: (item[0], item[1], item[2]))[2]
1847
1848     def _discovery_scan(self, scan_points: list[tuple[float, float]]) -> None:
1849         if self.cfg.problem == 4 and self.cfg.q4_integrated_scan_clear:
1850             self._integrated_q4_discovery_scan(scan_points)
1851         return
1852

```

```

1853 unresolved = set(range(1, 21))
1854 positive_counts: dict[int, int] = {}
1855 target_directions = (
1856     self.cfg.q4_min_discovery_directions
1857     if self.cfg.problem == 4
1858     else self.cfg.q3_min_discovery_directions
1859 )
1860 visited_count = 0
1861 for point_index, point in enumerate(scan_points):
1862     if len(self.discovered) >= self.cfg.max_source_count:
1863         unresolved.clear()
1864     self._measure_discovery_point(
1865         point,
1866         unresolved,
1867         positive_counts,
1868         target_directions,
1869     )
1870     next_point = (
1871         scan_points[point_index + 1]
1872         if point_index + 1 < len(scan_points)
1873         else None
1874     )
1875     if self.cfg.problem in {3, 4}:
1876         self._opportunistic_q4_scan_clear(
1877             next_point,
1878             scan_points[point_index + 1 :],
1879             positive_counts,
1880             target_directions,
1881         )
1882     visited_count += 1
1883     if len(self.discovered) >= self.cfg.max_source_count:
1884         break
1885     if (
1886         self.cfg.problem == 4
1887         and self.cfg.q4_adaptive_outer_route
1888         and self.cfg.q4_layout in {"compact", "certified21", "compact25"}
1889         and len(scan_points) >= 13
1890         and visited_count == len(scan_points) - 12
1891     ):
1892         scan_points[point_index + 1 :] = self._plan_q4_outer_route(
1893             scan_points[point_index + 1 :]
1894         )
1895 self.absent = set(range(1, 21)) - self.discovered
1896 if self.cfg.problem == 4:
1897     sparse_channels = {
1898         channel
1899         for channel in self.discovered - self.cleared
1900         if 0 < positive_counts.get(channel, 0) < target_directions
1901     }
1902     if (
1903         self.cfg.q4_local_positive_cap
1904         and visited_count == len(scan_points)
1905     ):
1906         self.local_positive_cap_channels.update(sparse_channels)

```

```

1907         if (
1908             self.cfg.q4_local_triangle_support
1909             and self.cfg.q4_layout == "compact25"
1910         ):
1911             triangles = q4_compact_cover_triangles(
1912                 self.cfg.target_radius,
1913                 self.cfg.min_receive_radius,
1914             )
1915             for channel in sparse_channels:
1916                 region, _, _ = self._region_for(channel)
1917                 channel_observations = [
1918                     observation
1919                     for observation in self.obs
1920                     if observation.channel == channel
1921                 ]
1922                 allowed_components = []
1923                 for triangle in triangles:
1924                     triangle = convex_hull(triangle)
1925                     component = intersect_convex_polygons(region, triangle)
1926                     if len(component) == 0:
1927                         continue
1928                     vertex_results = []
1929                     for vertex in triangle:
1930                         match = next(
1931                             (
1932                                 observation.result
1933                                 for observation in channel_observations
1934                                 if _distance(observation.position, vertex) < 1e-5
1935                             ),
1936                             None,
1937                         )
1938                     vertex_results.append(match)
1939                     if all(result == "no_signal" for result in vertex_results):
1940                         continue
1941                     allowed_components.append(component)
1942                 if allowed_components:
1943                     self.discovery_support_polygons[channel] = convex_hull(
1944                         np.vstack(allowed_components)
1945                     )
1946
1947     def _q3_time_order(self, original: list[int], centers: dict[int, np.ndarray]) -> list[int]:
1948         """One-step completion-time proxy using the actual next action point.
1949
1950         Measures/switches/failed-estimate recovery and travel are counted in
1951         seconds. Future centers remain a proxy, not a probabilistic forecast or
1952         worst-case time certificate. Replan after each completed source.
1953         """
1954         if len(original) < 2 or self._low_budget():
1955             return original
1956         nearest = sorted(centers, key=lambda ch: (_distance(self.position, centers[ch]), ch))
1957         choices = list(dict.fromkeys([original[0]] + nearest))[:max(1,
1958             self.cfg.q3_joint_candidate_limit)]
1959         ranked = []
1960         for channel in choices:

```

```

1960         if self._low_budget():
1961             return original
1962         region, _, circle = self._region_for(channel)
1963         if not len(region) or circle is None:
1964             continue
1965         tail = [ch for ch in original if ch != channel]
1966         following = centers.get(tail[0]) if tail else None
1967         if circle.radius <= self.cfg.clear_guarantee_radius:
1968             point = (
1969                 self._certified_service_point(region, circle, following)
1970                 if self.cfg.q3_service_region else circle.center
1971             )
1972             finish = point
1973             action_cost = 5.0
1974         elif (
1975             circle.radius <= self.cfg.q3_estimated_clear_radius
1976             and len(self._direction_observations(channel)) >= 2
1977             and self.estimated_clear_attempts.get(channel, 0) <
1978                 self.cfg.q3_estimated_clear_attempts
1979             and self.estimated_clear_bearing_counts.get(channel) !=
1980                 len(self._direction_observations(channel))
1981         ):
1982             point = self._bearing_center_estimate(channel, region)
1983             finish = circle.center
1984             # Explicit failed-trial + remeasurement + eventual clear proxy;
1985             # no uncalibrated success probability is assumed.
1986             action_cost = 3.0 + 5.0 + float(self.channel != channel) + 5.0
1987         else:
1988             candidates = self._view_candidates(channel, region)
1989             if not candidates:
1990                 continue
1991             point, finish = np.asarray(candidates[0]), circle.center
1992             action_cost = 5.0 + float(self.channel != channel) + 5.0
1993             length = _distance(self.position, point) + _distance(point, finish)
1994             previous = finish
1995             for other in tail:
1996                 if other in centers:
1997                     length += _distance(previous, centers[other])
1998                     previous = centers[other]
1999             ranked.append((length / 5.0 + action_cost, original.index(channel), channel, tail))
2000         if not ranked:
2001             return original
2002         _, _, first, tail = min(ranked, key=lambda item: item[:3])
2003         self.joint_order_changes += int(first != original[0])
2004         return [first] + tail
2005
2006     def _localization_order(self) -> list[int]:
2007         remaining = self.discovered - self.cleared
2008         centers: dict[int, np.ndarray] = {}
2009         for channel in remaining:
2010             region, _, circle = self._region_for(channel)
2011             if len(region) and circle is not None:
2012                 if (
2013                     self.cfg.problem == 4

```

```

2012         and self.cfg.q4_estimate_route_centers
2013         and self.cfg.q4_estimated_clear_radius > 0.0
2014         and circle.radius <= self.cfg.q4_estimated_clear_radius
2015         and len(self._direction_observations(channel)) >= 2
2016     ):
2017         centers[channel] = self._bearing_center_estimate(channel, region)
2018     else:
2019         centers[channel] = circle.center
2020 if not centers:
2021     return sorted(remaining)
2022 if self._low_budget():
2023     return sorted(remaining, key=lambda ch: (_distance(self.position, centers[ch]) if ch in
2024         centers else math.inf, ch))
2025
2026 if (
2027     self.cfg.problem == 4
2028     and self.cfg.q4_multistart_localization_route
2029 ) or (
2030     self.cfg.problem == 3
2031     and self.cfg.q3_multistart_localization_route
2032 ):
2033     order = _multistart_open_order(centers, self.position)
2034     order.extend(sorted(remaining - set(centers)))
2035     return self._q3_time_order(order, centers) if self.cfg.problem == 3 and
2036         self.cfg.q3_joint_time_routing else order
2037
2038 position = np.asarray(self.position, dtype=float)
2039
2040 unvisited = set(centers)
2041 order: list[int] = []
2042 while unvisited:
2043     chosen = min(
2044         unvisited,
2045         key=lambda channel: (np.linalg.norm(centers[channel] - position), channel),
2046     )
2047     order.append(chosen)
2048     position = centers[chosen]
2049     unvisited.remove(chosen)
2050
2051 improved = True
2052 passes = 0
2053 start = np.asarray(self.position, dtype=float)
2054 while improved and passes < 12:
2055     improved = False
2056     passes += 1
2057     for i in range(0, len(order) - 1):
2058         for j in range(i + 1, len(order)):
2059             left = start if i == 0 else centers[order[i - 1]]
2060             before = float(np.linalg.norm(left - centers[order[i]]))
2061             after = float(np.linalg.norm(left - centers[order[j]]))
2062             if j + 1 < len(order):
2063                 before += float(np.linalg.norm(centers[order[j]] - centers[order[j + 1]]))
2064                 after += float(np.linalg.norm(centers[order[i]] - centers[order[j + 1]]))
2065             if after + 1e-8 < before:

```

```

2064         order[i:j + 1] = reversed(order[i:j + 1])
2065         improved = True
2066
2067     order.extend(sorted(remaining - set(centers)))
2068     return self._q3_time_order(order, centers) if self.cfg.problem == 3 and
2069         self.cfg.q3_joint_time_routing else order
2070
2071 def run(self):
2072     self.start_wall_time = time.monotonic()
2073     scan_points: list[tuple[float, float]] = []
2074     failure: str | None = None
2075     exit_failure: str | None = None
2076     enter_attempted = False
2077     exit_succeeded = False
2078     initial_scan_points: list[tuple[float, float]] = []
2079     try:
2080         if self.cfg.max_actions < 1:
2081             raise ValueError("max_actions must be positive")
2082         if not (
2083             0.0
2084             < self.cfg.clear_guarantee_radius
2085             <= self.cfg.clear_radius
2086             <= 20.0
2087         ):
2088             raise ValueError(
2089                 "clear radii must satisfy 0 < guarantee <= clear <= 20"
2090             )
2091         if (
2092             not math.isfinite(self.cfg.real_time_reserve_s)
2093             or self.cfg.real_time_reserve_s < 0
2094         ):
2095             raise ValueError("real_time_reserve_s must be finite and nonnegative")
2096         scan_points = self._scan_points()
2097         initial_scan_points = list(scan_points)
2098         enter_attempted = True
2099         enter_started = time.monotonic()
2100         enter_response = self._post("/enter")
2101         if "remaining_real_duration_s" in enter_response:
2102             self.remaining_real_duration_s = float(enter_response["remaining_real_duration_s"])
2103             if (
2104                 not math.isfinite(self.remaining_real_duration_s)
2105                 or self.remaining_real_duration_s <= 0
2106             ):
2107                 raise ValueError("invalid remaining_real_duration_s")
2108             self.real_deadline = enter_started + self.remaining_real_duration_s
2109             self.exit_reserve_s = (
2110                 max(0.0, self.cfg.exit_reserve_s)
2111                 if self.cfg.problem == 3
2112                 else min(
2113                     self.cfg.real_time_reserve_s,
2114                     self.remaining_real_duration_s / 5.0,
2115                 )
2116             )
2117         if hasattr(self.client, "set_deadline"):

```

```

2117         self.client.set_deadline(
2118             self.real_deadline if self.cfg.budget_aware else None,
2119             self.exit_reserve_s,
2120         )
2121     self._check_budget()
2122     self._discovery_scan(scan_points)
2123     self.discovery_complete = True
2124     while self.discovered - self.cleared:
2125         self._check_budget()
2126         order = self._localization_order()
2127         channel = order[0]
2128         self.next_task_hint = None
2129         if (
2130             self.cfg.problem == 4
2131             and self.cfg.q4_safe_clear_route
2132             and len(order) > 1
2133         ):
2134             _, _, following = self._region_for(order[1])
2135             if following is not None:
2136                 self.next_task_hint = following.center.copy()
2137             self._q3_following_hint = None
2138             if self.cfg.problem == 3 and self.cfg.q3_service_region and len(order) > 1:
2139                 _, _, next_circle = self._region_for(order[1])
2140                 if next_circle is not None:
2141                     self._q3_following_hint = next_circle.center.copy()
2142             if not self._localize_and_clear(channel):
2143                 failure = f"failed to clear discovered channel {channel}"
2144                 break
2145     except Exception as error:
2146         failure = repr(error)
2147     finally:
2148         if enter_attempted:
2149             try:
2150                 self._post("/exit")
2151                 exit_succeeded = True
2152             except Exception as error:
2153                 exit_failure = repr(error)
2154                 if failure is None:
2155                     failure = f"exit failed: {error!r}"
2156
2157     wall_runtime = time.monotonic() - self.start_wall_time
2158     summary = {
2159         "problem": self.cfg.problem,
2160         "coverage_method": (
2161             (
2162                 "7-point disk cover"
2163                 if self.cfg.q3_layout == "legacy"
2164                 else (
2165                     f"{self.cfg.q3_ring_sides}-point minimax ring cover"
2166                     if self.cfg.q3_layout == "minimax"
2167                     else f"{self.cfg.q3_ring_sides + 1}-point optimized ring cover"
2168                 )
2169             )
2170         )
2171     }
2172     if self.cfg.problem == 3

```

```

2171         else (
2172             "21-point continuously certified positive-spanning cover"
2173             if self.cfg.q4_layout in {"compact", "certified21"}
2174             else (
2175                 "25-point compact positive-spanning triangulation"
2176                 if self.cfg.q4_layout == "compact25"
2177                 else "980 m triangular positive-spanning lattice"
2178             )
2179         )
2180     ),
2181     "scan_point_count": len(scan_points),
2182     "q3_policy": (
2183         {
2184             "layout": self.cfg.q3_layout,
2185             "ring_sides": self.cfg.q3_ring_sides,
2186             "discovery_directions": self.cfg.q3_min_discovery_directions,
2187             "entry_candidates": self.cfg.q3_entry_candidates,
2188             "information_floor_fraction": self.cfg.q3_information_floor_fraction,
2189             "range_order_constraints": self.cfg.q3_range_order_constraints,
2190             "estimated_clear_radius": self.cfg.q3_estimated_clear_radius,
2191             "estimated_clear_attempts": self.cfg.q3_estimated_clear_attempts,
2192             "scan_estimated_clear": self.cfg.q3_scan_estimated_clear,
2193             "scan_clear_max_detour": self.cfg.q3_scan_clear_max_detour,
2194             "multistart_localization_route": self.cfg.q3_multistart_localization_route,
2195             "skip_impossible_rechecks": self.cfg.q3_skip_impossible_rechecks,
2196             "service_region": self.cfg.q3_service_region,
2197             "joint_time_routing": self.cfg.q3_joint_time_routing,
2198         }
2199         if self.cfg.problem == 3
2200         else None
2201     ),
2202     "q4_policy": (
2203         {
2204             "layout": self.cfg.q4_layout,
2205             "discovery_directions": self.cfg.q4_min_discovery_directions,
2206             "symmetric_recovery": self.cfg.q4_symmetric_recovery,
2207             "information_floor_fraction": self.cfg.q4_information_floor_fraction,
2208             "integrated_scan_clear": self.cfg.q4_integrated_scan_clear,
2209             "positive_hull_views": self.cfg.q4_positive_hull_views,
2210             "estimated_clear_radius": self.cfg.q4_estimated_clear_radius,
2211             "estimated_clear_attempts": self.cfg.q4_estimated_clear_attempts,
2212             "estimator": self.cfg.q4_estimator,
2213             "shadow_bisection": self.cfg.q4_shadow_bisection,
2214             "scan_estimated_clear": self.cfg.q4_scan_estimated_clear,
2215             "scan_clear_max_detour": self.cfg.q4_scan_clear_max_detour,
2216             "scan_shadow_recovery": self.cfg.q4_scan_shadow_recovery,
2217             "scan_clear_lookahead": self.cfg.q4_scan_clear_lookahead,
2218             "adaptive_outer_route": self.cfg.q4_adaptive_outer_route,
2219             "estimate_route_centers": self.cfg.q4_estimate_route_centers,
2220             "multistart_localization_route": self.cfg.q4_multistart_localization_route,
2221             "one_bearing_posterior": self.cfg.q4_one_bearing_posterior,
2222             "one_bearing_initial_probe": self.cfg.q4_one_bearing_initial_probe,
2223             "adaptive_clear_stop": self.cfg.adaptive_clear_stop,
2224             "joint_constraints": self.cfg.q4_joint_constraints,

```

```

2225         "time_lookahead": self.cfg.q4_time_lookahead,
2226         "safe_clear_route": self.cfg.q4_safe_clear_route,
2227     }
2228     if self.cfg.problem == 4
2229     else None
2230 ),
2231 "discovered_channels": sorted(self.discovered),
2232 "absent_channels": sorted(self.absent),
2233 "cleared_channels": sorted(self.cleared),
2234 "cleared_count": len(self.cleared),
2235 "actions": self.action_count,
2236 "virtual_time_s": self.last_virtual_time,
2237 "program_runtime_s": wall_runtime,
2238 "failure": failure,
2239 "exit_succeeded": exit_succeeded,
2240 "exit_failure": exit_failure,
2241 "remaining_real_duration_s": self.remaining_real_duration_s,
2242 "config": asdict(self.cfg),
2243 "initial_scan_points": initial_scan_points,
2244 "config_sha256": hashlib.sha256(
2245     json.dumps(asdict(self.cfg), sort_keys=True).encode("utf-8")
2246 ).hexdigest(),
2247 "scan_points_sha256": hashlib.sha256(
2248     json.dumps(initial_scan_points, separators=(",", ":")).encode("utf-8")
2249 ).hexdigest(),
2250 "source_sha256": {
2251     name: hashlib.sha256(
2252         (Path(__file__).parent / name).read_bytes()
2253     ).hexdigest()
2254     for name in (
2255         "robot.py",
2256         "geometry.py",
2257         "offline_sim.py",
2258         "q4_refinement.py",
2259     )
2260     if (Path(__file__).parent / name).is_file()
2261 },
2262 "refinement_stats": dict(self.refinement_stats),
2263 "completion_certified_by_policy": self.discovery_complete and not (self.discovered -
self.cleared) and failure is None,
2264 "discovery_complete": self.discovery_complete,
2265 "remaining_real_duration_s_at_entry": self.remaining_real_duration_s,
2266 "budget_degraded": self.budget_degraded,
2267 "skipped_rechecks": self.skipped_rechecks,
2268 "inferred_no_signal": {str(ch): sorted(points) for ch, points in
self.inferred_no_signal.items()},
2269 "region_cache_hits": self.region_cache_hits,
2270 "region_cache_misses": self.region_cache_misses,
2271 "service_point_uses": self.service_point_uses,
2272 "joint_order_changes": self.joint_order_changes,
2273 "cost_breakdown": {
2274     "distance_m": self.distance_m,
2275     "movement_s": self.distance_m / 5.0,
2276     "measurement_s": 5.0 * self.measure_count,

```

```

2277         "switch_s": self.switch_count,
2278         "clear_s": 5.0 * self.clear_success_count + 3.0 * self.clear_failure_count,
2279         "measure_count": self.measure_count,
2280         "clear_failures": self.clear_failure_count,
2281     },
2282     "full_config": asdict(self.cfg),
2283 }
2284 (self.result_dir / "summary.json").write_text(
2285     json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
2286 )
2287 return summary
2288
2289
2290 class _OfflineClient:
2291     def __init__(self, world, log_path: str | Path):
2292         self.world = world
2293         self.log_path = Path(log_path)
2294         self.log_path.parent.mkdir(parents=True, exist_ok=True)
2295         self.counter = 0
2296
2297     def post(self, path: str, body: dict[str, Any]):
2298         self.counter += 1
2299         request_body = dict(body)
2300         request_id = request_body.setdefault("request_id", f"{path.strip('/')}-{self.counter}")
2301         if path == "/enter":
2302             response = self.world.enter(request_id)
2303         elif path == "/measure":
2304             response = self.world.measure(request_body["position"], request_body["channel"],
2305                                         request_id)
2306         elif path == "/clear":
2307             response = self.world.clear(request_body["position"], request_body["channel"],
2308                                         request_id)
2309         elif path == "/exit":
2310             response = self.world.exit(request_id)
2311         else:
2312             raise ValueError(path)
2313         with self.log_path.open("a", encoding="utf-8") as stream:
2314             stream.write(json.dumps({"path": path, "request": request_body, "response": response},
2315                                   ensure_ascii=False) + "\n")
2316         return response
2317
2318
2319 def run_offline(
2320     seed: int,
2321     problem: int,
2322     out_dir: str | Path,
2323     world_options: dict[str, Any] | None = None,
2324     robot_options: dict[str, Any] | None = None,
2325 ):
2326     from offline_sim import JammerWorld
2327
2328     world = JammerWorld.random_case(seed, problem, **(world_options or {}))
2329     client = _OfflineClient(world, Path(out_dir) / "events.jsonl")
2330     robot = Robot(client, RobotConfig(problem=problem, **(robot_options or {})), out_dir)

```

```

2328     result = robot.run()
2329     result["total_sources"] = len(world.sources)
2330     result["source_directional"] = sum(source.directional for source in world.sources)
2331     simulator_cleared_count = world.cleared_count
2332     result["simulator_cleared_count"] = simulator_cleared_count
2333     result["coverage"] = simulator_cleared_count / len(world.sources)
2334     result["average_localization_clear_time_s"] = world.virtual_time / max(1,
simulator_cleared_count)
2335     if simulator_cleared_count != result["cleared_count"] and not result["failure"]:
2336         result["failure"] = (
2337             "robot/world cleared-state mismatch: "
2338             f"robot={result['cleared_count']}, simulator={simulator_cleared_count}"
2339         )
2340     world.save(Path(out_dir) / "world.json")
2341     (Path(out_dir) / "summary.json").write_text(
2342         json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8"
2343     )
2344     return result
2345
2346
2347 def main() -> None:
2348     parser = argparse.ArgumentParser()
2349     parser.add_argument("--problem", type=int, default=3, choices=[3, 4])
2350     parser.add_argument("--seed", type=int, default=42)
2351     parser.add_argument("--out", default="results/offline")
2352     parser.add_argument("--online", action="store_true")
2353     parser.add_argument("--base-url", default="http://127.0.0.1:2026")
2354     parser.add_argument(
2355         "--q3-layout",
2356         choices=["legacy", "ring", "minimax"],
2357         default=RobotConfig.q3_layout,
2358     )
2359     parser.add_argument("--q3-ring-sides", type=int, default=7)
2360     parser.add_argument("--q3-directions", type=int, default=2)
2361     parser.add_argument("--q3-view-travel-weight", type=float, default=0.2)
2362     parser.add_argument(
2363         "--q3-entry-candidates",
2364         action=argparse.BooleanOptionalAction,
2365         default=True,
2366     )
2367     parser.add_argument("--q3-information-floor-fraction", type=float, default=0.1)
2368     parser.add_argument(
2369         "--q3-skip-impossible-rechecks", action=argparse.BooleanOptionalAction,
2370         default=RobotConfig.q3_skip_impossible_rechecks)
2371     parser.add_argument(
2372         "--q3-service-region", action=argparse.BooleanOptionalAction,
2373         default=RobotConfig.q3_service_region)
2374     parser.add_argument(
2375         "--q3-joint-time-routing", action=argparse.BooleanOptionalAction,
2376         default=RobotConfig.q3_joint_time_routing)
2377     parser.add_argument(
2378         "--budget-aware", action=argparse.BooleanOptionalAction, default=True)
2379     parser.add_argument(
2380         "--q3-range-order-constraints",
2381         action=argparse.BooleanOptionalAction,
2382         default=False,
2383         help="use valid positive/no-signal distance-order constraints for Q3",
2384     )

```

```

2378 parser.add_argument(
2379     "--q4-layout",
2380     choices=["compact", "certified21", "compact25", "legacy"],
2381     default="compact",
2382 )
2383 parser.add_argument(
2384     "--q4-profile",
2385     choices=["default", "baseline", "safe", "joint", "joint-safe", "combined"],
2386     default="default",
2387     help="ablatable Q4 refinement profile; baseline keeps reliability fixes",
2388 )
2389 parser.add_argument(
2390     "--adaptive-clear-stop",
2391     action=argparse.BooleanOptionalAction,
2392     default=True,
2393     help="stop adaptive localization immediately when a guaranteed clear is possible",
2394 )
2395 parser.add_argument(
2396     "--robot-id",
2397     default=None,
2398     help="current simulator login team number; required for --online and intentionally not
2399         embedded in source",
2400 )
2401 args = parser.parse_args()
2402 profile_options = {} if args.q4_profile == "default" else {
2403     "q4_joint_constraints": args.q4_profile in {"joint", "joint-safe", "combined"},
2404     "q4_safe_clear_route": args.q4_profile in {"safe", "joint-safe", "combined"},
2405     "q4_time_lookahead": args.q4_profile == "combined",
2406 }
2407 if args.online:
2408     if not args.robot_id:
2409         parser.error("--robot-id is required when --online is selected")
2410     client = HttpClient(args.base_url, args.robot_id, Path(args.out) / "events.jsonl")
2411     result = Robot(
2412         client,
2413         RobotConfig(
2414             problem=args.problem,
2415             q3_layout=args.q3_layout,
2416             q3_ring_sides=args.q3_ring_sides,
2417             q3_min_discovery_directions=args.q3_directions,
2418             q3_view_travel_weight=args.q3_view_travel_weight,
2419             q3_entry_candidates=args.q3_entry_candidates,
2420             q3_information_floor_fraction=args.q3_information_floor_fraction,
2421             q3_range_order_constraints=args.q3_range_order_constraints,
2422             q3_skip_impossible_rechecks=args.q3_skip_impossible_rechecks,
2423             q3_service_region=args.q3_service_region,
2424             q3_joint_time_routing=args.q3_joint_time_routing,
2425             budget_aware=args.budget_aware,
2426             q4_layout=args.q4_layout,
2427             adaptive_clear_stop=args.adaptive_clear_stop,
2428             **profile_options,
2429         ),
2430         args.out,

```

```

2431         ).run()
2432     else:
2433         result = run_offline(
2434             args.seed,
2435             args.problem,
2436             args.out,
2437             robot_options={
2438                 "q3_layout": args.q3_layout,
2439                 "q3_ring_sides": args.q3_ring_sides,
2440                 "q3_min_discovery_directions": args.q3_directions,
2441                 "q3_view_travel_weight": args.q3_view_travel_weight,
2442                 "q3_entry_candidates": args.q3_entry_candidates,
2443                 "q3_information_floor_fraction": args.q3_information_floor_fraction,
2444                 "q3_range_order_constraints": args.q3_range_order_constraints,
2445                 "q3_skip_impossible_rechecks": args.q3_skip_impossible_rechecks,
2446                 "q3_service_region": args.q3_service_region,
2447                 "q3_joint_time_routing": args.q3_joint_time_routing,
2448                 "budget_aware": args.budget_aware,
2449                 "q4_layout": args.q4_layout,
2450                 "adaptive_clear_stop": args.adaptive_clear_stop,
2451                 **profile_options,
2452             },
2453         )
2454     print(json.dumps(result, ensure_ascii=False, indent=2))
2455
2456
2457 if __name__ == "__main__":
2458     main()

```

B.4 q4_refinement.py

```

1  """Bounded-cost Q4 refinements that never use simulator truth.
2
3  The guarantee layer keeps conservative cells with one shared reception radius
4  and one shared emission orientation per source. Finite hypotheses are used only
5  to rank actions; they never authorize a clear or remove a feasible source.
6  """
7  from __future__ import annotations
8
9  import math
10 import time
11 from dataclasses import dataclass
12
13 import numpy as np
14
15 from geometry import clip_convex_polygon_halfplane, convex_hull, min_enclosing_circle
16
17 TAU = 2.0 * math.pi
18 GEOM_TOL = 1e-6
19
20
21 def _arc(center: float, half_width: float) -> list[tuple[float, float]]:
22     if half_width >= math.pi:

```

```

23     return [(0.0, TAU)]
24     lo = (center - half_width) % TAU
25     hi = (center + half_width) % TAU
26     return [(lo, hi)] if lo <= hi else [(0.0, hi), (lo, TAU)]
27
28
29 def _intersect_arcs(left, right):
30     return [
31         (max(a, c), min(b, d))
32         for a, b in left
33         for c, d in right
34         if max(a, c) <= min(b, d) + 1e-12
35     ]
36
37
38 def _orientation_arcs(center, radius, positive, shadow):
39     """Return an existential relaxation over a ball containing a position cell."""
40     arcs = [(0.0, TAU)]
41     for sign, observers in ((1.0, positive), (-1.0, shadow)):
42         for observer in observers:
43             vector = sign * (observer - center)
44             distance = float(np.linalg.norm(vector))
45             if distance <= radius + GEOM_TOL:
46                 continue
47             half_width = math.acos(
48                 max(-1.0, min(1.0, -(radius + GEOM_TOL) / distance))
49             )
50             arcs = _intersect_arcs(
51                 arcs,
52                 _arc(math.atan2(vector[1], vector[0]), half_width),
53             )
54             if not arcs:
55                 return []
56     return arcs
57
58
59 def _arrays(observations):
60     positive = np.asarray(
61         [o.position for o in observations if o.result in {"direction", "near"}],
62         dtype=float,
63     ).reshape((-1, 2))
64     negative = np.asarray(
65         [o.position for o in observations if o.result == "no_signal"],
66         dtype=float,
67     ).reshape((-1, 2))
68     return positive, negative
69
70
71 def cell_can_contain_source(
72     cell,
73     positive,
74     negative,
75     failed,
76     min_radius=1000.0,

```

```

77     max_radius=1500.0,
78     clear_radius=20.0,
79 ):
80     """Return False only when a cell is impossible for both source types.
81
82     True is a relaxation, not a statement that every point in the cell is
83     feasible. Distance and angle tests are widened across the cell so a true
84     source cannot be rejected by this predicate.
85     """
86     if not len(cell):
87         return False
88     center = (np.min(cell, axis=0) + np.max(cell, axis=0)) / 2.0
89     radius = float(np.max(np.linalg.norm(cell - center, axis=1)))
90     if len(failed):
91         maximums = np.max(
92             np.linalg.norm(cell[None, :, :] - failed[:, None, :], axis=2),
93             axis=1,
94         )
95         if np.any(maximums < clear_radius - GEOM_TOL):
96             return False
97     if not len(positive):
98         return True
99     positive_lower = np.maximum(
100         0.0,
101         np.linalg.norm(positive - center, axis=1) - radius - GEOM_TOL,
102     )
103     shared_radius_lower = max(min_radius, float(np.max(positive_lower)))
104     if shared_radius_lower > max_radius + GEOM_TOL:
105         return False
106     if not len(negative):
107         return True
108     negative_upper = (
109         np.max(
110             np.linalg.norm(cell[None, :, :] - negative[:, None, :], axis=2),
111             axis=1,
112         )
113         + GEOM_TOL
114     )
115     forced_shadow = negative[negative_upper < shared_radius_lower - GEOM_TOL]
116     if not len(forced_shadow):
117         return True
118     return bool(_orientation_arcs(center, radius, positive, forced_shadow))
119
120
121 @dataclass
122 class JointRefinement:
123     region: np.ndarray
124     cells: list[np.ndarray]
125     rejected_cells: int = 0
126     timed_out: bool = False
127
128
129 def refine_region(
130     region,

```

```

131 observations,
132 failed_points,
133 *,
134 min_radius=1000.0,
135 max_radius=1500.0,
136 clear_radius=20.0,
137 max_cells=96,
138 deadline=math.inf,
139 ):
140     """Remove certified-impossible cells and return an outer convex hull.
141
142     Unexamined cells are retained. A deadline hit returns the original region,
143     so incomplete refinement cannot weaken the fallback guarantee.
144     """
145     original = np.asarray(region, dtype=float)
146     if len(original) < 3:
147         return JointRefinement(original, [original])
148     positive, negative = _arrays(observations)
149     failed = np.asarray(failed_points, dtype=float).reshape((-1, 2))
150     if not len(negative) and not len(failed):
151         return JointRefinement(original, [original])
152     pending = [original]
153     kept = []
154     rejected = 0
155     splits = 0
156     while pending:
157         if time.monotonic() > deadline:
158             return JointRefinement(original, [original], rejected, True)
159         index = max(
160             range(len(pending)),
161             key=lambda i: float(np.max(np.ptp(pending[i], axis=0))),
162         )
163         cell = pending.pop(index)
164         if not cell_can_contain_source(
165             cell,
166             positive,
167             negative,
168             failed,
169             min_radius,
170             max_radius,
171             clear_radius,
172         ):
173             rejected += 1
174             continue
175         widths = np.ptp(cell, axis=0)
176         axis = int(np.argmax(widths))
177         if splits >= max_cells or widths[axis] <= 8.0:
178             kept.append(cell)
179             continue
180         cut = 0.5 * (float(np.min(cell[:, axis])) + float(np.max(cell[:, axis])))
181         normal = np.zeros(2)
182         normal[axis] = 1.0
183         halves = (
184             clip_convex_polygon_halfplane(cell, normal, cut),

```

```

185         clip_convex_polygon_halfplane(cell, -normal, -cut),
186     )
187     pending.extend(half for half in halves if len(half))
188     splits += 1
189     if not kept:
190         return JointRefinement(original, [original], rejected)
191     return JointRefinement(convex_hull(np.vstack(kept)), kept, rejected)
192
193
194 def safe_clear_point(
195     region,
196     current,
197     following=None,
198     *,
199     radius=19.5,
200     reference=None,
201     deadline=math.inf,
202 ):
203     """Choose a verified safe point no worse than a safe reference route."""
204     vertices = np.asarray(region, dtype=float)
205     current = np.asarray(current, dtype=float)
206     circle = min_enclosing_circle(vertices)
207     reference = circle.center.copy() if reference is None else np.asarray(reference, dtype=float)
208     if circle.radius > radius + 1e-8:
209         return None
210     following = None if following is None else np.asarray(following, dtype=float)
211
212     def safe(point):
213         return float(np.max(np.linalg.norm(vertices - point, axis=1))) <= radius + 1e-8
214
215     def cost(point):
216         value = float(np.linalg.norm(current - point))
217         if following is not None:
218             value += float(np.linalg.norm(following - point))
219         return value
220
221     if not safe(reference):
222         reference = circle.center.copy()
223     if not safe(reference):
224         return None
225     candidates = [reference]
226     if safe(current):
227         candidates.append(current)
228     if following is not None:
229         vector = following - current
230         quadratic = float(np.dot(vector, vector))
231         if quadratic > 1e-12:
232             lo, hi = 0.0, 1.0
233             for vertex in vertices:
234                 delta = current - vertex
235                 linear = 2.0 * float(np.dot(delta, vector))
236                 constant = float(np.dot(delta, delta)) - radius * radius
237                 discriminant = linear * linear - 4.0 * quadratic * constant
238                 if discriminant < 0.0:

```

```

239         lo, hi = 1.0, 0.0
240         break
241         root = math.sqrt(max(0.0, discriminant))
242         lo = max(lo, (-linear - root) / (2.0 * quadratic))
243         hi = min(hi, (-linear + root) / (2.0 * quadratic))
244     if lo <= hi:
245         point = current + (0.5 * (lo + hi)) * vector
246         if safe(point):
247             return point
248     for vertex in vertices:
249         vector = current - vertex
250         distance = float(np.linalg.norm(vector))
251         if distance > 1e-12:
252             candidate = vertex + radius * vector / distance
253             if safe(candidate):
254                 candidates.append(candidate)
255     for i, left in enumerate(vertices):
256         if time.monotonic() > deadline:
257             return reference
258         for right in vertices[i:]:
259             vector = right - left
260             distance = float(np.linalg.norm(vector))
261             if distance <= 1e-9 or distance > 2 * radius:
262                 continue
263             midpoint = (left + right) / 2.0
264             height = math.sqrt(max(0.0, radius * radius - distance * distance / 4.0))
265             normal = np.array([-vector[1], vector[0]]) / distance
266             for candidate in (midpoint + height * normal, midpoint - height * normal):
267                 if safe(candidate):
268                     candidates.append(candidate)
269     gradient = (reference - current) / max(
270         1e-9,
271         float(np.linalg.norm(reference - current)),
272     )
273     if following is not None:
274         gradient += (reference - following) / max(
275             1e-9,
276             float(np.linalg.norm(reference - following)),
277         )
278     if np.linalg.norm(gradient) > 1e-9:
279         direction = -gradient / np.linalg.norm(gradient)
280         lo, hi = 0.0, 2.0 * radius
281         for _ in range(32):
282             middle = (lo + hi) / 2.0
283             if safe(reference + middle * direction):
284                 lo = middle
285             else:
286                 hi = middle
287         candidates.append(reference + lo * direction)
288     return min(candidates, key=cost).copy()
289
290
291 @dataclass
292 class SourceHypothesis:

```

```

293 point: np.ndarray
294 radius: float
295 orientation_arcs: list[tuple[float, float]]
296 omni: bool
297 weight: float
298
299
300 def build_hypotheses(region, observations, failed_points=(), max_points=25):
301     """Build finite scenarios used only as a time-cost proxy."""
302     region = np.asarray(region, dtype=float)
303     positive, negative = _arrays(observations)
304     failed = np.asarray(failed_points, dtype=float).reshape((-1, 2))
305     center = np.mean(region, axis=0)
306     samples = [center]
307     for vertex in region:
308         for fraction in (0.25, 0.60, 0.90):
309             samples.append(center + fraction * (vertex - center))
310     if len(samples) > max_points:
311         indices = np.linspace(0, len(samples) - 1, max_points, dtype=int)
312         samples = [samples[index] for index in indices]
313     hypotheses = []
314     for point in samples:
315         if len(failed) and np.any(
316             np.linalg.norm(failed - point, axis=1) < 20.0 - GEOM_TOL
317         ):
318             continue
319         positive_distances = np.linalg.norm(positive - point, axis=1)
320         lo = max(1000.0, float(np.max(positive_distances, initial=0.0)))
321         if lo > 1500.0:
322             continue
323         negative_distances = np.linalg.norm(negative - point, axis=1)
324         radii = (lo + (1500.0 - lo) * np.array([1 / 6, 0.5, 5 / 6])).tolist()
325         for radius in radii:
326             close_negative = negative[negative_distances <= radius + GEOM_TOL]
327             if not len(close_negative):
328                 hypotheses.append(SourceHypothesis(point, radius, [], True, 0.5))
329             arcs = _orientation_arcs(point, 0.0, positive, close_negative)
330             arc_length = sum(max(0.0, b - a) for a, b in arcs)
331             if arcs and arc_length > 1e-9:
332                 hypotheses.append(
333                     SourceHypothesis(point, radius, arcs, False, 0.5 * arc_length / TAU)
334                 )
335     if hypotheses:
336         total = sum(h.weight for h in hypotheses)
337         for hypothesis in hypotheses:
338             hypothesis.weight /= total
339     return hypotheses
340
341
342 def visibility_probability(hypothesis, observer):
343     vector = np.asarray(observer) - hypothesis.point
344     distance = float(np.linalg.norm(vector))
345     if distance > hypothesis.radius + 1e-8:
346         return 0.0

```

```

347     if hypothesis.omni or distance <= 1e-9:
348         return 1.0
349     visible_arc = _arc(math.atan2(vector[1], vector[0]), math.pi / 2)
350     denominator = sum(max(0.0, b - a) for a, b in hypothesis.orientation_arcs)
351     overlap = _intersect_arcs(hypothesis.orientation_arcs, visible_arc)
352     return min(
353         1.0,
354         sum(max(0.0, b - a) for a, b in overlap) / max(denominator, 1e-12),
355     )
356
357
358 def rank_measurements(
359     candidates,
360     region,
361     observations,
362     current,
363     failed_points=(),
364     *,
365     error_deg=1.01,
366     switch_cost=0.0,
367     deadline=math.inf,
368 ):
369     """Rank robust candidates with a bounded two-stage time proxy."""
370     original = list(candidates)
371     if not original or len(region) < 3:
372         return original, False
373     hypotheses = build_hypotheses(region, observations, failed_points)
374     if not hypotheses:
375         return original, False
376     sources = np.asarray([h.point for h in hypotheses])
377     weights = np.asarray([h.weight for h in hypotheses])
378     positive, _ = _arrays(observations)
379     options = original[:6]
380     center = min_enclosing_circle(region).center
381     if not any(np.linalg.norm(np.asarray(p) - center) < 1.0 for p in options):
382         untried = all(
383             np.linalg.norm(np.asarray(o.position) - center) >= 1.0
384             for o in observations
385         )
386         if (
387             untried
388             and float(np.max(np.linalg.norm(region - center, axis=1))) <= 998.0
389         ):
390             options.append(tuple(map(float, center)))
391     scored = []
392     for candidate in options:
393         if time.monotonic() > deadline:
394             return original, False
395         point = np.asarray(candidate, dtype=float)
396         distance = np.linalg.norm(sources - point, axis=1)
397         visible = np.asarray(
398             [visibility_probability(hypothesis, point) for hypothesis in hypotheses]
399         )
400         response_weight = weights * visible

```

```

401     probability = float(np.sum(response_weight))
402     angles = np.arctan2(sources[:, 1] - point[1], sources[:, 0] - point[0])
403     residual = (angles[:, None] - angles[None, :] + math.pi) % TAU - math.pi
404     compatible = np.abs(residual) <= math.radians(2.0 * error_deg)
405     conditional = compatible * response_weight[None, :]
406     mass = conditional.sum(axis=1)
407     estimate = conditional @ sources / np.maximum(mass[:, None], 1e-12)
408     errors = np.linalg.norm(sources - estimate, axis=1)
409     clear_cost = np.linalg.norm(estimate - point, axis=1) / 5.0 + 5.0
410     clear_cost += (errors > 19.5) * (
411         8.0 + np.minimum(400.0, 2.0 * errors) / 5.0
412     )
413     clear_cost[distance <= 5.0] = 5.0
414     positive_cost = float(np.sum(response_weight * clear_cost))
415     anchor_distance = min(
416         (float(np.linalg.norm(p - point)) for p in positive),
417         default=500.0,
418     )
419     miss_cost = (
420         6.0
421         + 0.5 * anchor_distance / 5.0
422         + float(np.average(distance, weights=weights)) / 5.0
423         + 5.0
424     )
425     score = (
426         float(np.linalg.norm(point - np.asarray(current))) / 5.0
427         + 5.0
428         + switch_cost
429     )
430     score += positive_cost + (1.0 - probability) * miss_cost
431     scored.append((score, candidate))
432     baseline_score = scored[0][0]
433     best_score, best = min(scored, key=lambda item: item[0])
434     if best_score + 3.0 >= baseline_score:
435         return original, False
436     best_tuple = tuple(map(float, best))
437     return [best_tuple] + [
438         point
439         for point in original
440         if np.linalg.norm(np.asarray(point) - best_tuple) > 1e-7
441     ], True

```

B.5 test_solution.py

```

1  """Deterministic regression tests for the B-question models and protocol client."""
2  from __future__ import annotations
3
4  import json
5  import math
6  import tempfile
7  import unittest
8  import urllib.error
9  from pathlib import Path

```

```

10 from unittest.mock import patch
11
12 import numpy as np
13
14 from geometry import (
15     bearing_region,
16     bearing_region_bounds,
17     bearing_sector_polygon,
18     clip_convex_polygon_halfplane,
19     convex_hull,
20     cross,
21     diameter,
22     diameter_circle_certificate,
23     min_enclosing_circle,
24     optimal_second_point,
25     point_in_convex_polygon,
26     q3_cover_points,
27     q3_minimax_ring_points,
28     q3_ring_cover_points,
29     q4_certified21_cover_points,
30     q4_compact_cover_points,
31     q4_compact_cover_triangles,
32     q4_triangular_cover_points,
33     second_point_candidate_region,
34 )
35 from offline_sim import JammerWorld
36 from robot import (
37     HttpClient,
38     Observation,
39     Robot,
40     RobotConfig,
41     _OfflineClient,
42     _multistart_open_order,
43     run_offline,
44 )
45
46
47 class GeometryTests(unittest.TestCase):
48     def test_diameter_handles_degenerate_polygon(self):
49         self.assertEqual(diameter(np.array([[3.0, -2.0]])), 0.0)
50         self.assertEqual(diameter(np.array([[3.0, -2.0], [3.0, -2.0]])), 0.0)
51
52     def test_convex_membership_handles_degenerate_sets(self):
53         self.assertFalse(point_in_convex_polygon((100.0, 100.0), np.array([[0.0, 0.0]])))
54         self.assertFalse(
55             point_in_convex_polygon(
56                 (100.0, 100.0),
57                 np.array([[0.0, 0.0], [0.0, 0.0]]),
58             )
59         )
60         segment = np.array([[0.0, 0.0], [1.0, 0.0]])
61         self.assertFalse(point_in_convex_polygon((2.0, 0.0), segment))
62         self.assertTrue(point_in_convex_polygon((0.5, 0.0), segment))
63         self.assertFalse(point_in_convex_polygon((0.5, 1.0), segment))

```

```

64
65 def test_q1_diameter_circle_certificate_has_checkable_witness(self):
66     source = np.array([420.0, 680.0])
67     observers = [
68         np.array([-500.0, 0.0]),
69         np.array([800.0, -200.0]),
70         np.array([-200.0, 1100.0]),
71     ]
72     errors = (-0.73, 0.42, -0.18)
73     readings = []
74     for observer, error in zip(observers, errors):
75         theta = math.atan2(source[1] - observer[1], source[0] - observer[0])
76         readings.append((observer, theta + math.radians(error)))
77     region = bearing_region(
78         readings,
79         1800.0,
80         1.0,
81         [(observer, 1500.0) for observer in observers],
82     )
83     certificate = diameter_circle_certificate(region)
84     left, right = certificate["diameter_endpoints"]
85     midpoint = certificate["diameter_center"]
86     witness = certificate["witness"]
87     self.assertAlmostEqual(float(np.linalg.norm(left - right)), certificate["diameter"],
88                             places=8)
89     self.assertAlmostEqual(float(np.linalg.norm((left + right) / 2.0 - midpoint)), 0.0, places=8)
90     self.assertGreater(float(np.linalg.norm(witness - midpoint)), certificate["diameter_radius"])
91     self.assertFalse(certificate["covers"])
92     self.assertGreater(certificate["overflow"], 4.0)
93
94 def test_q1_pure_sector_intersection_reports_bounded_empty_and_unbounded(self):
95     source = np.array([420.0, 680.0])
96     observers = [
97         np.array([-500.0, 0.0]),
98         np.array([800.0, -200.0]),
99         np.array([-200.0, 1100.0]),
100     ]
101     errors = (-0.73, 0.42, -0.18)
102     readings = []
103     for observer, error in zip(observers, errors):
104         theta = math.atan2(source[1] - observer[1], source[0] - observer[0])
105         readings.append((observer, theta + math.radians(error)))
106     bounded, bounded_status = bearing_sector_polygon(readings, 1.0)
107     self.assertEqual(bounded_status, "polygon")
108     self.assertTrue(point_in_convex_polygon(source, bounded, tol=1e-6))
109
110     _, unbounded_status = bearing_sector_polygon(
111         [((0.0, 0.0), 0.0), ((10.0, 0.0), 0.0)],
112         1.0,
113     )
114     self.assertEqual(unbounded_status, "unbounded")
115     empty, empty_status = bearing_sector_polygon(
116         [((0.0, 0.0), 0.0), ((-100.0, 0.0), math.pi)],
117         1.0,

```

```

117     )
118     self.assertEqual(empty_status, "empty")
119     self.assertEqual(len(empty), 0)
120
121     def test_q1_inner_outer_disk_bounds_tighten(self):
122         source = np.array([1798.0, 0.0])
123         observers = [
124             np.array([500.0, 0.0]),
125             np.array([800.0, -800.0]),
126             np.array([800.0, 800.0]),
127         ]
128         errors = (0.8, -0.7, 0.3)
129         readings = []
130         for observer, error in zip(observers, errors):
131             theta = math.atan2(source[1] - observer[1], source[0] - observer[0])
132             readings.append((observer, theta + math.radians(error)))
133         disks = [(observer, 1500.0) for observer in observers]
134         gaps = []
135         for sides in (360, 720):
136             inner, outer = bearing_region_bounds(readings, 1800.0, 1.0, disks, sides)
137             self.assertGreater(len(inner), 0)
138             self.assertGreater(len(outer), 0)
139             self.assertTrue(
140                 all(point_in_convex_polygon(point, outer, tol=1e-6) for point in inner)
141             )
142             diameter_gap = diameter(outer) - diameter(inner)
143             radius_gap = min_enclosing_circle(outer).radius - min_enclosing_circle(inner).radius
144             self.assertGreaterEqual(diameter_gap, -1e-8)
145             self.assertGreaterEqual(radius_gap, -1e-8)
146             gaps.append((diameter_gap, radius_gap))
147         self.assertLess(gaps[1][0], gaps[0][0])
148         self.assertLess(gaps[1][1], gaps[0][1])
149
150     def test_q3_cover_is_below_minimum_receive_radius(self):
151         points = q3_cover_points()
152         worst = 0.0
153         for radius in np.linspace(0.0, 1800.0, 361):
154             for angle in np.linspace(0.0, 2.0 * math.pi, 1440, endpoint=False):
155                 source = radius * np.array([math.cos(angle), math.sin(angle)])
156                 worst = max(worst, float(np.min(np.linalg.norm(points - source, axis=1))))
157         self.assertLess(worst, 1000.0)
158
159     def test_q3_optimized_ring_covers_target_disk(self):
160         for sides in (6, 7, 8, 10, 12, 14, 16):
161             points = q3_ring_cover_points(sides=sides)
162             worst = 0.0
163             for radius in np.linspace(0.0, 1800.0, 181):
164                 for angle in np.linspace(0.0, 2.0 * math.pi, 720, endpoint=False):
165                     source = radius * np.array([math.cos(angle), math.sin(angle)])
166                     worst = max(worst, float(np.min(np.linalg.norm(points - source, axis=1))))
167             self.assertLess(worst, 1000.0)
168
169     def test_q3_optimized_ring_has_continuous_cover_certificate(self):
170         target_radius = 1800.0

```

```

171 receive_radius = 1000.0
172 for sides in (6, 7):
173     points = q3_ring_cover_points(
174         target_radius=target_radius,
175         min_receive_radius=receive_radius,
176         sides=sides,
177     )
178     ring_radius = float(np.linalg.norm(points[1]))
179     half_step = math.pi / sides
180     boundary_gap = math.sqrt(
181         target_radius**2
182         + ring_radius**2
183         - 2.0 * target_radius * ring_radius * math.cos(half_step)
184     )
185
186     # The center covers  $r \leq \text{receive\_radius}$ . Beyond that radius,
187     # distance to the nearest ring point is largest at the target
188     # boundary and sector bisector.
189     self.assertGreater(receive_radius, ring_radius * math.cos(half_step))
190     self.assertLess(boundary_gap, receive_radius)
191
192 def test_q3_minimax_ring_covers_target_disk_without_origin(self):
193     points = q3_minimax_ring_points()
194     self.assertEqual(len(points), 7)
195     worst = 0.0
196     for radius in np.linspace(0.0, 1800.0, 361):
197         for angle in np.linspace(0.0, 2.0 * math.pi, 1440, endpoint=False):
198             source = radius * np.array([math.cos(angle), math.sin(angle)])
199             worst = max(worst, float(np.min(np.linalg.norm(points - source, axis=1))))
200     self.assertLess(worst, 1000.0)
201     with self.assertRaises(ValueError):
202         q3_minimax_ring_points(sides=6)
203
204 def test_q3_range_order_constraints_keep_the_true_source(self):
205     with tempfile.TemporaryDirectory() as directory:
206         world = JammerWorld.random_case(
207             7041,
208             3,
209             count=10,
210             receive_radius_range=(1000.0, 1000.0),
211             boundary_bias=True,
212         )
213         robot = Robot(
214             _OfflineClient(world, Path(directory) / "events.jsonl"),
215             RobotConfig(problem=3, q3_range_order_constraints=True),
216             Path(directory),
217         )
218         robot._post("/enter")
219         source = world.sources[0]
220         for point in robot._scan_points():
221             robot.measure(point, source.channel)
222         region, _, _ = robot._region_for(source.channel)
223         self.assertGreater(len(region), 0)
224         self.assertTrue(

```

```

225         point_in_convex_polygon(source.position, region, tol=1e-6)
226     )
227
228 def test_q4_mesh_positive_spans_every_sampled_source(self):
229     points = q4_triangular_cover_points()
230     self.assertEqual(len(points), 31)
231     for x in np.arange(-1800.0, 1800.1, 60.0):
232         for y in np.arange(-1800.0, 1800.1, 60.0):
233             source = np.array([x + 0.137, y + 0.271])
234             if np.linalg.norm(source) > 1800.0:
235                 continue
236             vectors = points - source
237             close = vectors[np.linalg.norm(vectors, axis=1) <= 1000.0 + 1e-8]
238             self.assertGreaterEqual(len(close), 3)
239             angles = sorted(math.atan2(v[1], v[0]) % (2.0 * math.pi) for v in close if
240                             np.linalg.norm(v) > 1e-8)
241             gaps = [angles[(i + 1) % len(angles)] - angles[i] for i in range(len(angles))]
242             gaps[-1] += 2.0 * math.pi
243             self.assertLessEqual(max(gaps), math.pi + 1e-8)
244
245 def test_q4_compact_cover_positive_spans_every_sampled_source(self):
246     points = q4_compact_cover_points()
247     self.assertEqual(len(points), 25)
248     for x in np.arange(-1800.0, 1800.1, 40.0):
249         for y in np.arange(-1800.0, 1800.1, 40.0):
250             source = np.array([x + 0.137, y + 0.271])
251             if np.linalg.norm(source) > 1800.0:
252                 continue
253             vectors = points - source
254             close = vectors[np.linalg.norm(vectors, axis=1) <= 1000.0 + 1e-8]
255             self.assertGreaterEqual(len(close), 3)
256             angles = sorted(
257                 math.atan2(v[1], v[0]) % (2.0 * math.pi)
258                 for v in close
259                 if np.linalg.norm(v) > 1e-8
260             )
261             gaps = [angles[(i + 1) % len(angles)] - angles[i] for i in range(len(angles))]
262             gaps[-1] += 2.0 * math.pi
263             self.assertLessEqual(max(gaps), math.pi + 1e-8)
264
265 def test_q4_certified21_cover_positive_spans_every_sampled_source(self):
266     points = q4_certified21_cover_points()
267     self.assertEqual(len(points), 21)
268     for x in np.arange(-1800.0, 1800.1, 40.0):
269         for y in np.arange(-1800.0, 1800.1, 40.0):
270             source = np.array([x + 0.137, y + 0.271])
271             if np.linalg.norm(source) > 1800.0:
272                 continue
273             vectors = points - source
274             close = vectors[np.linalg.norm(vectors, axis=1) <= 997.0 + 1e-8]
275             self.assertGreaterEqual(len(close), 2)
276             angles = sorted(
277                 math.atan2(v[1], v[0]) % (2.0 * math.pi)
278                 for v in close

```

```

278         if np.linalg.norm(v) > 1e-8
279     )
280     gaps = [angles[(i + 1) % len(angles)] - angles[i] for i in range(len(angles))]
281     gaps[-1] += 2.0 * math.pi
282     self.assertLessEqual(max(gaps), math.pi + 1e-8)
283
284 def test_q4_symmetric_probe_pair_preserves_directional_detection(self):
285     """At least one guaranteed-reception probe sees every source beyond the split."""
286     error = math.radians(1.01)
287     upper = 1500.0
288     split = 750.0
289     lateral = upper * math.tan(error) + 0.5
290     probes = (
291         np.array([split, lateral], dtype=float),
292         np.array([split, -lateral], dtype=float),
293     )
294     for delta in np.linspace(-error, error, 9):
295         minimum_radius = split / math.cos(delta)
296         for radius in np.linspace(minimum_radius, 1500.0, 9):
297             source = radius * np.array([math.cos(delta), math.sin(delta)])
298             self.assertLessEqual(max(np.linalg.norm(probe - source) for probe in probes), 998.0)
299             for orientation in np.linspace(0.0, 2.0 * math.pi, 721):
300                 normal = np.array([math.cos(orientation), math.sin(orientation)])
301                 if float(np.dot(normal, -source)) < -1e-9:
302                     continue
303                 visible = max(float(np.dot(normal, probe - source)) for probe in probes)
304                 self.assertGreaterEqual(visible, -1e-8)
305
306 def test_q4_recovery_halfplane_is_applied_to_future_regions(self):
307     with tempfile.TemporaryDirectory() as directory:
308         robot = Robot(None, RobotConfig(problem=4), Path(directory))
309         robot.obs.append(Observation((0.0, 0.0), 1, "direction", 0.0, 0.0))
310         robot.region_halfplanes[1] = [(np.array([-1.0, 0.0]), -500.0)]
311         region, _, _ = robot._region_for(1)
312         self.assertGreater(len(region), 0)
313         self.assertLessEqual(float(np.max(region[:, 0])), 500.0 + 1e-7)
314
315 def test_q4_optimized_policy_is_the_default(self):
316     config = RobotConfig(problem=4)
317     self.assertEqual(config.q4_layout, "compact")
318     self.assertTrue(config.q4_symmetric_recovery)
319     self.assertEqual(config.q4_information_floor_fraction, 0.10)
320     self.assertTrue(config.q4_skip_impossible_rechecks)
321     self.assertTrue(config.q4_local_positive_cap)
322     self.assertFalse(config.q4_local_triangle_support)
323     self.assertEqual(config.q4_estimated_clear_radius, 100.0)
324     self.assertEqual(config.q4_estimated_clear_attempts, 2)
325     self.assertTrue(config.q4_shadow_bisection)
326     self.assertTrue(config.q4_scan_estimated_clear)
327     self.assertEqual(config.q4_scan_clear_max_detour, 200.0)
328     self.assertFalse(config.q4_integrated_scan_clear)
329     self.assertTrue(config.q4_joint_constraints)
330     self.assertTrue(config.q4_time_lookahead)
331     self.assertTrue(config.q4_safe_clear_route)

```

```

332
333 def test_q3_optimized_policy_is_the_default(self):
334     config = RobotConfig(problem=3)
335     self.assertEqual(config.q3_layout, "minimax")
336     self.assertEqual(config.q3_ring_sides, 7)
337     self.assertEqual(config.q3_estimated_clear_radius, 100.0)
338     self.assertEqual(config.q3_estimated_clear_attempts, 1)
339     self.assertTrue(config.q3_scan_estimated_clear)
340     self.assertEqual(config.q3_scan_clear_max_detour, 150.0)
341     self.assertTrue(config.q3_multistart_localization_route)
342
343 def test_q3_fast_path_retains_complete_fallback(self):
344     with tempfile.TemporaryDirectory() as directory:
345         result = run_offline(
346             12001,
347             3,
348             Path(directory),
349             world_options={
350                 "count": 10,
351                 "receive_radius_range": (1000.0, 1000.0),
352                 "boundary_bias": True,
353             },
354         )
355     self.assertEqual(result["coverage"], 1.0)
356     self.assertIsNone(result["failure"])
357
358 def test_q4_default_robot_uses_certified21_layout(self):
359     with tempfile.TemporaryDirectory() as directory:
360         world = JammerWorld.random_case(20260912, 4)
361         robot = Robot(
362             _OfflineClient(world, Path(directory) / "events.jsonl"),
363             RobotConfig(problem=4),
364             Path(directory),
365         )
366         self.assertEqual(len(robot._scan_points()), 21)
367
368 def test_q4_sparse_positive_triangle_support_contains_source(self):
369     points = q4_compact_cover_points()
370     triangles = q4_compact_cover_triangles()
371     rng = np.random.default_rng(20260912)
372     for _ in range(800):
373         radius = math.sqrt(float(rng.random())) * 1800.0
374         angle = float(rng.random()) * 2.0 * math.pi
375         source = radius * np.array([math.cos(angle), math.sin(angle)])
376         orientation = float(rng.random()) * 2.0 * math.pi
377         normal = np.array([math.cos(orientation), math.sin(orientation)])
378         vectors = points - source
379         visible = points[
380             (np.linalg.norm(vectors, axis=1) <= 1000.0 + 1e-9)
381             & ((vectors @ normal) >= -1e-9)
382         ]
383         self.assertGreaterEqual(len(visible), 1)
384         if len(visible) >= 3:
385             continue

```

```

386         incident = [
387             triangle
388             for triangle in triangles
389             if any(
390                 float(np.linalg.norm(vertex - positive)) < 1e-6
391                 for vertex in triangle
392                 for positive in visible
393             )
394         ]
395         support = convex_hull(np.vstack(incident))
396         self.assertTrue(point_in_convex_polygon(source, support, tol=1e-6))
397
398     def test_q4_local_positive_cap_keeps_union_of_possible_near_bearings(self):
399         truth = np.array([700.0, 200.0])
400         observers = [np.array([-500.0, 200.0]), np.array([700.0, -600.0])]
401         with tempfile.TemporaryDirectory() as directory:
402             robot = Robot(None, RobotConfig(problem=4), Path(directory))
403             for observer in observers:
404                 angle = math.degrees(math.atan2(*(truth - observer)[::-1]))
405                 robot.obs.append(
406                     Observation(tuple(observer), 1, "direction", angle, 0.0)
407                 )
408             robot.local_positive_cap_channels.add(1)
409             region, _, _ = robot._region_for(1)
410             self.assertTrue(point_in_convex_polygon(truth, region, tol=1e-6))
411
412     def test_q4_positive_observer_segment_extends_safe_candidate_set(self):
413         with tempfile.TemporaryDirectory() as directory:
414             robot = Robot(
415                 None,
416                 RobotConfig(problem=4, q4_positive_hull_views=True),
417                 Path(directory),
418             )
419             truth = np.zeros(2)
420             observers = (np.array([1400.0, -100.0]), np.array([1400.0, 100.0]))
421             for observer in observers:
422                 angle = math.degrees(math.atan2(*(truth - observer)[::-1]))
423                 robot.obs.append(Observation(tuple(observer), 1, "direction", angle, 0.0))
424             region, _, _ = robot._region_for(1)
425             candidates = robot._view_candidates(1, region)
426             extended = [
427                 np.asarray(candidate)
428                 for candidate in candidates
429                 if abs(candidate[0] - 1400.0) < 1e-6 and abs(candidate[1]) < 100.0
430             ]
431             self.assertGreater(len(extended), 0)
432             self.assertTrue(
433                 any(float(np.max(np.linalg.norm(region - candidate, axis=1))) > 998.0 for candidate in
434                     extended)
435             )
436
437     def test_q4_bearing_center_estimate_recovers_exact_intersection(self):
438         with tempfile.TemporaryDirectory() as directory:
439             robot = Robot(None, RobotConfig(problem=4), Path(directory))

```

```

439         truth = np.array([320.0, -270.0])
440         for observer in (np.array([-700.0, 100.0]), np.array([600.0, 800.0]), np.array([900.0,
441             -500.0])):
442             angle = math.degrees(math.atan2(*(truth - observer)[::-1]))
443             robot.obs.append(Observation(tuple(observer), 1, "direction", angle, 0.0))
444             region, _, _ = robot._region_for(1)
445             estimate = robot._bearing_center_estimate(1, region)
446             self.assertLess(float(np.linalg.norm(estimate - truth)), 1e-6)
447
448     def test_q4_one_bearing_posterior_stays_in_feasible_region(self):
449         with tempfile.TemporaryDirectory() as directory:
450             robot = Robot(
451                 None,
452                 RobotConfig(problem=4, q4_one_bearing_posterior="mean"),
453                 Path(directory),
454             )
455             robot.obs.append(Observation((0.0, 0.0), 1, "direction", 0.0, 0.0))
456             for point in ((0.0, 930.0), (0.0, -930.0), (-930.0, 0.0)):
457                 robot.obs.append(Observation(point, 1, "no_signal", None, 0.0))
458                 region, _, _ = robot._region_for(1)
459                 estimate = robot._one_bearing_posterior_estimate(1, region)
460                 self.assertTrue(point_in_convex_polygon(estimate, region, tol=1e-6))
461
462     def test_q4_shadow_bisection_recovers_a_visible_probe(self):
463         class Client:
464             def __init__(self, world):
465                 self.world = world
466                 self.counter = 0
467
468             def post(self, path, body):
469                 self.counter += 1
470                 request_id = f"shadow-{self.counter}"
471                 if path == "/measure":
472                     return self.world.measure(body["position"], body["channel"], request_id)
473                 if path == "/clear":
474                     return self.world.clear(body["position"], body["channel"], request_id)
475                 raise ValueError(path)
476
477         from offline_sim import Source
478
479         source = Source(1, 0.0, 0.0, 1000.0, directional_angle=0.0)
480         world = JammerWorld([source], seed=7)
481         world.entered = True
482         with tempfile.TemporaryDirectory() as directory:
483             robot = Robot(
484                 Client(world),
485                 RobotConfig(problem=4, q4_shadow_bisection_rounds=4),
486                 Path(directory),
487             )
488             robot.obs.append(Observation((600.0, 0.0), 1, "direction", 180.0, 0.0))
489             cleared, new_bearing = robot._q4_recover_from_shadow(1, (-100.0, 0.0))
490             self.assertFalse(cleared)
491             self.assertTrue(new_bearing)

```

```

492 def test_multistart_open_route_is_not_worse_than_nearest_first(self):
493     points = {
494         1: np.array([8.0, 0.0]),
495         2: np.array([9.0, 8.0]),
496         3: np.array([0.0, 9.0]),
497         4: np.array([-7.0, 1.0]),
498     }
499     route = _multistart_open_order(points, (0.0, 0.0))
500     self.assertEqual(set(route), set(points))
501
502     def length(order):
503         total = 0.0
504         previous = np.zeros(2)
505         for stop in order:
506             total += float(np.linalg.norm(points[stop] - previous))
507             previous = points[stop]
508         return total
509
510     self.assertLessEqual(length(route), length([4, 1, 2, 3]) + 1e-9)
511
512 def test_bounded_error_region_contains_true_source(self):
513     rng = np.random.default_rng(2026)
514     for _ in range(30):
515         radius = math.sqrt(float(rng.random())) * 1790.0
516         angle = float(rng.random()) * 2.0 * math.pi
517         source = radius * np.array([math.cos(angle), math.sin(angle)])
518         observers = []
519         errors = []
520         while len(observers) < 3:
521             observer = rng.uniform(-1200.0, 1200.0, 2)
522             if np.linalg.norm(source - observer) <= 1490.0:
523                 observers.append(observer)
524                 errors.append(float(rng.uniform(-1.0, 1.0)))
525         readings = []
526         for observer, error in zip(observers, errors):
527             true = math.atan2(source[1] - observer[1], source[0] - observer[0])
528             readings.append((observer, true + math.radians(error)))
529         region = bearing_region(readings, 1800.0, 1.0, [(observer, 1500.0) for observer in
530             observers])
531         self.assertTrue(point_in_convex_polygon(source, region, tol=1e-6))
532
533 def test_enclosing_circle_and_diameter_bounds(self):
534     rng = np.random.default_rng(17)
535     points = rng.normal(size=(30, 2)) * np.array([3.0, 1.0])
536     circle = min_enclosing_circle(points)
537     self.assertTrue(np.all(np.linalg.norm(points - circle.center, axis=1) <= circle.radius +
538         1e-7))
539     self.assertGreaterEqual(circle.radius + 1e-8, diameter(points) / 2.0)
540
541 def test_q2_candidate_guarantees_reception_and_crossing_angle(self):
542     region, candidates, _ = second_point_candidate_region((0.0, 0.0), 0.0)
543     best, _ = optimal_second_point((0.0, 0.0), 0.0)
544     self.assertGreater(len(candidates), 0)
545     self.assertLessEqual(float(np.max(np.linalg.norm(region - best, axis=1))), 999.0 + 1e-7)

```

```

544     worst_angle = 90.0
545     for radius in np.linspace(5.01, 1500.0, 500):
546         for error in np.linspace(-1.0, 1.0, 21):
547             source = radius * np.array([math.cos(math.radians(error)),
548                                         math.sin(math.radians(error))])
549             a, b = source, source - best
550             sine = abs(cross(a, b)) / (np.linalg.norm(a) * np.linalg.norm(b))
551             worst_angle = min(worst_angle, math.degrees(math.asin(min(1.0, sine))))
552             self.assertLessEqual(np.linalg.norm(source - best), 1000.0)
553     self.assertGreaterEqual(worst_angle, 30.0)
554
555 def test_q3_entry_candidates_preserve_guaranteed_reception(self):
556     with tempfile.TemporaryDirectory() as directory:
557         robot = Robot(
558             None,
559             RobotConfig(
560                 problem=3,
561                 q3_entry_candidates=True,
562                 q3_information_floor_fraction=0.1,
563             ),
564             Path(directory),
565         )
566         robot.position = (300.0, 900.0)
567         robot.obs.append(
568             Observation((0.0, 0.0), 1, "direction", 0.0, 0.0)
569         )
570         region, _, _ = robot._region_for(1)
571         candidates = robot._view_candidates(1, region)
572         self.assertGreater(len(candidates), 0)
573         for candidate in candidates:
574             maximum = float(
575                 np.max(np.linalg.norm(region - np.asarray(candidate), axis=1))
576             )
577             self.assertLessEqual(maximum, 998.0 + 1e-7)
578
579 class SimulatorAndProtocolTests(unittest.TestCase):
580     def test_offline_coverage_uses_simulator_truth(self):
581         with tempfile.TemporaryDirectory() as directory:
582             result = run_offline(2, 4, Path(directory))
583             self.assertEqual(result["cleared_count"], result["simulator_cleared_count"])
584             self.assertEqual(result["coverage"], 1.0)
585
586     def test_location_error_is_repeatable_across_world_instances(self):
587         worlds = [JammerWorld.random_case(31, 4) for _ in range(2)]
588         source = worlds[0].sources[0]
589         point = {"x": source.x + 100.0, "y": source.y}
590         values = [world._fixed_error_deg(world.sources[0], point) for world in worlds]
591         self.assertEqual(values[0], values[1])
592
593     def test_http_retry_reuses_identical_payload_and_request_id(self):
594         requests: list[dict] = []
595
596         class Response:

```

```

597         status = 200
598
599     def __enter__(self):
600         return self
601
602     def __exit__(self, *_):
603         return False
604
605     def read(self):
606         return b'{"accepted":true,"virtual_time_s":0}'
607
608     def fake_urlopen(request, timeout):
609         requests.append(json.loads(request.data.decode("utf-8")))
610         if len(requests) == 1:
611             raise urllib.error.URLError("response lost")
612         return Response()
613
614     with tempfile.TemporaryDirectory() as directory:
615         log_path = Path(directory) / "log.jsonl"
616         client = HttpClient("http://127.0.0.1:2026", "runtime-id", log_path)
617         with patch("urllib.request.urlopen", side_effect=fake_urlopen):
618             client.post("/enter", {}, retries=2)
619             log_text = log_path.read_text(encoding="utf-8")
620             self.assertEqual(requests[0], requests[1])
621             self.assertEqual(requests[0]["request_id"], requests[1]["request_id"])
622             self.assertNotIn("runtime-id", log_text)
623             self.assertIn("<ROBOT_ID>", log_text)
624
625     def test_q4_adaptive_discovery_keeps_directional_channel_active(self):
626         """A found Q4 channel remains active until its bearing target is met."""
627         class Client:
628             def __init__(self, world):
629                 self.world = world
630                 self.counter = 0
631
632             def post(self, path, body):
633                 self.counter += 1
634                 request_id = f"t-{self.counter}"
635                 if path == "/enter":
636                     return self.world.enter(request_id)
637                 if path == "/measure":
638                     return self.world.measure(body["position"], body["channel"], request_id)
639                 if path == "/clear":
640                     return self.world.clear(body["position"], body["channel"], request_id)
641                 return self.world.exit(request_id)
642
643         world = JammerWorld.random_case(
644             91,
645             4,
646             directional_probability=1.0,
647             count=10,
648             receive_radius_range=(1000.0, 1000.0),
649         )
650         with tempfile.TemporaryDirectory() as directory:

```

```

651     robot = Robot(Client(world), RobotConfig(problem=4), Path(directory))
652     robot._post("/enter")
653     points = robot._scan_points()
654     robot._discovery_scan(points)
655     truth_channels = {source.channel for source in world.sources}
656     self.assertEqual(robot.discovered, truth_channels)
657     self.assertEqual(robot.absent, set(range(1, 21)) - truth_channels)
658     self.assertEqual(robot.discovered | robot.absent, set(range(1, 21)))
659     self.assertEqual(robot.discovered & robot.absent, set())
660
661
662 class Q3ImprovementTests(unittest.TestCase):
663     def test_degenerate_membership_and_distance_are_bounded(self):
664         from geometry import point_polygon_distance
665         for vertices in (
666             np.array([[0.0, 0.0]]),
667             np.array([[0.0, 0.0], [1.0, 0.0]]),
668             np.array([[0.0, 0.0], [1.0, 0.0], [0.5, 0.0]]),
669         ):
670             self.assertFalse(point_in_convex_polygon((2000.0, 0.0), vertices))
671             self.assertGreater(point_polygon_distance((2000.0, 0.0), vertices), 1900.0)
672             self.assertTrue(point_in_convex_polygon((0.5, 0.0), np.array([[0.0, 0.0], [1.0, 0.0]])))
673
674     def test_q3_skip_preserves_unknown_channel_scan(self):
675         class Client:
676             def __init__(self):
677                 self.measured = []
678
679             def post(self, path, body):
680                 self.measured.append(body["channel"])
681                 return {"accepted": True, "measure_result": "no_signal", "virtual_time_s": 6.0}
682
683         with tempfile.TemporaryDirectory() as directory:
684             client = Client()
685             robot = Robot(client, RobotConfig(problem=3, q3_skip_impossible_rechecks=True),
686                           directory)
687             robot.obs.append(Observation((0.0, 0.0), 1, "direction", 0.0, 0.0))
688             robot.discovered.add(1)
689             counts = {1: 1}
690             robot._measure_discovery_point((-1700.0, 0.0), {2}, counts, 2)
691             self.assertEqual(client.measured, [2])
692             self.assertEqual(robot.skipped_rechecks, 1)
693             self.assertIn((-1700.0, 0.0), robot.inferred_no_signal[1])
694             self.assertEqual(len([o for o in robot.obs if o.channel == 1]), 1)
695
696     def test_cache_invalidates_for_observations_configuration_and_inferences(self):
697         with tempfile.TemporaryDirectory() as directory:
698             robot = Robot(None, RobotConfig(problem=3, q3_range_order_constraints=True), directory)
699             robot.obs.append(Observation((0.0, 0.0), 1, "direction", 0.0, 0.0))
700             first, _, _ = robot._region_for(1)
701             robot._region_for(1)
702             self.assertEqual(robot.region_cache_hits, 1)
703             robot.inferred_no_signal[1] = {(600.0, 0.0)}
704             shrunk, _, _ = robot._region_for(1)

```

```

704         self.assertEqual(float(np.max(shrunk[:, 0])), 300.0 + 1e-6)
705         self.assertGreater(float(np.max(first[:, 0])), 1000.0)
706         robot.cfg.max_error_deg = 1.02
707         robot._region_for(1)
708         self.assertEqual(robot.region_cache_misses, 3)
709         robot.obs.append(Observation((-10.0, 100.0), 1, "direction", 340.0, 0.0))
710         robot._region_for(1)
711         self.assertEqual(robot.region_cache_misses, 4)
712
713     def test_service_point_is_certified_and_no_longer_than_center(self):
714         rng = np.random.default_rng(6071)
715         with tempfile.TemporaryDirectory() as directory:
716             robot = Robot(None, RobotConfig(problem=3, q3_service_region=True), directory)
717             for _ in range(12):
718                 region = convex_hull(rng.uniform(-6.0, 6.0, size=(8, 2)))
719                 circle = min_enclosing_circle(region)
720                 robot.position = (-100.0, 25.0)
721                 following = np.array([80.0, 40.0])
722                 point = robot._certified_service_point(region, circle, following)
723                 self.assertEqual(float(np.max(np.linalg.norm(region - point, axis=1))), 19.5 +
724                                1e-7)
725                 cost = lambda q: np.linalg.norm(q - robot.position) + np.linalg.norm(q - following)
726                 self.assertEqual(float(cost(point)), float(cost(circle.center)) + 1e-7)
727
728     def test_low_budget_preserves_reception_constraints(self):
729         with tempfile.TemporaryDirectory() as directory:
730             robot = Robot(None, RobotConfig(problem=3), directory)
731             robot.obs.append(Observation((0.0, 0.0), 1, "direction", 0.0, 0.0))
732             robot.deadline_monotonic = 110.0
733             with patch("robot.time.monotonic", return_value=100.0):
734                 region, _, _ = robot._region_for(1)
735                 candidates = robot._view_candidates(1, region)
736                 self.assertTrue(robot.budget_degraded)
737                 self.assertGreater(len(candidates), 0)
738                 for point in candidates:
739                     self.assertEqual(float(np.max(np.linalg.norm(region - point, axis=1))), 998.0 +
740                                    1e-7)
741
742     def test_exit_is_allowed_after_action_limit(self):
743         class Client:
744             def post(self, path, body):
745                 return {"accepted": True, "virtual_time_s": 0.0}
746
747         with tempfile.TemporaryDirectory() as directory:
748             robot = Robot(Client(), RobotConfig(max_actions=2), directory)
749             robot._post("/enter")
750             with self.assertRaises(RuntimeError):
751                 robot._post("/measure", channel=1, position={"x": 0, "y": 0})
752             self.assertTrue(robot._post("/exit")["accepted"])
753
754     def test_exhausted_budget_is_not_reported_as_complete(self):
755         class Client:
756             def __init__(self):
757                 self.paths = []

```

```

756
757     def post(self, path, body):
758         self.paths.append(path)
759         return {"accepted": True, "virtual_time_s": 0.0, "remaining_real_duration_s": 1.0}
760
761     with tempfile.TemporaryDirectory() as directory:
762         client = Client()
763         robot = Robot(client, RobotConfig(problem=3), directory)
764         with patch("robot.time.monotonic", return_value=100.0):
765             result = robot.run()
766             self.assertEqual(client.paths, ["/enter", "/exit"])
767             self.assertIsNotNone(result["failure"])
768             self.assertFalse(result["completion_certified_by_policy"])
769             self.assertFalse(result["discovery_complete"])
770
771     def test_invalid_layout_still_exits_and_reports_failure(self):
772         world = JammerWorld.random_case(6099, 3)
773         with tempfile.TemporaryDirectory() as directory:
774             robot = Robot(_OfflineClient(world, Path(directory) / "events.jsonl"),
775                           RobotConfig(q3_layout="invalid"), directory)
776             result = robot.run()
777             self.assertFalse(world.entered)
778             self.assertIsNotNone(result["failure"])
779             self.assertFalse(result["completion_certified_by_policy"])
780
781     def test_rejected_enter_produces_failure_summary(self):
782         class Client:
783             def post(self, path, body):
784                 return {"accepted": False}
785         with tempfile.TemporaryDirectory() as directory:
786             result = Robot(Client(), RobotConfig(), directory).run()
787             self.assertIsNotNone(result["failure"])
788             self.assertFalse(result["completion_certified_by_policy"])
789
790     def test_deadline_stops_http_without_transmission(self):
791         with tempfile.TemporaryDirectory() as directory:
792             client = HttpClient("http://127.0.0.1:2026", "test-id", Path(directory) / "events.jsonl")
793             client.set_deadline(99.0)
794             with patch("robot.time.monotonic", return_value=100.0), patch("urllib.request.urlopen")
795                 as urlopen:
796                 with self.assertRaises(TimeoutError):
797                     client.post("/measure", {"channel": 1})
798                 urlopen.assert_not_called()
799
800     def test_clear_does_not_switch_measurement_channel(self):
801         from offline_sim import Source
802         world = JammerWorld([Source(2, 10.0, 0.0, 1000.0)])
803         with tempfile.TemporaryDirectory() as directory:
804             robot = Robot(_OfflineClient(world, Path(directory) / "events.jsonl"), RobotConfig(),
805                           directory)
806             robot._post("/enter")
807             robot.clear((10.0, 0.0), 2)
808             self.assertEqual(robot.channel, 1)
809             self.assertEqual(robot.switch_count, 0)

```

```

807         self.assertEqual(robot.clear_success_count, 1)
808         self.assertAlmostEqual(robot.last_virtual_time, 7.0)
809
810     def test_joint_time_order_is_complete_and_preserves_region(self):
811         with tempfile.TemporaryDirectory() as directory:
812             robot = Robot(None, RobotConfig(q3_joint_time_routing=True, q3_service_region=True),
813                           directory)
814             for ch, target in ((1, np.array([100.0, 0.0])), (2, np.array([400.0, 200.0]))):
815                 robot.discovered.add(ch)
816                 for observer in (np.array([-400.0, 100.0]), np.array([300.0, -500.0])):
817                     angle = math.degrees(math.atan2(*(target - observer)[::-1]))
818                     robot.obs.append(Observation(tuple(observer), ch, "direction", angle, 0.0))
819             regions = {ch: robot._region_for(ch)[0].copy() for ch in robot.discovered}
820             order = robot._localization_order()
821             self.assertEqual(set(order), {1, 2})
822             for ch in regions:
823                 self.assertTrue(np.array_equal(regions[ch], robot._region_for(ch)[0]))
824
825 if __name__ == "__main__":
826     unittest.main(verbosity=2)

```

B.6 benchmark_q3_improvements.py

```

1  """Independent-seed Q3 policy ablations; synthetic evidence, never official scores.
2
3  No online clients are used. A frozen pre-change source directory supplies the
4  original policy. Every run receives a fresh result directory and identical
5  hidden source data within its matched-case group. Source truth is used only by
6  the simulator and post-run metrics, never by Robot decisions.
7  """
8  from __future__ import annotations
9
10 import argparse
11 import hashlib
12 import importlib
13 import json
14 import math
15 import os
16 import statistics
17 import sys
18 import time
19 from concurrent.futures import ProcessPoolExecutor, as_completed
20 from dataclasses import asdict
21 from datetime import datetime
22 from pathlib import Path
23
24 import numpy as np
25
26 sys.dont_write_bytecode = True
27 ROOT = Path(__file__).resolve().parent
28 POLICIES = {
29     "original": {},

```

```

30 # Keep ablation groups on the former center-plus-ring layout. Only the
31 # selected policy may inherit the current default layout, so layout gains
32 # are not mislabeled as pruning or service-point gains.
33 "cache_only": {"q3_layout": "ring", "q3_skip_impossible_rechecks": False, "q3_service_region":
34 False, "q3_joint_time_routing": False},
35 "prune": {"q3_layout": "ring", "q3_skip_impossible_rechecks": True, "q3_service_region": False,
36 "q3_joint_time_routing": False},
37 "service": {"q3_layout": "ring", "q3_skip_impossible_rechecks": True, "q3_service_region": True,
38 "q3_joint_time_routing": False},
39 "joint": {"q3_layout": "ring", "q3_skip_impossible_rechecks": True, "q3_service_region": True,
40 "q3_joint_time_routing": True},
41 "selected": None,
42 }
43
44 SCENARIOS = (
45     "random", "n10_boundary", "n13_boundary", "n16_boundary", "n10_uniform",
46     "n16_uniform", "plus_one", "minus_one", "smooth_error", "narrow_angles",
47 )
48
49 def load_modules(root: Path):
50     for name in ("robot", "geometry", "offline_sim"):
51         sys.modules.pop(name, None)
52     sys.path.insert(0, str(root))
53     try:
54         importlib.invalidate_caches()
55         policy = importlib.import_module("robot")
56         simulator = importlib.import_module("offline_sim")
57     finally:
58         sys.path.pop(0)
59     return policy, simulator
60
61 def error_plus(self, source, position):
62     return 1.0
63
64 def error_minus(self, source, position):
65     return -1.0
66
67 def error_smooth(self, source, position):
68     return math.sin(position["x"] / 400.0 + position["y"] / 600.0 + source.channel / 4.0)
69
70 def run_one(job):
71     import types
72     source_root, baseline_root, output_root, scenario, seed, label = job
73     policy, simulator = load_modules(Path(baseline_root if label == "original" else source_root))
74     options = {}
75     if scenario != "random":
76         count = 10 if scenario.startswith("n10") else (16 if scenario.startswith("n16") else 13)
77         options = {"count": count, "receive_radius_range": (1000.0, 1000.0),
78                 "boundary_bias": "uniform" not in scenario}
79     world = simulator.JammerWorld.random_case(seed, 3, **options)

```

```

80 if scenario == "plus_one":
81     world._fixed_error_deg = types.MethodType(error_plus, world)
82 elif scenario == "minus_one":
83     world._fixed_error_deg = types.MethodType(error_minus, world)
84 elif scenario == "smooth_error":
85     world._fixed_error_deg = types.MethodType(error_smooth, world)
86 elif scenario == "narrow_angles":
87     # Source layout with narrow bearing spread and boundary positions.
88     rng = np.random.default_rng(seed)
89     for source in world.sources:
90         radius = float(rng.uniform(1450.0, 1800.0))
91         angle = float(rng.uniform(-0.03, 0.03)) + (math.pi if source.channel % 2 else 0.0)
92         source.x, source.y = radius * math.cos(angle), radius * math.sin(angle)
93 hidden = [asdict(source) for source in world.sources]
94 world_hash = hashlib.sha256(json.dumps(hidden, sort_keys=True).encode()).hexdigest()
95 directory = Path(output_root) / "runs" / scenario / f"{label}_{seed}"
96 if directory.exists():
97     raise FileExistsError(f"Fresh run directory required: {directory}")
98 config = policy.RobotConfig(problem=3, **(POLICIES[label] or {}))
99 robot = policy.Robot(policy._OfflineClient(world, directory / "events.jsonl"), config, directory)
100 started = time.perf_counter()
101 result = robot.run()
102 runtime = time.perf_counter() - started
103 distance = 0.0
104 previous = (0.0, 0.0)
105 measured = switched = failed = cleared = 0
106 channel = 1
107 for event in world.events:
108     req, response = event["request"], event["response"]
109     if "position" in req:
110         point = (req["position"]["x"], req["position"]["y"])
111         distance += math.dist(previous, point)
112         previous = point
113     if event["path"] == "/measure":
114         measured += 1
115         switched += int(channel != req["channel"])
116         channel = req["channel"]
117     elif event["path"] == "/clear":
118         cleared += int(response.get("clear_result") == "success")
119         failed += int(response.get("clear_result") != "success")
120 reconstructed = distance / 5.0 + 5.0 * measured + switched + 5.0 * cleared + 3.0 * failed
121 if abs(reconstructed - world.virtual_time) > 1e-6:
122     raise AssertionError("Trace accounting does not match simulator time")
123 row = {
124     "scenario": scenario, "seed": seed, "policy": label, "world_hash": world_hash,
125     "count": len(world.sources), "cleared": world.cleared_count,
126     "coverage": world.cleared_count / len(world.sources),
127     "time_s": world.virtual_time, "per_source_s": world.virtual_time / max(1,
128     world.cleared_count),
129     "distance_m": distance, "measures": measured, "switches": switched, "failed_clears": failed,
130     "actions": result["actions"], "runtime_s": runtime, "failure": result["failure"],
131     "skipped_rechecks": result.get("skipped_rechecks", 0),
132     "service_point_uses": result.get("service_point_uses", 0),
133     "joint_order_changes": result.get("joint_order_changes", 0),

```

```

133     "region_cache_hits": result.get("region_cache_hits", 0),
134     "config": asdict(config),
135 }
136 (directory / "case_metrics.json").write_text(json.dumps(row, ensure_ascii=False, indent=2),
137 encoding="utf-8")
138 return row
139
140 def summary(rows, policies):
141     bases = {(r["scenario"], r["seed"]): r for r in rows if r["policy"] == "original"}
142     rng = np.random.default_rng(710317)
143     overall = {}
144     for label in policies:
145         values = [r for r in rows if r["policy"] == label]
146         gains = np.array([1.0 - r["time_s"] / bases[(r["scenario"], r["seed"])]["time_s"] for r in
147 values])
148         if any(r["world_hash"] != bases[(r["scenario"], r["seed"])]["world_hash"] for r in values):
149             raise AssertionError("Paired cases have different hidden sources")
150         # Stratified paired bootstrap: keep each stratum's weight fixed.
151         strata = [np.array([1.0 - r["time_s"] / bases[(r["scenario"], r["seed"])]["time_s"]
152 for r in values if r["scenario"] == name]) for name in SCENARIOS]
153         means = np.zeros(10000)
154         for group in strata:
155             if len(group):
156                 indices = rng.integers(0, len(group), size=(10000, len(group)))
157                 means += group[indices].sum(axis=1) / len(values)
158         overall[label] = {
159             "cases": len(values), "coverage_min": min(r["coverage"] for r in values),
160             "failures": sum(bool(r["failure"]) for r in values),
161             "paired_mean_reduction": float(gains.mean()),
162             "paired_median_reduction": float(np.median(gains)),
163             "paired_min_reduction": float(gains.min()),
164             "paired_bootstrap95": [float(v) for v in np.quantile(means, [0.025, 0.975])],
165             "faster_cases": int(np.sum(gains > 1e-10)),
166             "equal_cases": int(np.sum(np.abs(gains) <= 1e-10)),
167             "mean_time_s": statistics.mean(r["time_s"] for r in values),
168             "mean_per_source_s": statistics.mean(r["per_source_s"] for r in values),
169             "median_per_source_s": statistics.median(r["per_source_s"] for r in values),
170             "p95_per_source_s": float(np.quantile([r["per_source_s"] for r in values], 0.95)),
171             "max_per_source_s": max(r["per_source_s"] for r in values),
172             "mean_distance_m": statistics.mean(r["distance_m"] for r in values),
173             "mean_measures": statistics.mean(r["measures"] for r in values),
174             "mean_failed_clears": statistics.mean(r["failed_clears"] for r in values),
175             "mean_runtime_s": statistics.mean(r["runtime_s"] for r in values),
176             "max_runtime_s": max(r["runtime_s"] for r in values),
177             "mean_skipped_rechecks": statistics.mean(r["skipped_rechecks"] for r in values),
178             "mean_service_point_uses": statistics.mean(r["service_point_uses"] for r in values),
179             "mean_joint_order_changes": statistics.mean(r["joint_order_changes"] for r in values),
180         }
181     strata_results = {}
182     for name in SCENARIOS:
183         strata_results[name] = {}
184         for label in policies:
185             subset = [r for r in rows if r["scenario"] == name and r["policy"] == label]

```

```

185         if subset:
186             strata_results[name][label] = {
187                 "cases": len(subset), "coverage_min": min(r["coverage"] for r in subset),
188                 "mean_reduction": statistics.mean(1.0 - r["time_s"] / bases[r["scenario"],
189                 r["seed"]]["time_s"] for r in subset),
190                 "mean_per_source_s": statistics.mean(r["per_source_s"] for r in subset),
191             }
192     return {
193         "data_status": "offline_synthetic_independent_strata_not_official_scores",
194         "overall": overall, "strata": strata_results,
195         "unique_world_count": len({r["world_hash"] for r in bases.values()}),
196         "unique_seed_count": len({r["seed"] for r in bases.values()}),
197         "full_configs": {label: next(r["config"] for r in rows if r["policy"] == label) for label in
198         policies},
199         "note": "Stratified paired bootstrap is conditional on these synthetic stress strata; it is
200         not an official-score guarantee. Concurrent runtime figures are machine/load dependent.",
201     }
202
203 def main():
204     parser = argparse.ArgumentParser()
205     parser.add_argument("--baseline-root", type=Path, required=True)
206     parser.add_argument("--out", type=Path, required=True)
207     parser.add_argument("--phase", choices=("screen", "holdout"), required=True)
208     parser.add_argument("--per-stratum", type=int, default=3)
209     parser.add_argument("--workers", type=int, default=3)
210     parser.add_argument(
211         "--seed-start",
212         type=int,
213         default=None,
214         help="optional fresh seed block; defaults preserve the original screen/holdout sets",
215     )
216     parser.add_argument("--policies", nargs="+", choices=tuple(POLICIES), default=["original",
217     "prune", "service", "joint"])
218     args = parser.parse_args()
219     if "original" not in args.policies or args.per_stratum < 1:
220         parser.error("Include original and at least one case per stratum")
221     if args.out.exists() and any(args.out.iterdir()):
222         parser.error("Use a new empty output directory")
223     args.out.mkdir(parents=True, exist_ok=True)
224     base_seed = args.seed_start if args.seed_start is not None else (
225         730000 if args.phase == "screen" else 840000
226     )
227     jobs = [(str(ROOT), str(args.baseline_root.resolve()), str(args.out.resolve()), scenario,
228     base_seed + index * 1000 + replicate, label)
229     for index, scenario in enumerate(SCENARIOS)
230     for replicate in range(args.per_stratum) for label in args.policies]
231     hashes_before = {name: hashlib.sha256((ROOT / name).read_bytes()).hexdigest()
232     for name in ("robot.py", "geometry.py", "offline_sim.py",
233     "benchmark_q3_improvements.py")}
234
235     rows = []
236     with ProcessPoolExecutor(max_workers=args.workers) as pool:
237         futures = [pool.submit(run_one, job) for job in jobs]
238         for completed, future in enumerate(as_completed(futures), 1):

```

```

234         row = future.result()
235         rows.append(row)
236         if completed % 10 == 0 or row["failure"] or row["coverage"] < 1.0:
237             print(json.dumps({"completed": completed, "total": len(jobs), "last": row["policy"],
238                               "coverage": row["coverage"], "failure": row["failure"]}),
239                   flush=True)
240         rows.sort(key=lambda r: (r["scenario"], r["seed"], r["policy"]))
241         result = summary(rows, args.policies)
242         result.update({"phase": args.phase, "created_at": datetime.now().astimezone().isoformat(),
243                       "source_sha256": hashes_before,
244                       "source_unchanged_during_run": all(hashlib.sha256((ROOT /
245                               name).read_bytes()).hexdigest() == value for name, value in
246                               hashes_before.items()),
247                       "baseline_root": str(args.baseline_root.resolve())})
248         (args.out / "rows.json").write_text(json.dumps(rows, ensure_ascii=False, indent=2),
249                                             encoding="utf-8")
250         (args.out / "summary.json").write_text(json.dumps(result, ensure_ascii=False, indent=2),
251                                                 encoding="utf-8")
252         print(json.dumps(result["overall"], ensure_ascii=False, indent=2), flush=True)
253         if not result["source_unchanged_during_run"]:
254             raise SystemExit("Source changed during experiment; do not freeze mixed-version results")
255         if any(v["failures"] or v["coverage_min"] < 1.0 for v in result["overall"].values()):
256             raise SystemExit("A case failed; inspect rows before adopting any policy")
257
258 if __name__ == "__main__":
259     main()

```

B.7 benchmark_q4_improvements.py

```

1  """Paired, offline-only validation for the final Q4 refinement profile."""
2  from __future__ import annotations
3
4  import argparse
5  import concurrent.futures
6  import hashlib
7  import json
8  import math
9  import os
10 import subprocess
11 import sys
12 import time
13 from dataclasses import asdict
14 from pathlib import Path
15
16 ROOT = Path(__file__).resolve().parent
17 POLICIES = {
18     "baseline": {
19         "q4_joint_constraints": False,
20         "q4_time_lookahead": False,
21         "q4_safe_clear_route": False,
22     },
23     "safe": {

```

```

24     "q4_joint_constraints": False,
25     "q4_time_lookahead": False,
26     "q4_safe_clear_route": True,
27 },
28 "joint": {
29     "q4_joint_constraints": True,
30     "q4_time_lookahead": False,
31     "q4_safe_clear_route": False,
32 },
33 "joint-safe": {
34     "q4_joint_constraints": True,
35     "q4_time_lookahead": False,
36     "q4_safe_clear_route": True,
37 },
38 "combined": {
39     "q4_joint_constraints": True,
40     "q4_time_lookahead": True,
41     "q4_safe_clear_route": True,
42 },
43 }
44
45
46 def _position_hash_error(seed: int, position: dict[str, float]) -> float:
47     key = (
48         f"{seed}|{float(position['x']):.6f}|{float(position['y']):.6f}"
49     ).encode("ascii")
50     raw = int.from_bytes(hashlib.blake2b(key, digest_size=8).digest(), "big")
51     return 2.0 * (raw / (2**64 - 1)) - 1.0
52
53
54 def _apply_error_profile(world, profile: str) -> None:
55     if profile == "channel_hash":
56         return
57     if profile == "position_hash":
58         world._fixed_error_deg = (
59             lambda source, position: _position_hash_error(world.seed, position)
60         )
61     elif profile == "smooth_shared":
62         world._fixed_error_deg = lambda source, position: (
63             math.sin(position["x"] / 300.0) * math.cos(position["y"] / 500.0)
64         )
65     elif profile == "positive_endpoint":
66         world._fixed_error_deg = lambda source, position: 1.0
67     elif profile == "negative_endpoint":
68         world._fixed_error_deg = lambda source, position: -1.0
69     else:
70         raise ValueError(f"unknown error profile: {profile}")
71
72
73 def _scenario_specs(stage: str) -> list[dict]:
74     specs: list[tuple[str, dict, str]] = []
75     if stage == "replay":
76         specs.extend(("random", {}, "channel_hash") for _ in range(10))
77     profiles = (

```

```

78         "channel_hash",
79         "channel_hash",
80         "positive_endpoint",
81         "negative_endpoint",
82         "smooth_shared",
83     )
84     for count in (10, 13, 16):
85         for source_type, probability in (
86             ("omni", 0.0),
87             ("mixed", 0.57),
88             ("directional", 1.0),
89         ):
90             for profile in profiles:
91                 specs.append(
92                     (
93                         f"n{count}_{source_type}_boundary",
94                         {
95                             "count": count,
96                             "directional_probability": probability,
97                             "receive_radius_range": [1000, 1000],
98                             "boundary_bias": True,
99                         },
100                     profile,
101                 )
102             )
103     start = 96123000
104     else:
105         for index in range(6):
106             specs.append(
107                 (
108                     "random_shared_field",
109                     {},
110                     "position_hash" if index % 2 == 0 else "smooth_shared",
111                 )
112             )
113         for count in (10, 13, 16):
114             for source_type, probability in (
115                 ("omni", 0.0),
116                 ("mixed", 0.57),
117                 ("directional", 1.0),
118             ):
119                 for profile in ("position_hash", "smooth_shared"):
120                     specs.append(
121                         (
122                             f"n{count}_{source_type}_shared_boundary",
123                             {
124                                 "count": count,
125                                 "directional_probability": probability,
126                                 "receive_radius_range": [1000, 1000],
127                                 "boundary_bias": True,
128                             },
129                         profile,
130                     )
131                 )

```

```

132     start = 97123000
133     return [
134         {
135             "case": index,
136             "seed": start + index * 41,
137             "scenario": name,
138             "world": options,
139             "error_profile": profile,
140         }
141         for index, (name, options, profile) in enumerate(specs)
142     ]
143
144
145 def _worker(job: dict) -> dict:
146     sys.dont_write_bytecode = True
147     sys.path.insert(0, str(ROOT))
148     import numpy as np
149
150     from geometry import point_in_convex_polygon
151     from offline_sim import JammerWorld
152     from robot import Robot, RobotConfig, _OfflineClient
153
154     output = Path(job["out"])
155     output.mkdir(parents=True, exist_ok=False)
156     world = JammerWorld.random_case(job["seed"], 4, **job["world"])
157     _apply_error_profile(world, job["error_profile"])
158     truth = {source.channel: source for source in world.sources}
159     truth_violations = []
160
161     class VerifiedRobot(Robot):
162         def _region_for(self, channel):
163             region, region_diameter, circle = super()._region_for(channel)
164             if (
165                 self._direction_observations(channel)
166                 and channel in truth
167                 and channel not in self.cleared
168                 and (
169                     not len(region)
170                     or not point_in_convex_polygon(
171                         truth[channel].position,
172                         region,
173                         tol=1e-5,
174                     )
175                 )
176             ):
177                 truth_violations.append(
178                     {"channel": channel, "actions": self.action_count}
179                 )
180             return region, region_diameter, circle
181
182     robot = VerifiedRobot(
183         _OfflineClient(world, output / "events.jsonl"),
184         RobotConfig(problem=4, **POLICIES[job["policy"]]),
185         output,

```

```

186 )
187 result = robot.run()
188 position = (0.0, 0.0)
189 distance = 0.0
190 for event in world.events:
191     request = event["request"]
192     if "position" in request:
193         point = request["position"]
194         next_position = (point["x"], point["y"])
195         distance += math.dist(position, next_position)
196         position = next_position
197 record = {
198     "case": job["case"],
199     "seed": job["seed"],
200     "scenario": job["scenario"],
201     "error_profile": job["error_profile"],
202     "policy": job["policy"],
203     "world_options": job["world"],
204     "world_sha256": hashlib.sha256(
205         json.dumps(
206             [asdict(source) for source in world.sources],
207             sort_keys=True,
208             default=str,
209         ).encode("utf-8")
210     ).hexdigest(),
211     "config": asdict(robot.cfg),
212     "coverage": world.cleared_count / len(world.sources),
213     "sources": len(world.sources),
214     "discovery_exact": robot.discovered == set(truth),
215     "truth_violations": truth_violations,
216     "failure": result["failure"],
217     "virtual_time_s": world.virtual_time,
218     "time_per_source_s": world.virtual_time / len(world.sources),
219     "distance_m": distance,
220     "measures": sum(e["path"] == "/measure" for e in world.events),
221     "no_signal": sum(
222         e["response"].get("measure_result") == "no_signal"
223         for e in world.events
224     ),
225     "failed_clears": sum(
226         e["response"].get("clear_result") == "no_target_in_range"
227         for e in world.events
228     ),
229     "wall_runtime_s": result["program_runtime_s"],
230     "refinement_stats": result.get("refinement_stats", {}),
231     "source_sha256": result.get("source_sha256", {}),
232 }
233 (output / "validation.json").write_text(
234     json.dumps(record, ensure_ascii=False, indent=2),
235     encoding="utf-8",
236 )
237 return record
238
239

```

```

240 def _run_subprocess(job: dict) -> dict:
241     process = subprocess.run(
242         [sys.executable, "-B", str(Path(__file__).resolve()), "--worker", json.dumps(job)],
243         capture_output=True,
244         text=True,
245         encoding="utf-8",
246         errors="replace",
247         env={
248             *os.environ,
249             "PYTHONUTF8": "1",
250             "PYTHONDONTWRITEBYTECODE": "1",
251             "OPENBLAS_NUM_THREADS": "1",
252         },
253         timeout=240,
254     )
255     if process.returncode:
256         raise RuntimeError(
257             f"{job['policy']} case {job['case']}: "
258             f"{process.stderr[-3000:]} {process.stdout[-1000:]}"
259         )
260     return json.loads(process.stdout)
261
262
263 def _policy_summary(group: list[dict], baseline: dict[int, dict]) -> dict:
264     import numpy as np
265
266     ordered = sorted(group, key=lambda row: row["case"])
267     reductions = np.asarray(
268         [
269             (baseline[row["case"]]["virtual_time_s"] - row["virtual_time_s"])
270             / baseline[row["case"]]["virtual_time_s"]
271             for row in ordered
272         ]
273     )
274     rng = np.random.default_rng(20260912)
275     bootstrap = np.mean(
276         rng.choice(reductions, size=(10000, len(reductions)), replace=True),
277         axis=1,
278     )
279     return {
280         "cases": len(ordered),
281         "coverage_min": min(row["coverage"] for row in ordered),
282         "failure_runs": sum(bool(row["failure"]) for row in ordered),
283         "discovery_mismatches": sum(not row["discovery_exact"] for row in ordered),
284         "truth_violations": sum(len(row["truth_violations"]) for row in ordered),
285         "mean_virtual_time_s": float(
286             np.mean([row["virtual_time_s"] for row in ordered])
287         ),
288         "mean_time_per_source_s": float(
289             np.mean([row["time_per_source_s"] for row in ordered])
290         ),
291         "p95_time_per_source_s": float(
292             np.percentile([row["time_per_source_s"] for row in ordered], 95)
293         ),

```

```

294     "mean_distance_m": float(np.mean([row["distance_m"] for row in ordered])),
295     "mean_measures": float(np.mean([row["measures"] for row in ordered])),
296     "mean_failed_clears": float(
297         np.mean([row["failed_clears"] for row in ordered])
298     ),
299     "mean_wall_runtime_s": float(
300         np.mean([row["wall_runtime_s"] for row in ordered])
301     ),
302     "max_wall_runtime_s": max(row["wall_runtime_s"] for row in ordered),
303     "paired_reduction_mean": float(np.mean(reductions)),
304     "paired_reduction_median": float(np.median(reductions)),
305     "paired_reduction_min": float(np.min(reductions)),
306     "paired_reduction_max": float(np.max(reductions)),
307     "paired_bootstrap_ci95": np.percentile(bootstrap, [2.5, 97.5]).tolist(),
308     "faster_cases": int(np.sum(reductions > 1e-10)),
309     "slower_cases": int(np.sum(reductions < -1e-10)),
310 }
311
312
313 def _summarize(rows: list[dict], stage: str) -> dict:
314     baseline = {row["case"]: row for row in rows if row["policy"] == "baseline"}
315     policies = {}
316     for policy in sorted({row["policy"] for row in rows}):
317         policies[policy] = _policy_summary(
318             [row for row in rows if row["policy"] == policy],
319             baseline,
320         )
321     pairs_match = all(
322         row["world_sha256"] == baseline[row["case"]]["world_sha256"]
323         for row in rows
324     )
325     return {
326         "stage": stage,
327         "data_status": "offline_synthetic_independent_seeds_not_official",
328         "paired_worlds_match": pairs_match,
329         "policies": policies,
330     }
331
332
333 def main() -> None:
334     parser = argparse.ArgumentParser()
335     parser.add_argument("--worker")
336     parser.add_argument("--stage", choices=("replay", "independent"), default="independent")
337     parser.add_argument("--policies", nargs="+", choices=tuple(POLICIES), default=list(POLICIES))
338     parser.add_argument("--workers", type=int, default=4)
339     parser.add_argument("--out", default="results/q4_improvement_independent")
340     args = parser.parse_args()
341     if args.worker:
342         print(json.dumps(_worker(json.loads(args.worker)), ensure_ascii=False))
343         return
344     if "baseline" not in args.policies:
345         parser.error("baseline is required for paired comparison")
346     output = Path(args.out)
347     if output.exists() and any(output.iterdir()):

```

```

348     parser.error(f"output directory must be empty: {output}")
349 output.mkdir(parents=True, exist_ok=True)
350 jobs = [
351     {
352         **case,
353         "policy": policy,
354         "out": str(output / f"case{case['case']:03d}_{policy}"),
355     }
356     for case in _scenario_specs(args.stage)
357     for policy in args.policies
358 ]
359 rows = []
360 started = time.monotonic()
361 with concurrent.futures.ThreadPoolExecutor(max_workers=args.workers) as pool:
362     futures = [pool.submit(_run_subprocess, job) for job in jobs]
363     for future in concurrent.futures.as_completed(futures):
364         row = future.result()
365         rows.append(row)
366         print(
367             f"{len(rows)}/{len(jobs)} {row['policy']} case={row['case']} "
368             f"coverage={row['coverage']:.3f} time={row['virtual_time_s']:.2f}",
369             flush=True,
370         )
371 rows.sort(key=lambda row: (row["case"], row["policy"]))
372 (output / "rows.json").write_text(
373     json.dumps(rows, ensure_ascii=False, indent=2),
374     encoding="utf-8",
375 )
376 summary = _summarize(rows, args.stage)
377 summary["validation_wall_runtime_s"] = time.monotonic() - started
378 (output / "summary.json").write_text(
379     json.dumps(summary, ensure_ascii=False, indent=2),
380     encoding="utf-8",
381 )
382 print(json.dumps(summary, ensure_ascii=False, indent=2))
383 selected = summary["policies"].get("combined")
384 if not summary["paired_worlds_match"] or any(
385     metrics["coverage_min"] < 1.0
386     or metrics["failure_runs"]
387     or metrics["discovery_mismatches"]
388     or metrics["truth_violations"]
389     for metrics in summary["policies"].values()
390 ):
391     raise SystemExit(2)
392 if selected is not None and selected["paired_reduction_mean"] <= 0.0:
393     raise SystemExit(3)
394
395
396 if __name__ == "__main__":
397     main()

```

B.8 run_experiments.py

```
1  """Reproducible offline experiments for Questions 1--4.
2
3  Offline worlds are deliberately labeled synthetic. They validate the policy
4  and geometry but are never substituted for the three official simulator runs.
5  """
6  from __future__ import annotations
7
8  import argparse
9  import csv
10 import json
11 import math
12 import statistics
13 from pathlib import Path
14
15 import numpy as np
16
17 from geometry import (
18     bearing_region,
19     cross,
20     diameter,
21     diameter_circle_certificate,
22     min_enclosing_circle,
23     optimal_second_point,
24     q3_cover_points,
25     q4_compact_cover_points,
26     q4_triangular_cover_points,
27     second_point_candidate_region,
28 )
29 from robot import run_offline
30
31
32 def _q1_metrics() -> dict:
33     source = np.array([420.0, 680.0])
34     # Keep every observation within the largest possible 1500 m reception
35     # disk, so this is a feasible Q1 instance rather than an empty intersection.
36     observers = [np.array([-500.0, 0.0]), np.array([800.0, -200.0]), np.array([-200.0, 1100.0])]
37     errors_deg = [-0.73, 0.42, -0.18]
38     readings = []
39     for observer, error in zip(observers, errors_deg):
40         true_angle = math.atan2(source[1] - observer[1], source[0] - observer[0])
41         readings.append((observer, true_angle + math.radians(error)))
42     region = bearing_region(
43         readings,
44         target_radius=1800.0,
45         error_deg=1.0,
46         extra_disks=[(observer, 1500.0) for observer in observers],
47         n_disk=720,
48     )
49     certificate = diameter_circle_certificate(region)
50     circle = certificate["min_enclosing_circle"]
51     engineering_region = bearing_region(
52         readings,
```

```

53     target_radius=1800.0,
54     error_deg=1.01,
55     extra_disks=[(observer, 1500.0) for observer in observers],
56     n_disk=720,
57 )
58 engineering_circle = min_enclosing_circle(engineering_region)
59 return {
60     "error_bound_deg": 1.0,
61     "observer_count": len(observers),
62     "region_vertex_count": int(len(region)),
63     "diameter_m": float(diameter(region)),
64     "min_enclosing_radius_m": float(circle.radius),
65     "diameter_circle_covers_region": bool(certificate["covers"]),
66     "diameter_circle_overflow_m": float(certificate["overflow"]),
67     "engineering_1_01_deg": {
68         "diameter_m": float(diameter(engineering_region)),
69         "min_enclosing_radius_m": float(engineering_circle.radius),
70     },
71     "source_in_region": bool(
72         len(region) > 0
73         and np.all(
74             [
75                 cross(region[(i + 1) % len(region)] - region[i], source - region[i]) >= -1e-6
76                 for i in range(len(region))
77             ]
78         )
79     ),
80 }
81
82
83 def _q2_metrics() -> dict:
84     rows = []
85     for bearing_deg in range(0, 360, 45):
86         theta = math.radians(bearing_deg)
87         region, candidates, scores = second_point_candidate_region((0.0, 0.0), theta, grid_step=40.0)
88         best, _ = optimal_second_point((0.0, 0.0), theta, grid_step=40.0)
89         rows.append(
90             {
91                 "first_bearing_deg": bearing_deg,
92                 "first_region_vertices": int(len(region)),
93                 "candidate_count": int(len(candidates)),
94                 "best_x_m": float(best[0]),
95                 "best_y_m": float(best[1]),
96                 "best_score": float(np.max(scores)),
97             }
98         )
99     return {
100         "rows": rows,
101         "candidate_rule": "within 1000 m of every first-reading feasible point and worst crossing angle >=30 deg",
102     }
103
104
105 def _run_policy_experiments(out: Path, seeds: list[int]) -> list[dict]:

```

```

106 rows: list[dict] = []
107 for problem in (3, 4):
108     for seed in seeds:
109         run_dir = out / "offline_runs" / f"p{problem}_{seed:04d}"
110         result = run_offline(seed, problem, run_dir)
111         rows.append(
112             {
113                 "scenario": "random",
114                 "problem": problem,
115                 "seed": seed,
116                 "total_sources": result["total_sources"],
117                 "directional_sources": result["source_directional"],
118                 "cleared_count": result["cleared_count"],
119                 "coverage": result["coverage"],
120                 "virtual_time_s": result["virtual_time_s"],
121                 "average_time_s": result["average_localization_clear_time_s"],
122                 "actions": result["actions"],
123                 "failure": result["failure"] or "",
124             }
125         )
126
127 # Adversarial stress cases: every source is directional, all receive
128 # radii are the lower bound, and radial draws are boundary-biased. These
129 # belong to Q4 only; keeping them outside the problem loop prevents the
130 # same run from being counted once under Q3 and once under Q4.
131 for seed in seeds[: min(5, len(seeds))]:
132     run_dir = out / "offline_runs" / f"p4_stress_{seed:04d}"
133     result = run_offline(
134         seed,
135         4,
136         run_dir,
137         {
138             "directional_probability": 1.0,
139             "count": 16,
140             "receive_radius_range": (1000.0, 1000.0),
141             "boundary_bias": True,
142         },
143     )
144     rows.append(
145         {
146             "scenario": "adversarial_all_directional_min_radius",
147             "problem": 4,
148             "seed": seed,
149             "total_sources": result["total_sources"],
150             "directional_sources": result["source_directional"],
151             "cleared_count": result["cleared_count"],
152             "coverage": result["coverage"],
153             "virtual_time_s": result["virtual_time_s"],
154             "average_time_s": result["average_localization_clear_time_s"],
155             "actions": result["actions"],
156             "failure": result["failure"] or "",
157         }
158     )
159 return rows

```

```

160
161
162 def _write_policy_summary(out: Path, rows: list[dict]) -> dict:
163     csv_path = out / "monte_carlo.csv"
164     fields = list(rows[0]) if rows else []
165     with csv_path.open("w", newline="", encoding="utf-8-sig") as stream:
166         writer = csv.DictWriter(stream, fieldnames=fields)
167         writer.writeheader()
168         writer.writerows(rows)
169
170     grouped = {}
171     for key in sorted({(row["scenario"], row["problem"]) for row in rows}):
172         subset = [row for row in rows if (row["scenario"], row["problem"]) == key]
173         coverages = [float(row["coverage"]) for row in subset]
174         times = [float(row["average_time_s"]) for row in subset]
175         grouped[f"{key[0]}_p{key[1]}"] = {
176             "runs": len(subset),
177             "coverage_mean": statistics.mean(coverages),
178             "coverage_min": min(coverages),
179             "coverage_sd": statistics.pstdev(coverages) if len(coverages) > 1 else 0.0,
180             "average_time_mean_s": statistics.mean(times),
181             "average_time_sd_s": statistics.pstdev(times) if len(times) > 1 else 0.0,
182             "failure_runs": sum(bool(row["failure"]) for row in subset),
183         }
184     summary = {"data_status": "offline_synthetic_regression_only", "groups": grouped}
185     (out / "monte_carlo_summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2),
186         encoding="utf-8")
187
188     return summary
189
190 def main() -> None:
191     parser = argparse.ArgumentParser()
192     parser.add_argument("--out", default="results/experiments")
193     parser.add_argument("--seed-start", type=int, default=0)
194     parser.add_argument("--seed-count", type=int, default=20)
195     args = parser.parse_args()
196     out = Path(args.out)
197     out.mkdir(parents=True, exist_ok=True)
198     seeds = list(range(args.seed_start, args.seed_start + args.seed_count))
199
200     rows = _run_policy_experiments(out, seeds)
201     summary = _write_policy_summary(out, rows)
202     q1 = _q1_metrics()
203     q2 = _q2_metrics()
204     geometry = {
205         "q3_scan_point_count": int(len(q3_cover_points())),
206         "q4_scan_point_count": int(len(q4_compact_cover_points())),
207         "q4_legacy_scan_point_count": int(len(q4_triangular_cover_points())),
208     }
209     (out / "q1_metrics.json").write_text(json.dumps(q1, ensure_ascii=False, indent=2),
210         encoding="utf-8")
211     (out / "q2_metrics.json").write_text(json.dumps(q2, ensure_ascii=False, indent=2),
212         encoding="utf-8")

```

```

210 (out / "geometry_metrics.json").write_text(json.dumps(geometry, ensure_ascii=False, indent=2),
    encoding="utf-8")
211 print(json.dumps({"summary": summary, "q1": q1, "q2": q2, "geometry": geometry},
    ensure_ascii=False, indent=2))
212
213
214 if __name__ == "__main__":
215     main()

```

B.9 tools/make_flowcharts.py

```

1 from pathlib import Path
2
3 import matplotlib.pyplot as plt
4 from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
5
6
7 OUT = Path(__file__).resolve().parents[1] / "figures"
8
9 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "DejaVu Sans"]
10 plt.rcParams["axes.unicode_minus"] = False
11 plt.rcParams["pdf.fonttype"] = 42
12 plt.rcParams["ps.fonttype"] = 42
13
14
15 def make_flow(filename, stages):
16     OUT.mkdir(parents=True, exist_ok=True)
17     count = len(stages)
18     fig, ax = plt.subplots(figsize=(14, 3.3))
19     ax.set_xlim(0, count)
20     ax.set_ylim(0, 1)
21     ax.axis("off")
22     colors = ["#dceaf7", "#e5f2e6", "#f7ead7", "#e8e2f4", "#f6e0e0"]
23
24     for index, (title, detail) in enumerate(stages):
25         x = index + 0.5
26         ax.text(
27             x,
28             0.89,
29             title,
30             ha="center",
31             va="center",
32             fontsize=16,
33             fontweight="bold",
34         )
35         box = FancyBboxPatch(
36             (x - 0.39, 0.23),
37             0.78,
38             0.42,
39             boxstyle="round,pad=0.025,rounding_size=0.035",
40             linewidth=1.6,
41             edgecolor="#404040",
42             facecolor=colors[index % len(colors)],

```

```

43     )
44     ax.add_patch(box)
45     ax.text(
46         x,
47         0.44,
48         detail,
49         ha="center",
50         va="center",
51         fontsize=16,
52         linespacing=1.25,
53     )
54     if index < count - 1:
55         arrow = FancyArrowPatch(
56             (x + 0.40, 0.44),
57             (x + 0.60, 0.44),
58             arrowstyle="-|>",
59             mutation_scale=18,
60             linewidth=1.5,
61             color="#606060",
62         )
63         ax.add_patch(arrow)
64
65     fig.savefig(OUT / filename, format="pdf", bbox_inches="tight", pad_inches=0.08)
66     plt.close(fig)
67
68
69 def main():
70     make_flow(
71         "flow-overall.pdf",
72         [
73             (" 题面输入", " 误差、半径\n 计时与边界"),
74             (" 保守定位", " 闭角楔\n 相容位置域"),
75             (" 连续发现", " 圆覆盖\n 正向包围"),
76             (" 安全清除", " 覆盖圆\n 三角格兜底"),
77             (" 时间调度", " 滚动路线\n 有限前瞻"),
78         ],
79     )
80     make_flow(
81         "flow-q1.pdf",
82         [
83             (" 读取观测", " 检测点与\n 示向度"),
84             (" 构造角楔", " 有界误差\n 半平面"),
85             (" 求相容域", " 交会定位\n 距离收紧"),
86             (" 几何计算", " 直径与\n 最小覆盖圆"),
87             (" 输出核验", " 点积判据\n 反例结论"),
88         ],
89     )
90     make_flow(
91         "flow-q2.pdf",
92         [
93             (" 首测归一化", " 平移与\n 旋转坐标"),
94             (" 保证接收", " 所有可能源\n 均距不超 1000"),
95             (" 角度筛选", " 连续最坏\n 交会角"),
96             (" 候选评估", " 网格计算\n 距离与角度"),

```

```

97         (" 确定第二点", " 排序后\n 反变换"),
98     ],
99 )
100 make_flow(
101     "flow-q3.pdf",
102     [
103         (" 七点环覆盖", " 解析极小极大\n 连续发现"),
104         (" 轮询频道", " 未发现频道\n 完整扫描"),
105         (" 更新域", " 角楔交\n 安全补测"),
106         (" 保证清除", " 服务区域\n 三角格兜底"),
107         (" 滚动路线", " 批量访问\n2-opt 调度"),
108     ],
109 )
110 make_flow(
111     "flow-q4.pdf",
112     [
113         (" 正向包围", "21 点扫描集\n 连续核验"),
114         (" 联合观测", " 正负反馈\n 相容单元"),
115         (" 有限前瞻", " 候选假设\n 时间排序"),
116         (" 安全服务", " 保证域内\n 短路选点"),
117         (" 失败保护", " 测向恢复\n 三角格兜底"),
118     ],
119 )
120
121
122 if __name__ == "__main__":
123     main()

```

B.10 运行说明

```

1 # CUMCM 2026 B 题求解工程
2
3 本目录保存 B 题的可复现实验与论文源稿。题目 PDF、附件和模拟器为上游输入；`资料`目录只用于方法线索，
  不作为本题数值结果来源。
4
5 ## 目录
6
7 - `geometry.py`：角误差扇区、半平面裁剪、定位区域直径与最小覆盖圆。
8 - `offline_sim.py`：按附件规则生成全向/定向干扰源并模拟 HTTP 动作，用于策略回归测试。
9 - `robot.py`：连接真实模拟器的串行 HTTP 机器人；同一逻辑也可注入离线模拟器。
10 - `run_experiments.py`：离线 Monte Carlo、Q1/Q2 几何实验、敏感性分析。
11 - `benchmark_q3.py`：问题 3 新旧策略的同案例留出集与边界压力测试。
12 - `benchmark_q4_recovery.py`：问题 4 单示向线恢复策略的同案例配对测试。
13 - `analyze_run.py`：按移动、测量、无信号和失败清除分解单局日志。
14 - `paper.md`：由冻结结果撰写的中文论文源稿。
15 - `results/`：运行日志、表格、图和冻结数字。
16
17 所有实验固定随机种子并记录参数。正式测试的案例编码、清除数量和运行时间只从模拟器日志补入，不预填。
18
19 问题 3 当前默认采用不含原点的正七边形极小极大连续覆盖，环半径与最坏最近距离均约 998.925 米；
  保留两条示向线发现策略、100 米交点试探、150 米扫描绕路阈值、多起点 2-opt 和 19.11 米覆盖半径的三角格兜底。
  本轮新增并默认开启：已发现频道的必无信号补测剪枝、保守可行域缓存、保证清除服务区域内的短路选点，
  以及预算感知退化和退出保护。未知频道的完整覆盖不变。推断无信号与真实测量分开记录；汇总记录完整配置、
  源码哈希和移动/检测/切频/清除耗时。

```

20
21 新布局先经 36 个场景筛选，再以 60 个新场景独立留出验证；相对原中心加七环点布局平均降时 1.5796%，95% 区间为
0.7879%-2.3717%。完整终版再在 `results/q3\_final\_policy\_holdout\_v2/` 的 50
个新场景上与本轮开始前源码端到端配对：逐场景相对时间降幅均值为 4.1550%，分层配对 Bootstrap 95% 区间为
2.7846%-5.3846%，42/50 局更快，三组策略均全部清除；平均每源时间由 310.642 秒降至 297.205 秒。
布局与完整策略的百分比来自不同对照，不能相加。全部数据为离线合成实验，不替代官方成绩。

22
23 有限步总时间调度已实现为可选实验 `--q3-joint-time-routing`，因筛选中的增益很小而真实计算明显变慢，默认关闭。
消融开关为 `--no-q3-skip-impossible-rechecks`、`--no-q3-service-region`，
关闭二者可保留本轮前的主要动作策略（缓存和资源保护仍保留）。运行目录必须是新目录，以免追加式事件日志混入旧记录。

24
25 Q3 材料以 `python build\_artifacts.py --q3-only` 更新；不传参数也只同步 Q3，避免覆盖其他题。
生成前核验当前配置、源码哈希与留出验证一致。当前图为 `artifacts/figures/fig\_q3\_current\_cover`、
`fig\_q3\_improvement`；方法、写作包、镜像目录、支撑代码和 Q3 冻结记录由同一入口生成。旧综合图中的 Q3
和旧离线性能图不应再用于当前版论文。`--q1-validation-only` 保留原 Q1 流程；历史全量重建仅在显式
`--legacy-full-build` 下运行，不建议用于当前提交。

26
27 所有新增实验通过 `benchmark\_q3\_improvements.py` 复现，基线必须提供本轮前的独立源码目录；
没有启动在线演练或正式测试，也不改写正式成绩记录。

28
29 问题 4 默认采用通过连续覆盖充分性检验的 21 点正向包围布局和三条示向线发现策略：原点、半径 996 米的 8
点内环及包含目标圆域的 12 点外环。定位阶段使用 10% 信息量下限的近路候选；当定位域半径不超过 100 米时，
先用示向线交点尝试快速清除；扫描途中只插入增量路程不超过 200 米的清除点。失败后仍保留测向、阴影恢复、
确定性定位域和 19.11 米覆盖半径的三角格兜底。旧 25 点布局可用 `--q4-layout compact25` 做回归对照。

30
31 当前默认还启用三个不改变保证层的细化：联合正负观测只删除严格不可能的位置单元，
有限源假设只用于按预计虚拟时间重排测点，保证清除域内选择进出路程更短且经全部顶点复核的服务点。当前源码的 55
局分层重放与另一组 24 局共享误差压力集均保持 100% 离线合成清除，配对平均总时间降幅分别为 1.157% 和 1.126%。
消融结果保存在 `results/q4\_ablation\_current\_v2/`，冻结材料由 `python build\_q4\_artifacts.py` 生成。
上述数字均为离线算法回归，不是官方成绩；现存官方演练只覆盖 21 点核心策略，最终三项细化仍需官方演练锚定。